

THE UNIVERSITY OF MELBOURNE

COMP90038 Algorithms and Complexity

Lecture 11: Sorting with Divide-and-Conquer

Olya Ohrimenko
(with thanks to Harald Søndergaard)

Divide and Conquer



THE UNIVERSITY OF MELBOURNE

- We earlier studied recursion as a powerful problem solving technique.

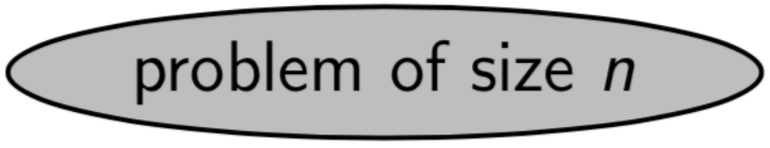
- The **divide-and-conquer** strategy tries to make the most of this idea:

  1. Divide the given problem instance into smaller instances.
  2. Solve the smaller instances recursively.
  3. Combine the smaller solutions to solve the original instance.

- This works best when the smaller instances can be made to be of equal (or near-equal) size.

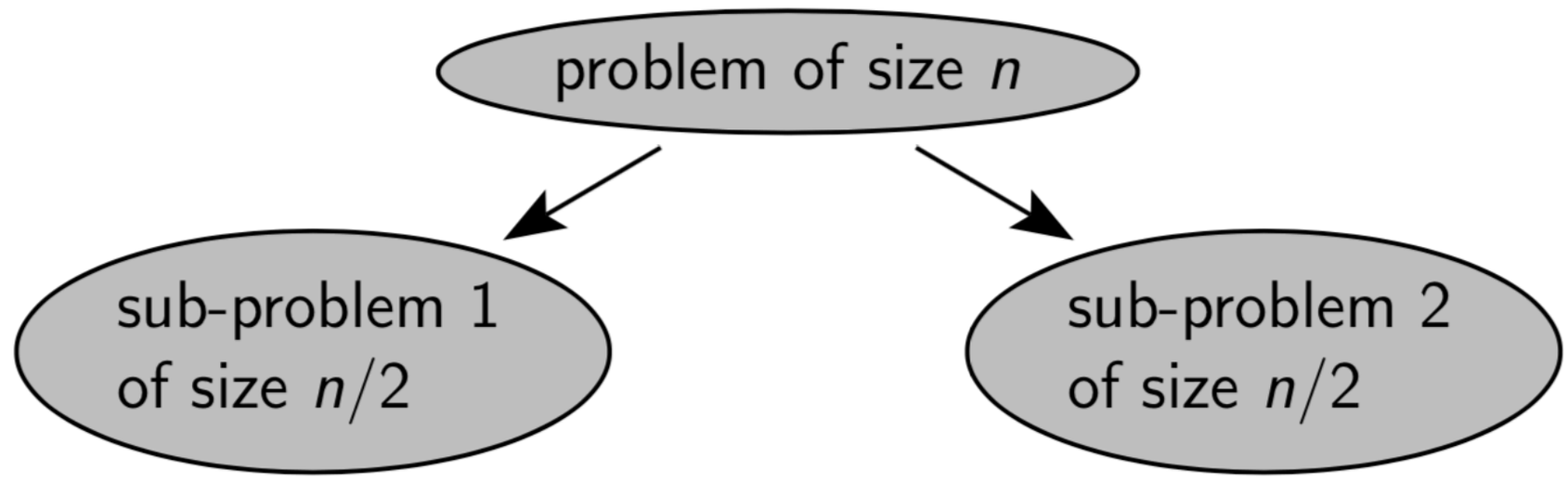
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Split-Solve-and-Join Approach



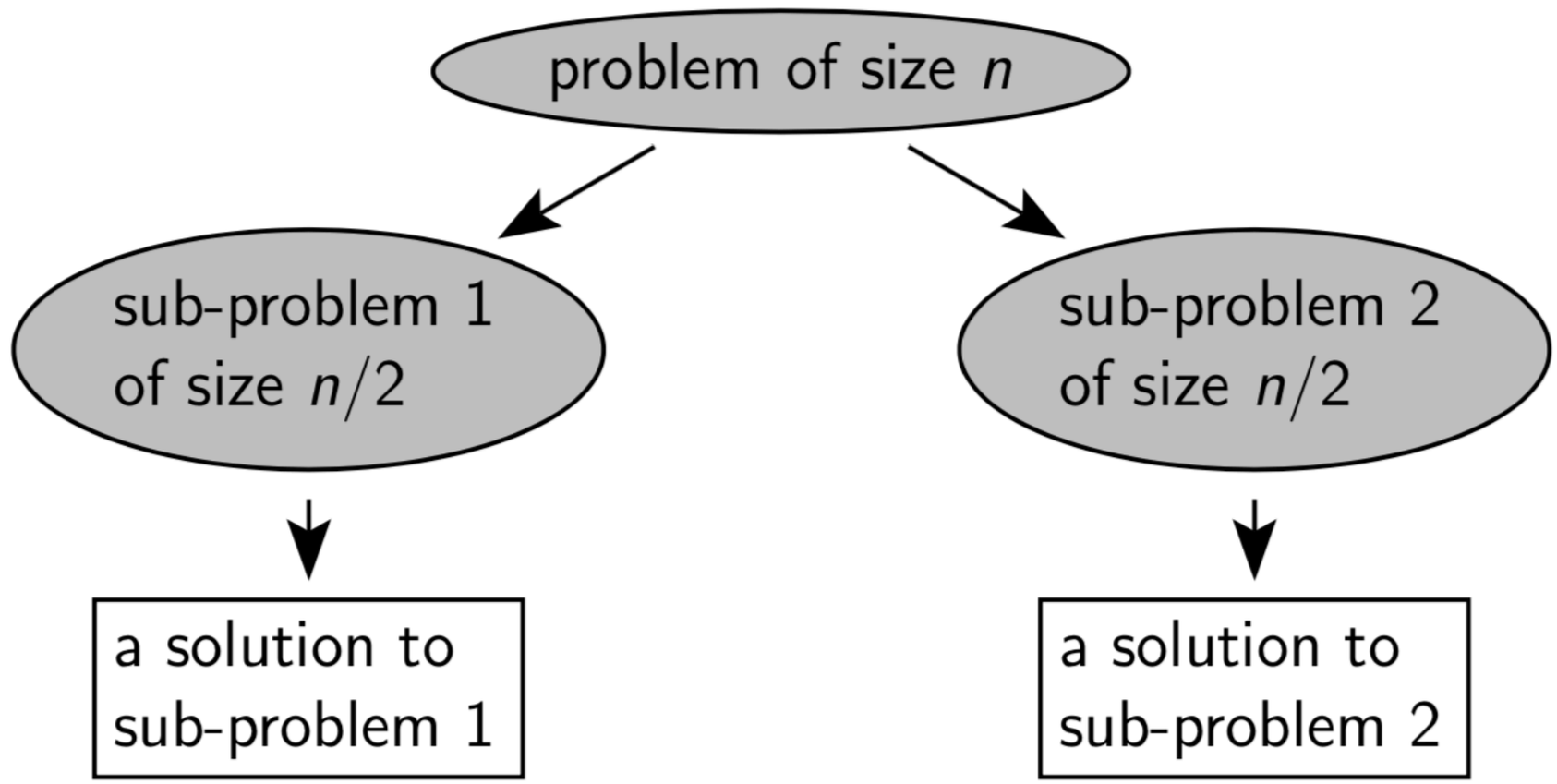
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Split-Solve-and-Join Approach



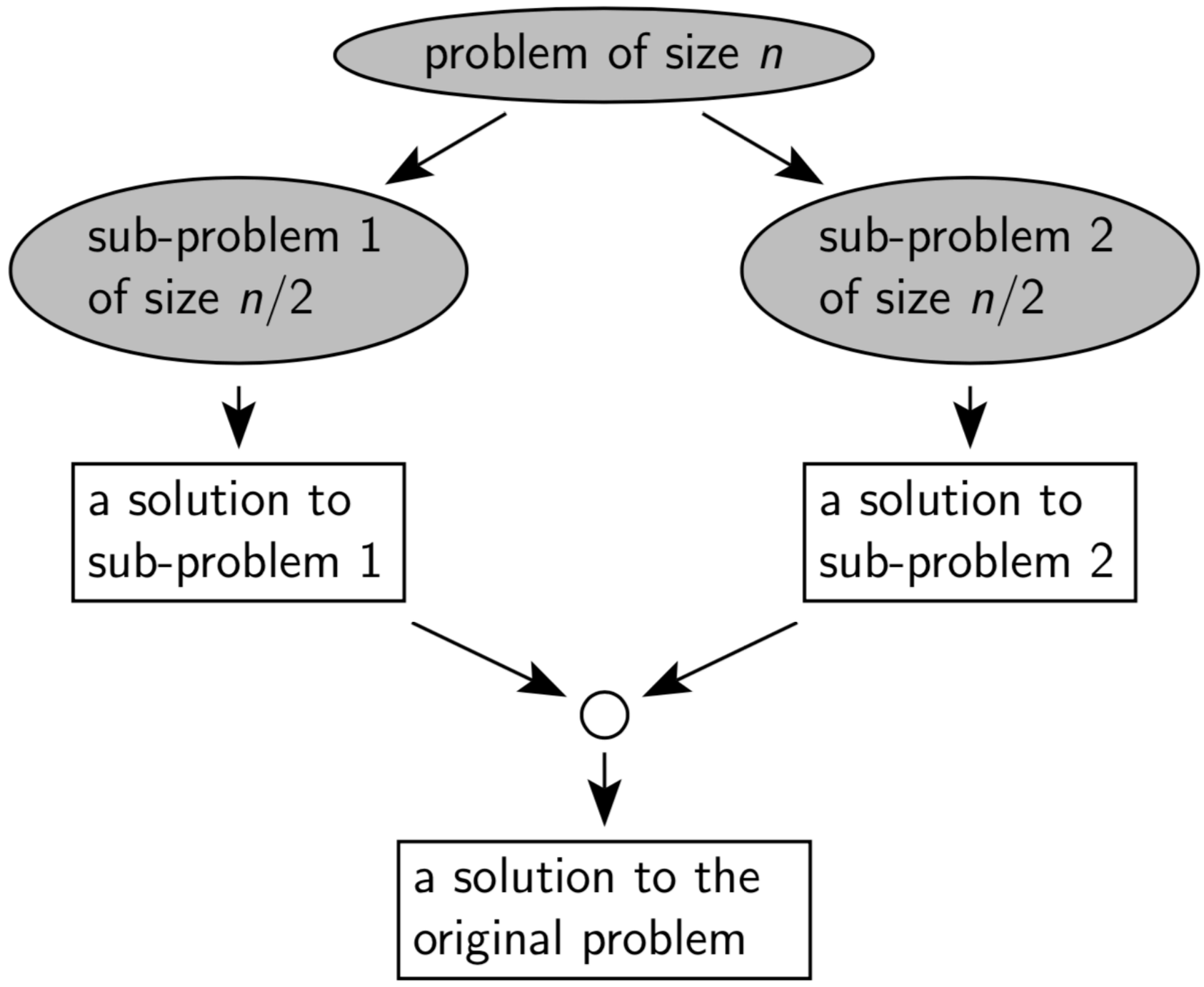
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Split-Solve-and-Join Approach



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Split-Solve-and-Join Approach



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Divide and Conquer Algorithms

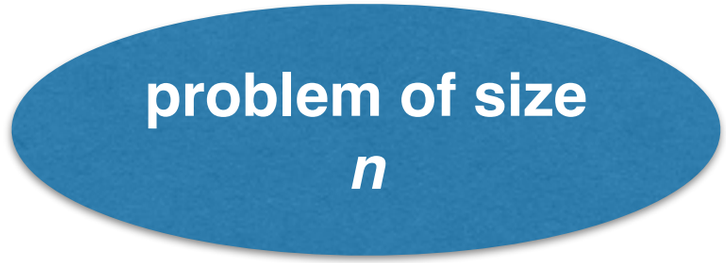


THE UNIVERSITY OF MELBOURNE

- We will discuss:
  - The Master Theorem
  - Mergesort
  - Quicksort
  - Tree traversal
  - Closest Pair revisited

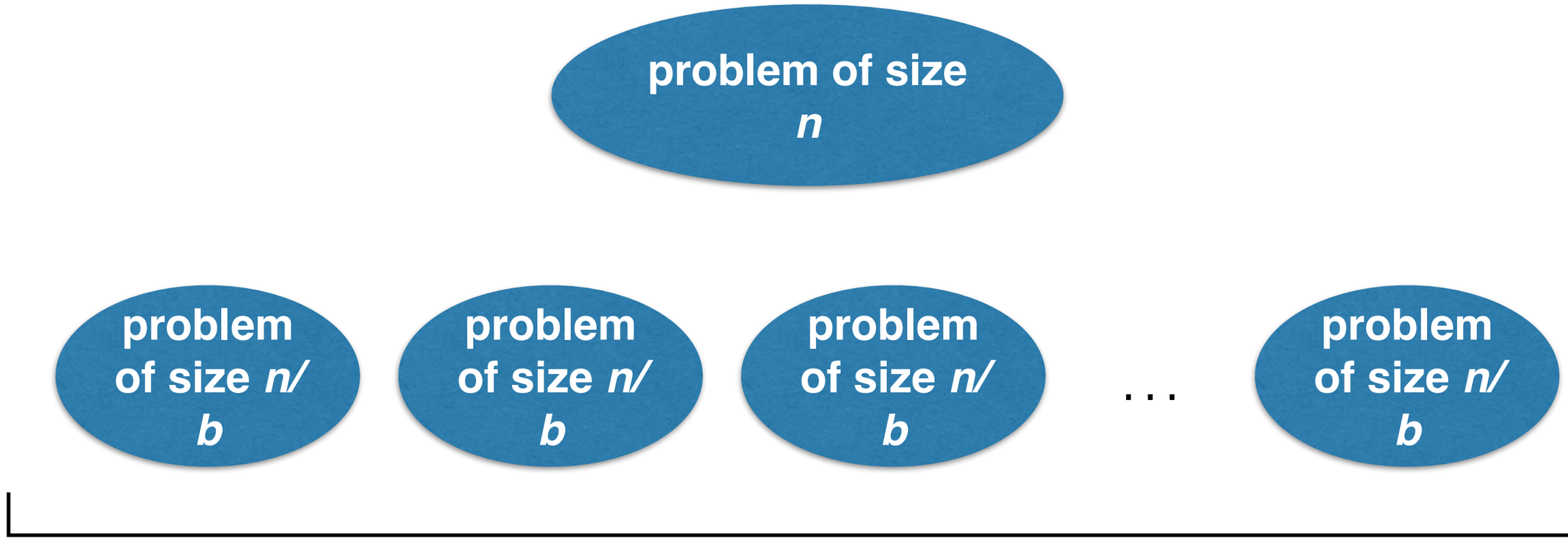
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Divide-and-Conqer
General Case



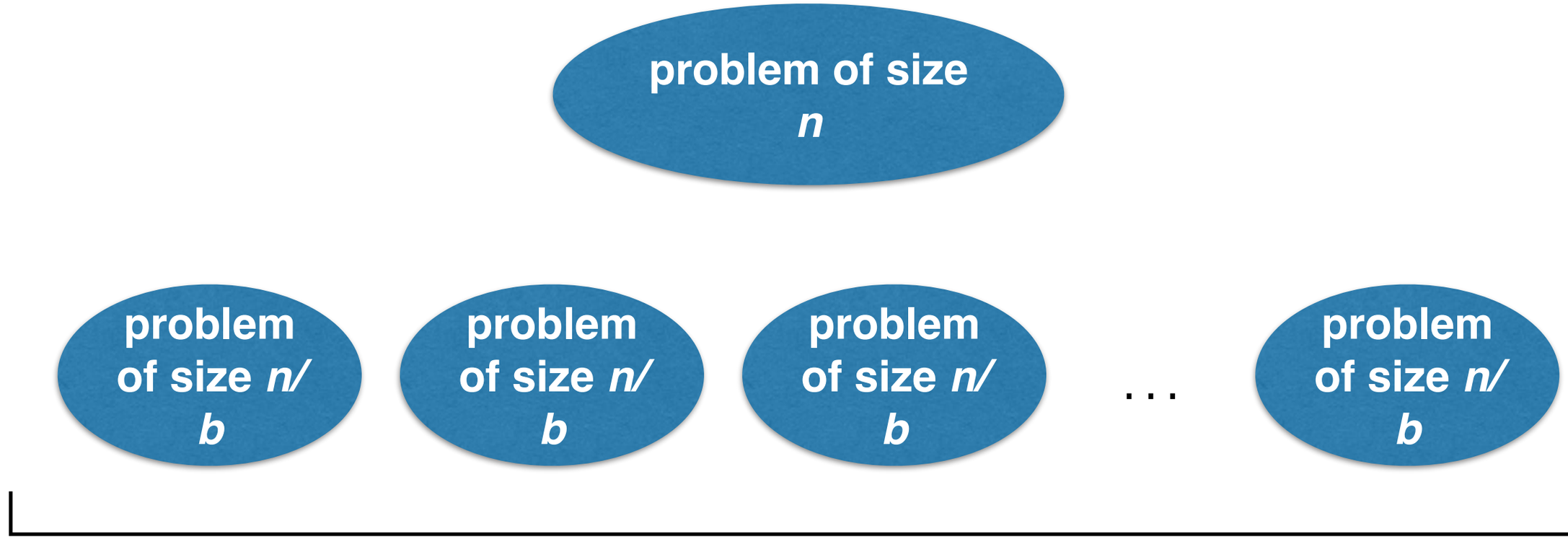
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Divide-and-Conqer General Case



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

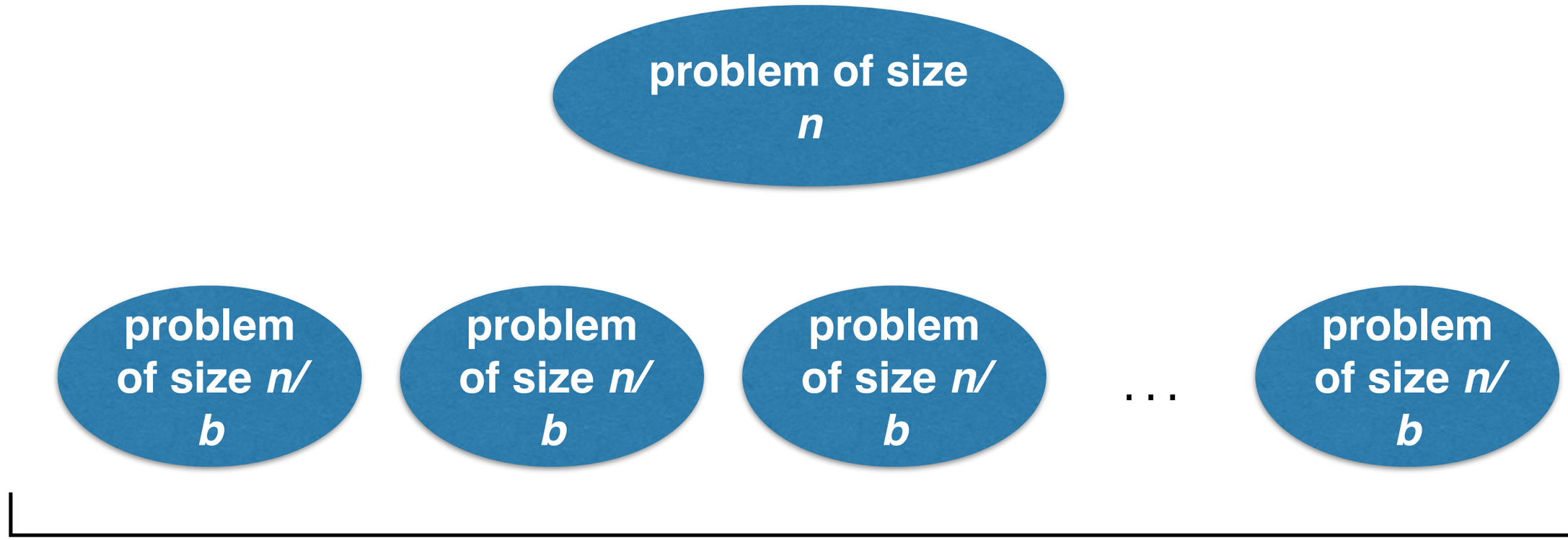
Divide-and-Conqer General Case



only ***a*** sub-problems need to be solved

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Divide-and-Conqer General Case



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Divide-and-Conquer Recurrences

- What is the time required to solve a problem of size ***n*** by divide-and-conquer?

$$T(n) = aT(n/b) + f(n)$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Divide-and-Conquer Recurrences

- What is the time required to solve a problem of size ***n*** by divide-and-conquer?

- For the general case, assume we split the problem into ***b*** instances (each of size ***n/b***), of which ***a*** need to be solved:

$$T(n) = aT(n/b) + f(n)$$

where $f(n)$ expresses the time spent on dividing a problem into ***b*** sub-problems and combining the ***a*** results.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Divide-and-Conquer Recurrences

- What is the time required to solve a problem of size ***n*** by divide-and-conquer?

- For the general case, assume we split the problem into ***b*** instances (each of size ***n/b***), of which ***a*** need to be solved:

$$T(n) = aT(n/b) + f(n)$$

  where $f(n)$ expresses the time spent on dividing a problem into ***b*** sub-problems and combining the ***a*** results.

- (A very common case is $T(n) = 2T(n/2) + n$.)

- How to find closed forms for these recurrences?

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

The Master Theorem



THE UNIVERSITY OF MELBOURNE

- For integer constants $a \geq 1$ and $b > 1$, and function f with $f(n) \in \Theta(n^d)$, $d \geq 0$, the recurrence

$$T(n) = aT(n/b) + f(n)$$

(with T(1) = c) has solutions, and

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

The Master Theorem

- For integer constants $a \geq 1$ and $b > 1$, and function f with $f(n) \in \Theta(n^d)$, $d \geq 0$, the recurrence

$$T(n) = aT(n/b) + f(n)$$

(with T(1) = c) has solutions, and

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \end{cases}$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

The Master Theorem

- For integer constants $a \geq 1$ and $b > 1$, and function f with $f(n) \in \Theta(n^d)$, $d \geq 0$, the recurrence

$$T(n) = aT(n/b) + f(n)$$

(with T(1) = c) has solutions, and

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \end{cases}$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

The Master Theorem

- For integer constants $a \geq 1$ and $b > 1$, and function f with $f(n) \in \Theta(n^d)$, $d \geq 0$, the recurrence

$$T(n) = aT(n/b) + f(n)$$

(with T(1) = c) has solutions, and

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

The Master Theorem

- For integer constants $a \geq 1$ and $b > 1$, and function f with $f(n) \in \Theta(n^d)$, $d \geq 0$, the recurrence

$$T(n) = aT(n/b) + f(n)$$

(with T(1) = c) has solutions, and

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

- Note that we also allow a to be greater than b.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

The Master Theorem

- For integer constants $a \geq 1$ and $b > 1$, and function f with $f(n) \in \Theta(n^d)$, $d \geq 0$, the recurrence

$$T(n) = aT(n/b) + f(n)$$

(with T(1) = c) has solutions, and

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

- Note that we also allow a to be greater than b.
- (A proof is in Levitin's Appendix B.)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 0



THE UNIVERSITY OF MELBOURNE

$$T(n) = T(n/2) + 1$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 0



THE UNIVERSITY OF MELBOURNE

$$T(n) = T(n/2) + 1$$

$$T(n) = aT(n/b) + f(n)$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 0



THE UNIVERSITY OF MELBOURNE

$$T(n) = T(n/2) + 1$$

$$T(n) = aT(n/b) + f(n)$$

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 0



THE UNIVERSITY OF MELBOURNE

$T(n) = T(n/2) + 1$

a = 1, b = 2, d = 0

$$T(n) = aT(n/b) + f(n)$$

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 0



THE UNIVERSITY OF MELBOURNE

$T(n) = T(n/2) + 1$

a = 1, b = 2, d = 0

$$T(n) = aT(n/b) + f(n)$$

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

So, by the Master Theorem, $T(n) \in \Theta(\log n)$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 1



THE UNIVERSITY OF MELBOURNE

$$T(n) = 2T(n/2) + n$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 1



THE UNIVERSITY OF MELBOURNE

$$T(n) = 2T(n/2) + n$$

$$T(n) = aT(n/b) + f(n)$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 1



THE UNIVERSITY OF MELBOURNE

$$T(n) = 2T(n/2) + n$$

$$T(n) = aT(n/b) + f(n)$$

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 1



THE UNIVERSITY OF MELBOURNE

$T(n) = 2T(n/2) + n$

a = 2, b = 2, d = 1

$$T(n) = aT(n/b) + f(n)$$

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 1



THE UNIVERSITY OF MELBOURNE

$T(n) = 2T(n/2) + n$ a = 2, b = 2, d = 1

$$T(n) = aT(n/b) + f(n)$$

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

So, by the Master Theorem, $T(n) \in \Theta(n \log n)$

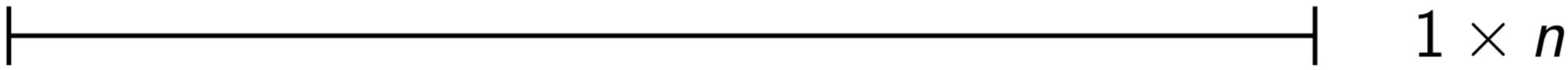
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 1



THE UNIVERSITY OF MELBOURNE

$$T(n) = 2T(n/2) + n$$



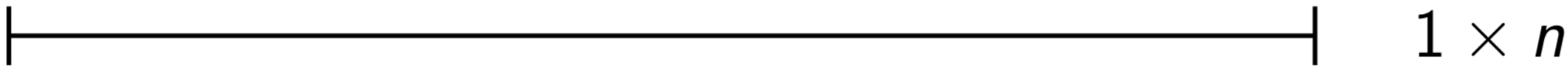
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 1

THE UNIVERSITY OF MELBOURNE

$$T(n) = 2T(n/2) + n$$

$$T(n) = 2(2T(n/4) + (n/2)) + n$$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

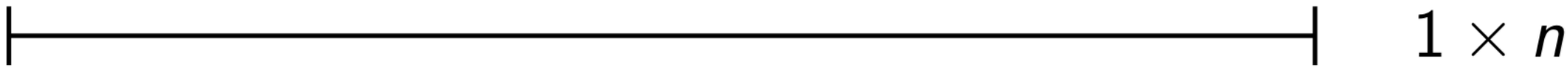
Master Theorem: Example 1



THE UNIVERSITY OF MELBOURNE

$$T(n) = 2T(n/2) + n$$

$$T(n) = 4T(n/4) + 2(n/2) + n$$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

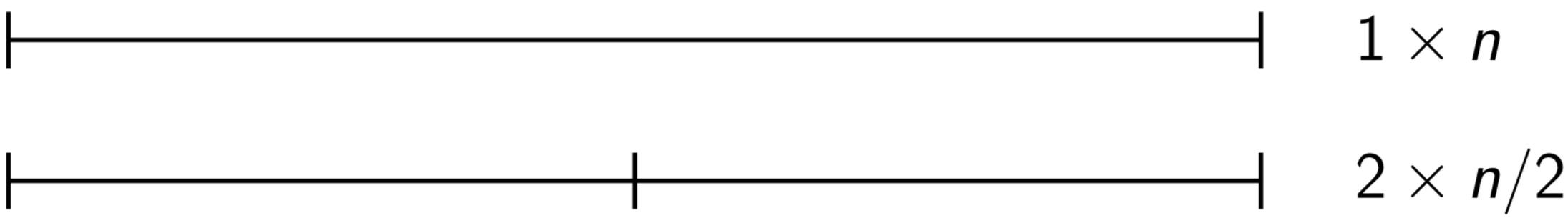
Master Theorem: Example 1



THE UNIVERSITY OF MELBOURNE

$$T(n) = 2T(n/2) + n$$

$$T(n) = 4T(n/4) + 2(n/2) + n$$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

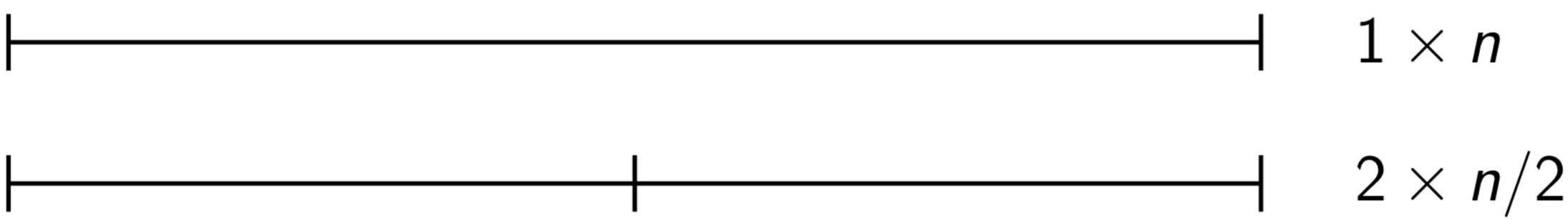
Master Theorem: Example 1



THE UNIVERSITY OF MELBOURNE

$$T(n) = 2T(n/2) + n$$

$$T(n) = 4(2T(n/8) + n/4) + 2(n/2) + n$$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

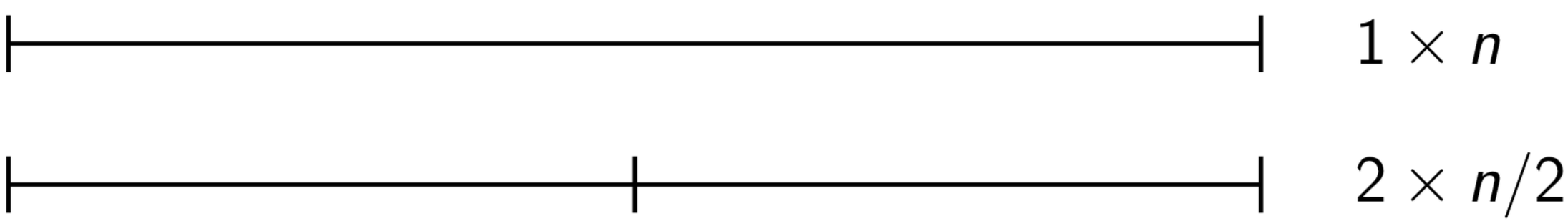
Master Theorem: Example 1



THE UNIVERSITY OF MELBOURNE

$$T(n) = 2T(n/2) + n$$

$$T(n) = 8T(n/8) + 4(n/4) + 2(n/2) + n$$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

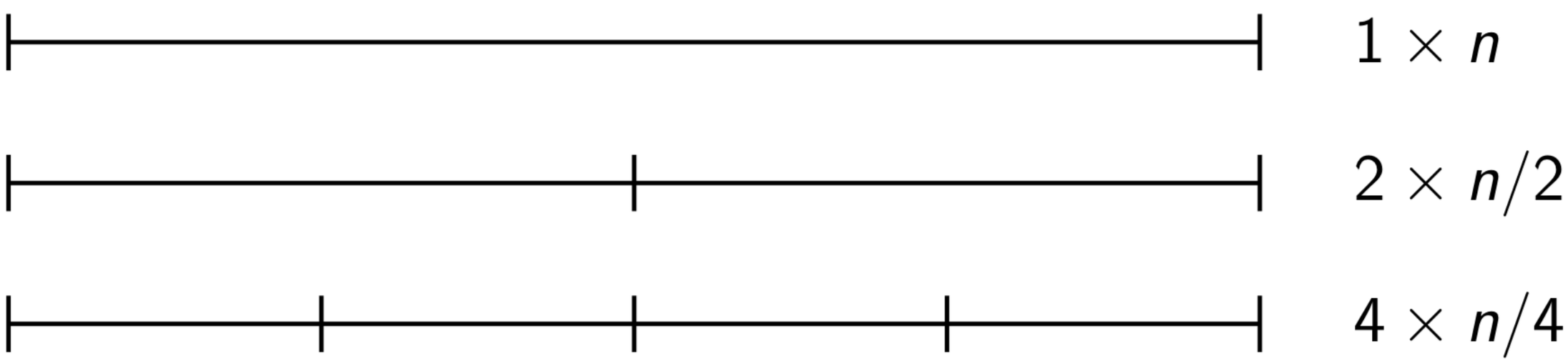
Master Theorem: Example 1



THE UNIVERSITY OF MELBOURNE

$$T(n) = 2T(n/2) + n$$

$$T(n) = 8T(n/8) + 4(n/4) + 2(n/2) + n$$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 1



THE UNIVERSITY OF MELBOURNE

$$T(n) = 2T(n/2) + n$$

$$T(n) = 8(2T(n/16) + n/8) + 4(n/4) + 2(n/2) + n$$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

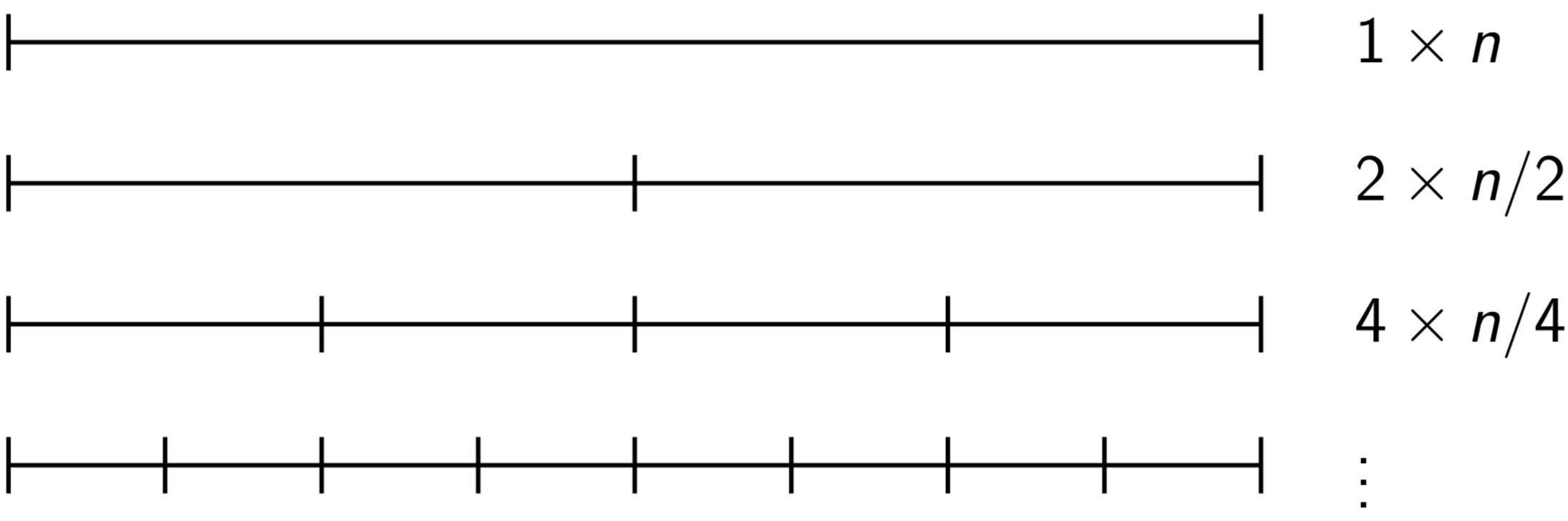
Master Theorem: Example 1



THE UNIVERSITY OF MELBOURNE

$$T(n) = 2T(n/2) + n$$

$$T(n) = 16T(n/16) + 8(n/8) + 4(n/4) + 2(n/2) + n$$



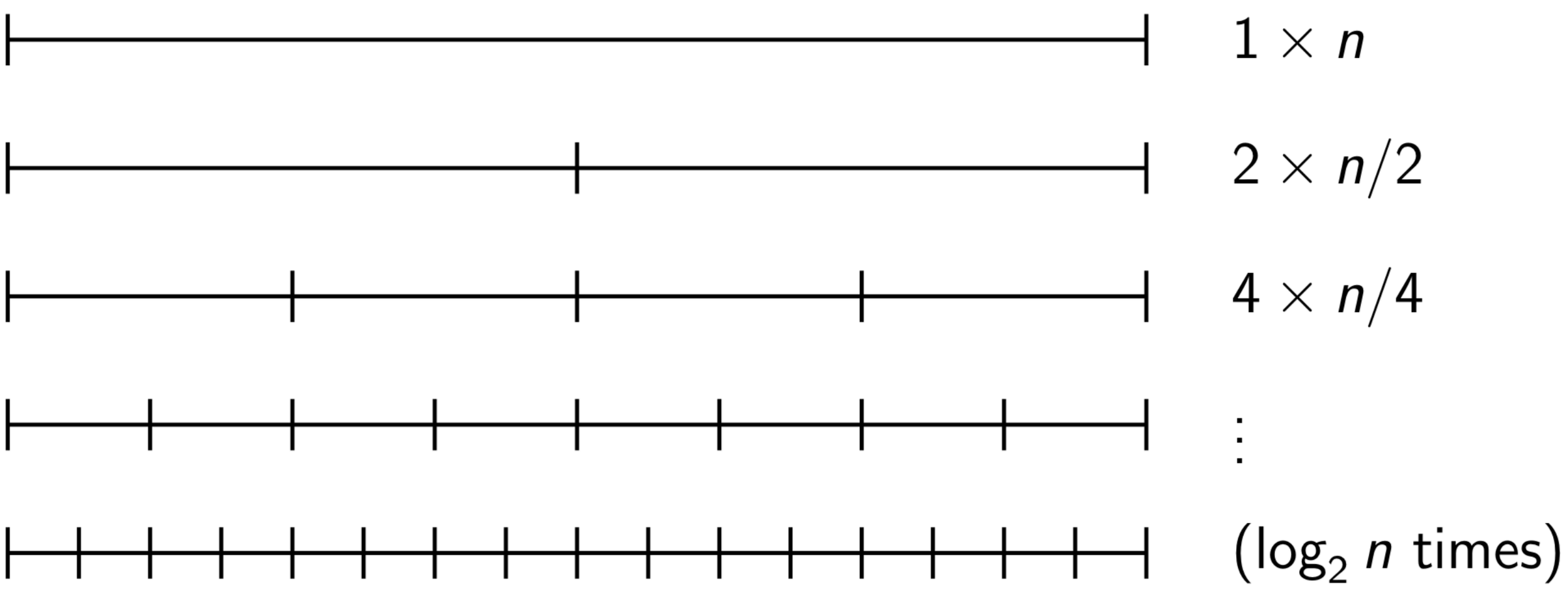
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 1



THE UNIVERSITY OF MELBOURNE

$$T(n) = 2T(n/2) + n$$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

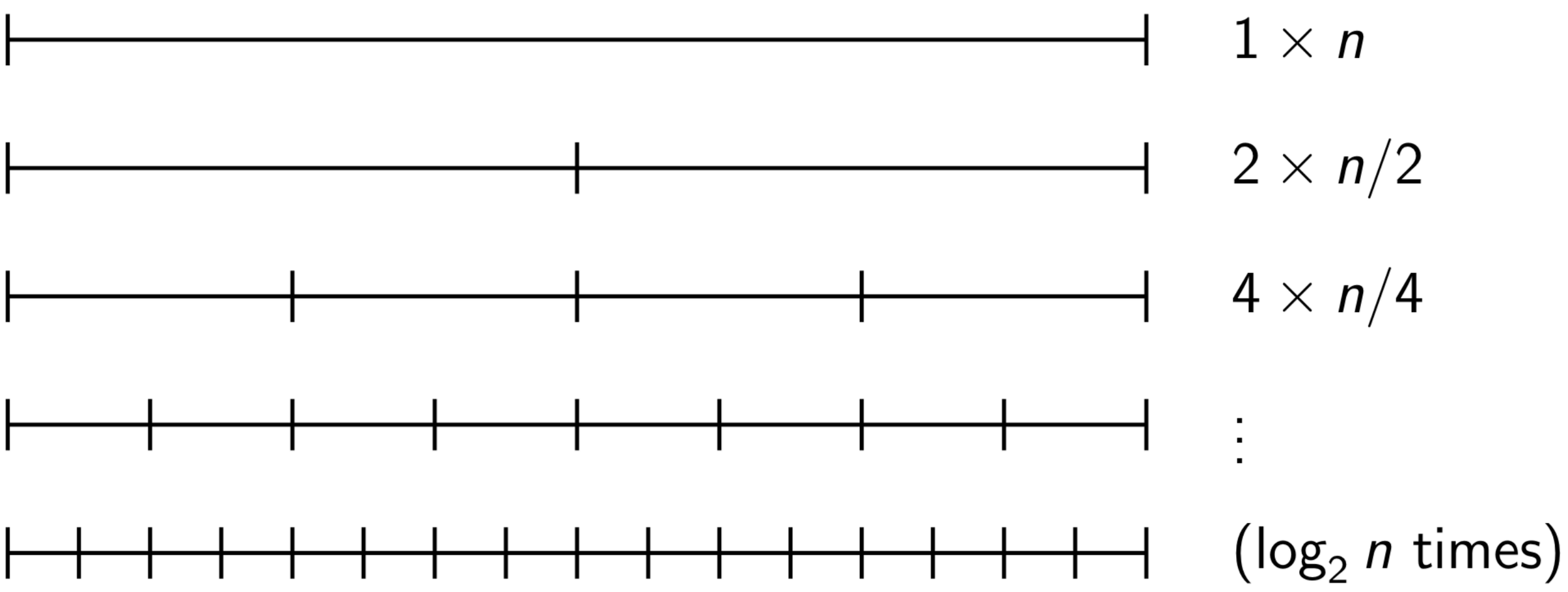
Master Theorem: Example 1



THE UNIVERSITY OF MELBOURNE

$T(n) = 2T(n/2) + n$

$T(n) \in \Theta(n \log n)$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 2



THE UNIVERSITY OF MELBOURNE

$$T(n) = 4T(n/4) + n$$

$$T(n) = aT(n/b) + f(n)$$

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 2



THE UNIVERSITY OF MELBOURNE

$$T(n) = 4T(n/4) + n$$

a = 4, b = 4, d = 1

$$T(n) = aT(n/b) + f(n)$$

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 2



THE UNIVERSITY OF MELBOURNE

$$T(n) = 4T(n/4) + n$$

a = 4, b = 4, d = 1

$$T(n) = aT(n/b) + f(n)$$

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

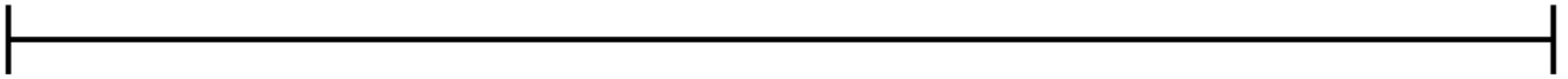
So, by the Master Theorem, $T(n) \in \Theta(n \log n)$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 2

THE UNIVERSITY OF MELBOURNE

$$T(n) = 4T(n/4) + n$$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

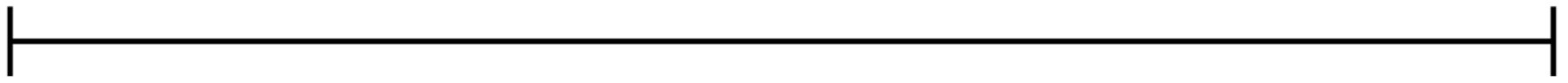
Master Theorem: Example 2



THE UNIVERSITY OF MELBOURNE

$$T(n) = 4T(n/4) + n$$

$$T(n) = 4(4T(n/16) + (n/4)) + n$$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

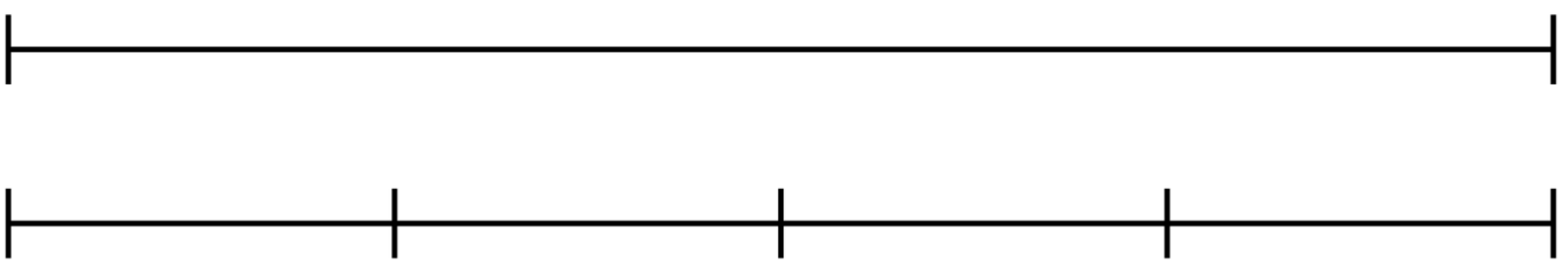
Master Theorem: Example 2



THE UNIVERSITY OF MELBOURNE

$$T(n) = 4T(n/4) + n$$

$$T(n) = 16T(n/16) + 4(n/4) + n$$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

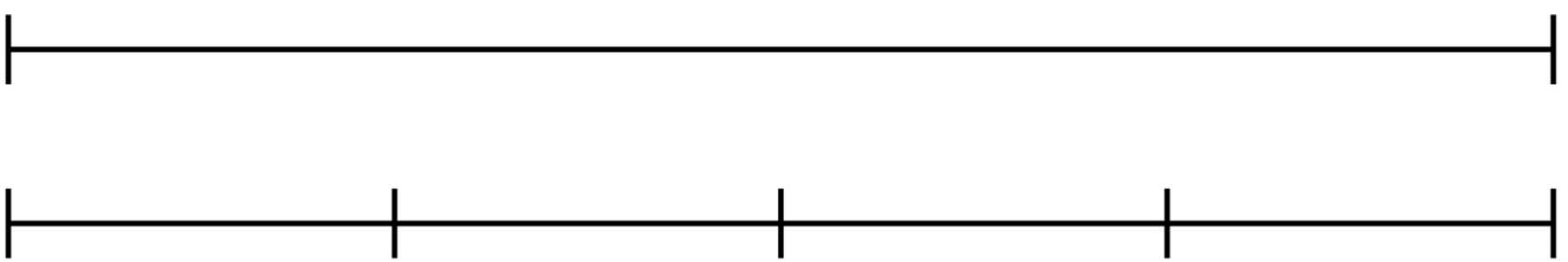
Master Theorem: Example 2



THE UNIVERSITY OF MELBOURNE

$$T(n) = 4T(n/4) + n$$

$$T(n) = 16T(n/16) + 4(n/4) + n$$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

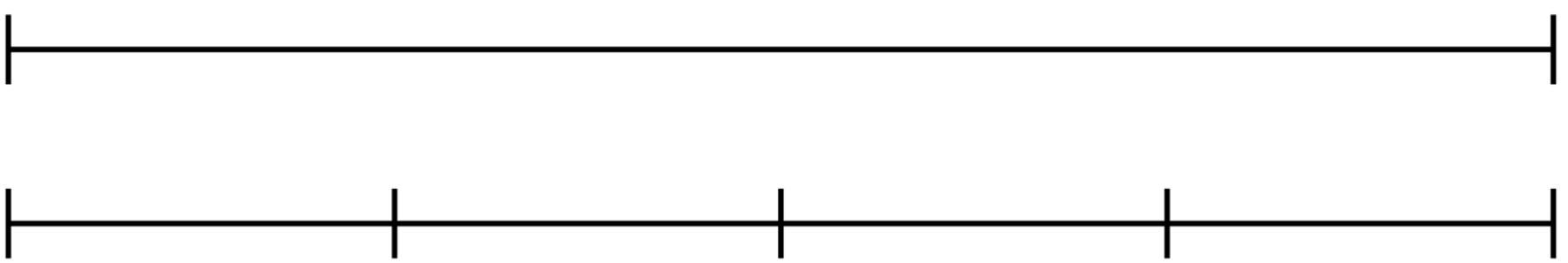
Master Theorem: Example 2



THE UNIVERSITY OF MELBOURNE

$$T(n) = 4T(n/4) + n$$

$$T(n) = 16(4T(n/64) + n/16)\ 4(n/4) + n$$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

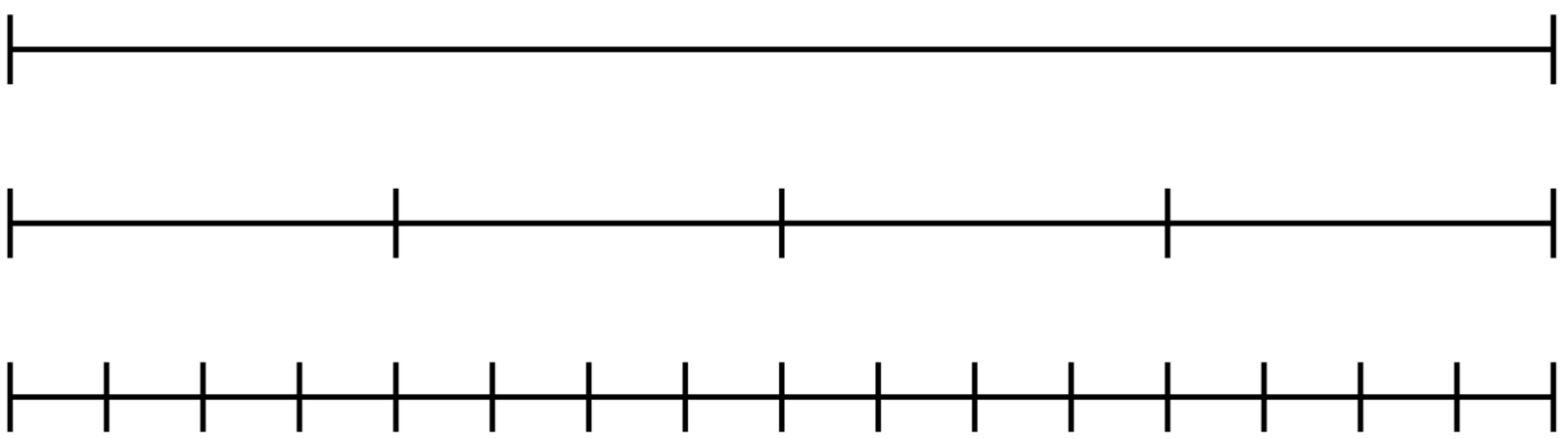
Master Theorem: Example 2



THE UNIVERSITY OF MELBOURNE

$$T(n) = 4T(n/4) + n$$

$$T(n) = 64T(n/64) + 16(n/16) + 4(n/4) + n$$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

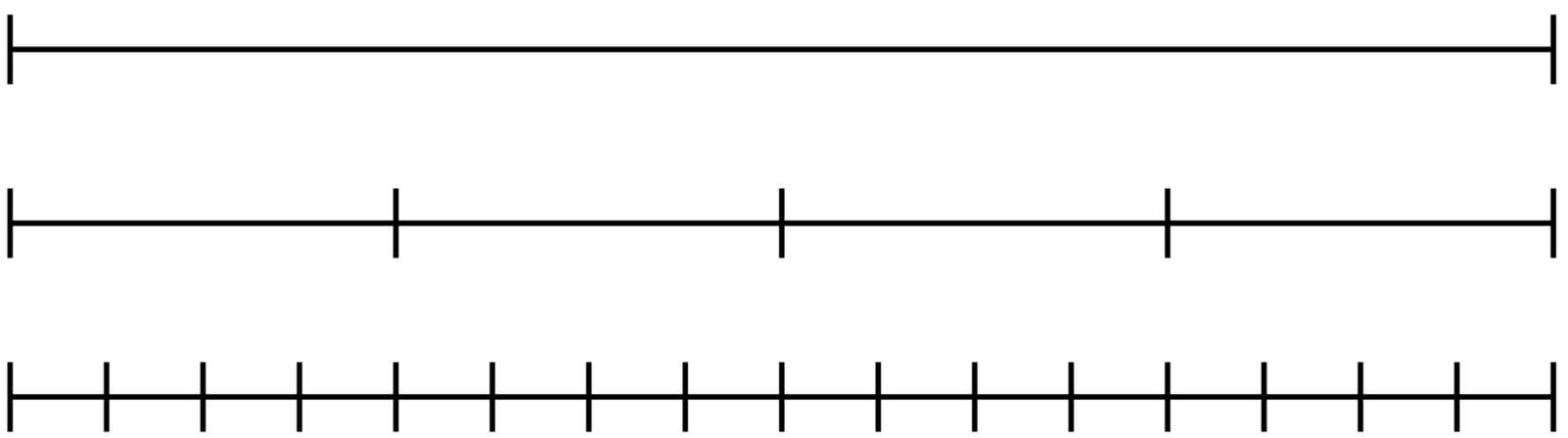
Master Theorem: Example 2



THE UNIVERSITY OF MELBOURNE

$T(n) = 4T(n/4) + n$

$T(n) = 64T(n/64) + 16(n/16) + 4(n/4) + n$



⋮

($\log_4 n$ times)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

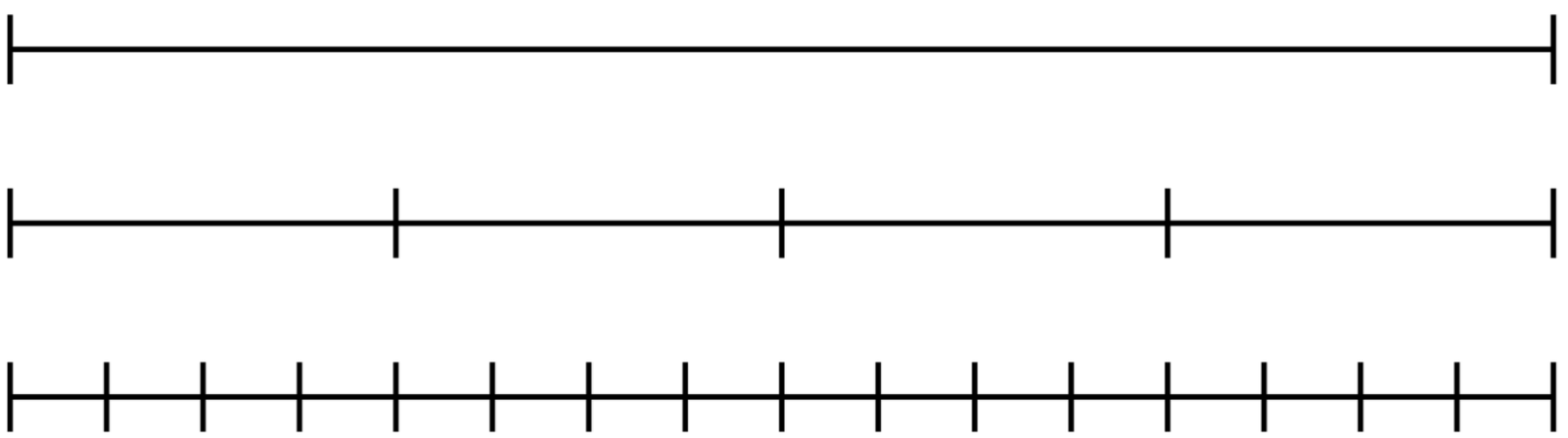
Master Theorem: Example 2



THE UNIVERSITY OF MELBOURNE

$T(n) = 4T(n/4) + n$

$T(n) = 64T(n/64) + 16(n/16) + 4(n/4) + n$



⋮

($\log_4 n$ times)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 2



THE UNIVERSITY OF MELBOURNE

$$T(n) = 4T(n/4) + n$$

$$T(n) = 64T(n/64) + 16(n/16) + 4(n/4) + n$$

$$T(n) \in \Theta(n \log n)$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 3



THE UNIVERSITY OF MELBOURNE

$$T(n) = T(n/2) + n$$

$$T(n) = aT(n/b) + f(n)$$

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 3



THE UNIVERSITY OF MELBOURNE

$$T(n) = T(n/2) + n$$

a = 1, b = 2, d = 1

$$T(n) = aT(n/b) + f(n)$$

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 3



THE UNIVERSITY OF MELBOURNE

$T(n) = T(n/2) + n$

a = 1, b = 2, d = 1

$$T(n) = aT(n/b) + f(n)$$

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

So, by the Master Theorem, $T(n) \in \Theta(n)$

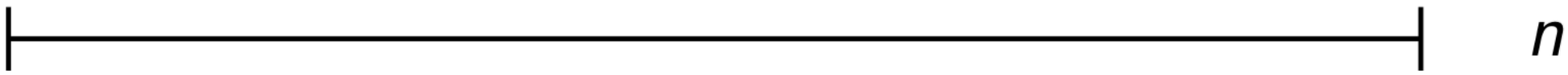
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 3



THE UNIVERSITY OF MELBOURNE

$T(n) = T(n/2) + n$ a = 1, b = 2, d = 1



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

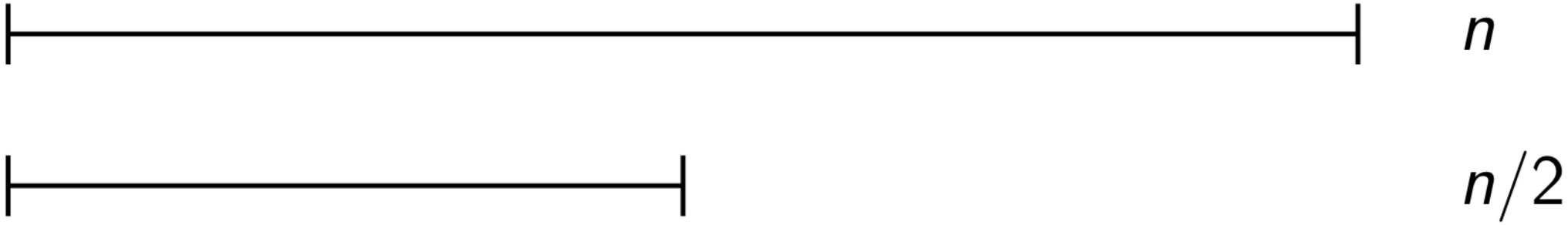
Master Theorem: Example 3



THE UNIVERSITY OF MELBOURNE

$T(n) = T(n/2) + n$ a = 1, b = 2, d = 1

$T(n) = T(n/4) + n/2 + n$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

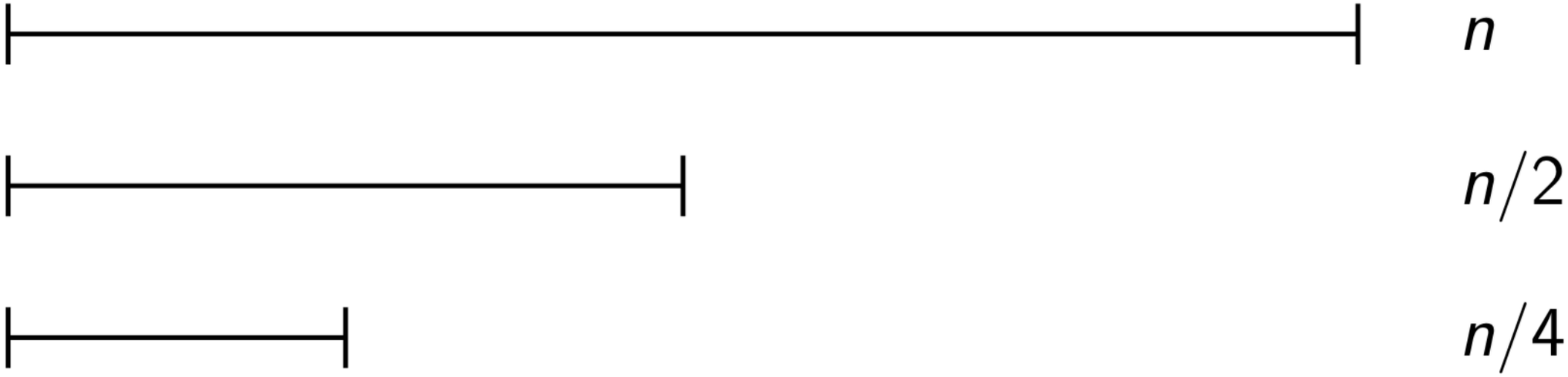
Master Theorem: Example 3



THE UNIVERSITY OF MELBOURNE

$T(n) = T(n/2) + n$ a = 1, b = 2, d = 1

$T(n) = T(n/8) + n/4 + n/2 + n$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

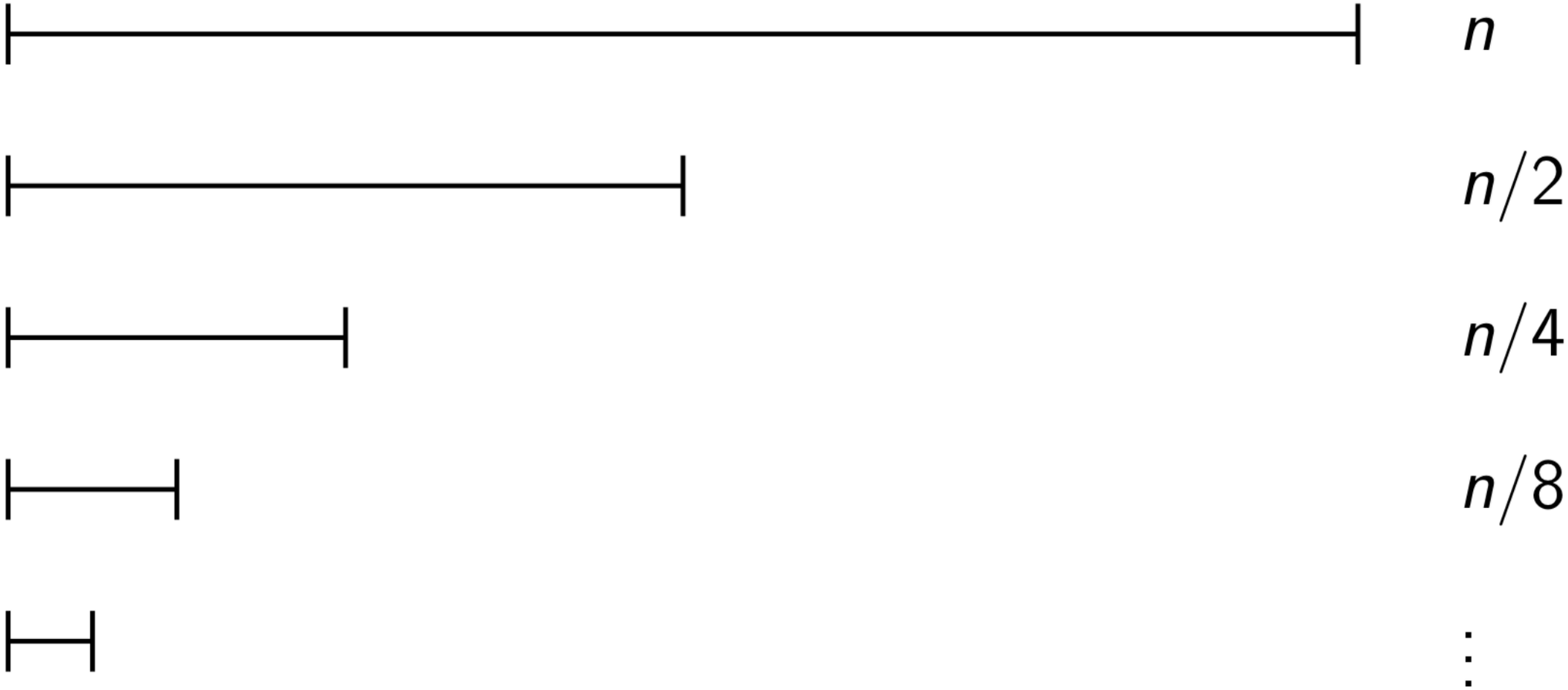
Master Theorem: Example 3



THE UNIVERSITY OF MELBOURNE

$T(n) = T(n/2) + n$ a = 1, b = 2, d = 1

$T(n) = T(n/8) + n/4 + n/2 + n$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 3



THE UNIVERSITY OF MELBOURNE

$T(n) = T(n/2) + n$ a = 1, b = 2, d = 1

$T(n) = T(n/8) + n/4 + n/2 + n$

$T(n) \in \Theta(n)$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort Intuition

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

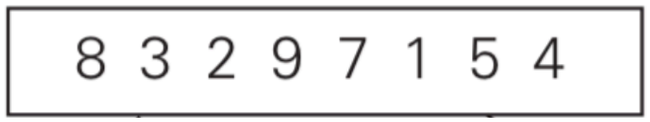
Mergesort

- Perhaps the most obvious application of divide-and-conquer:

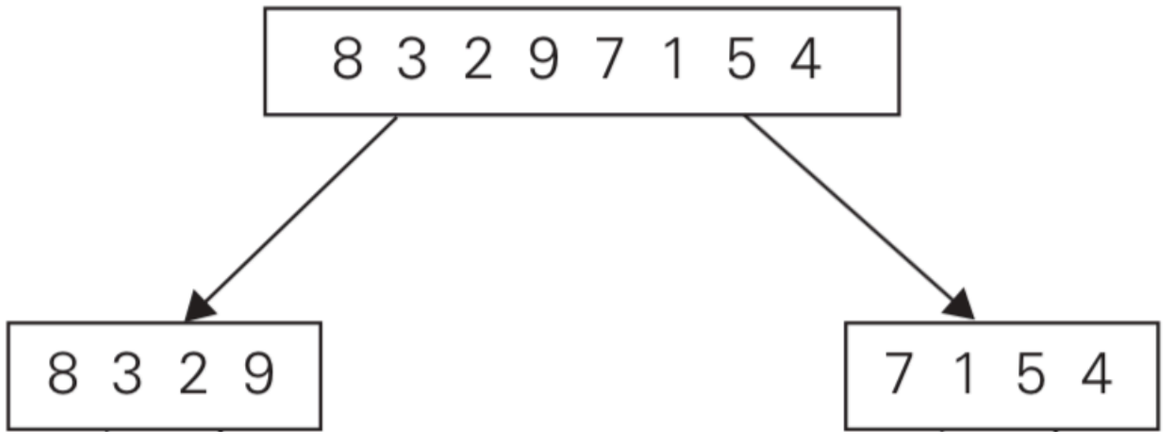
- To sort an array (or a list), cut it into two halves, sort each half, and merge the two results.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

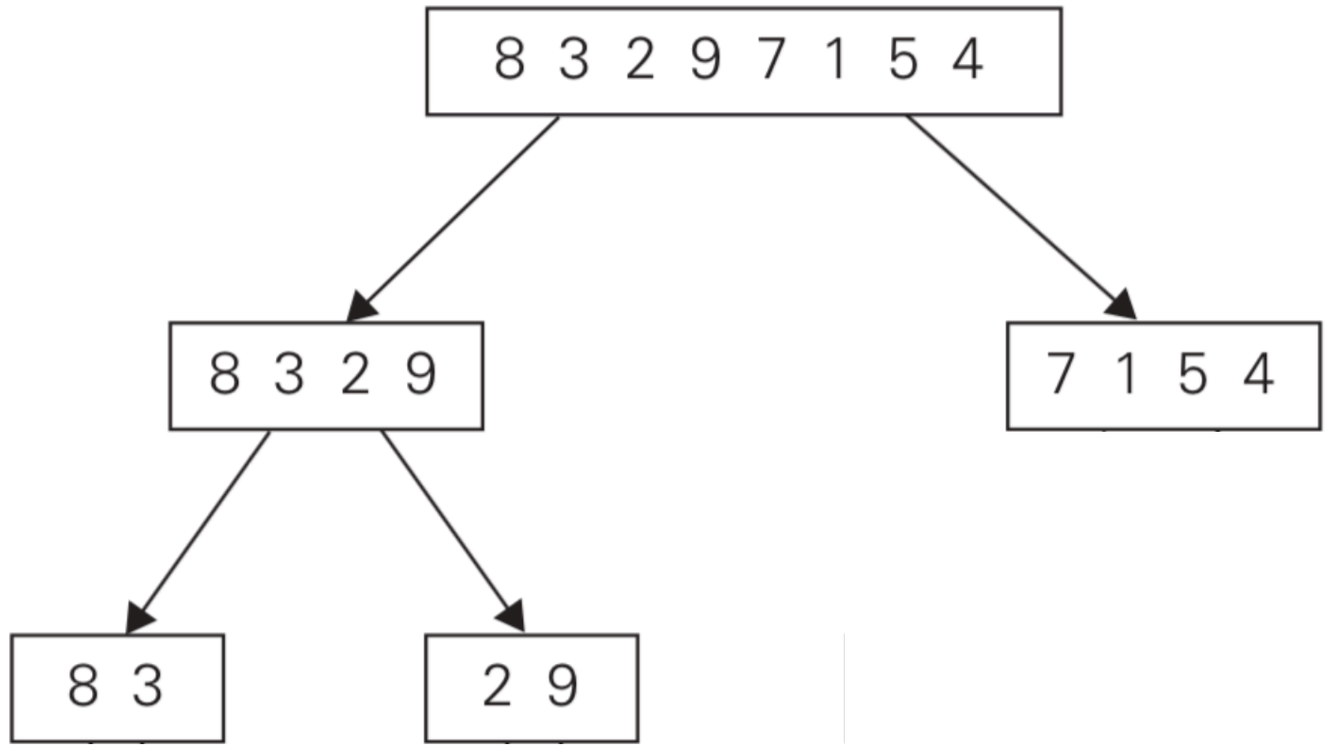
Mergesort



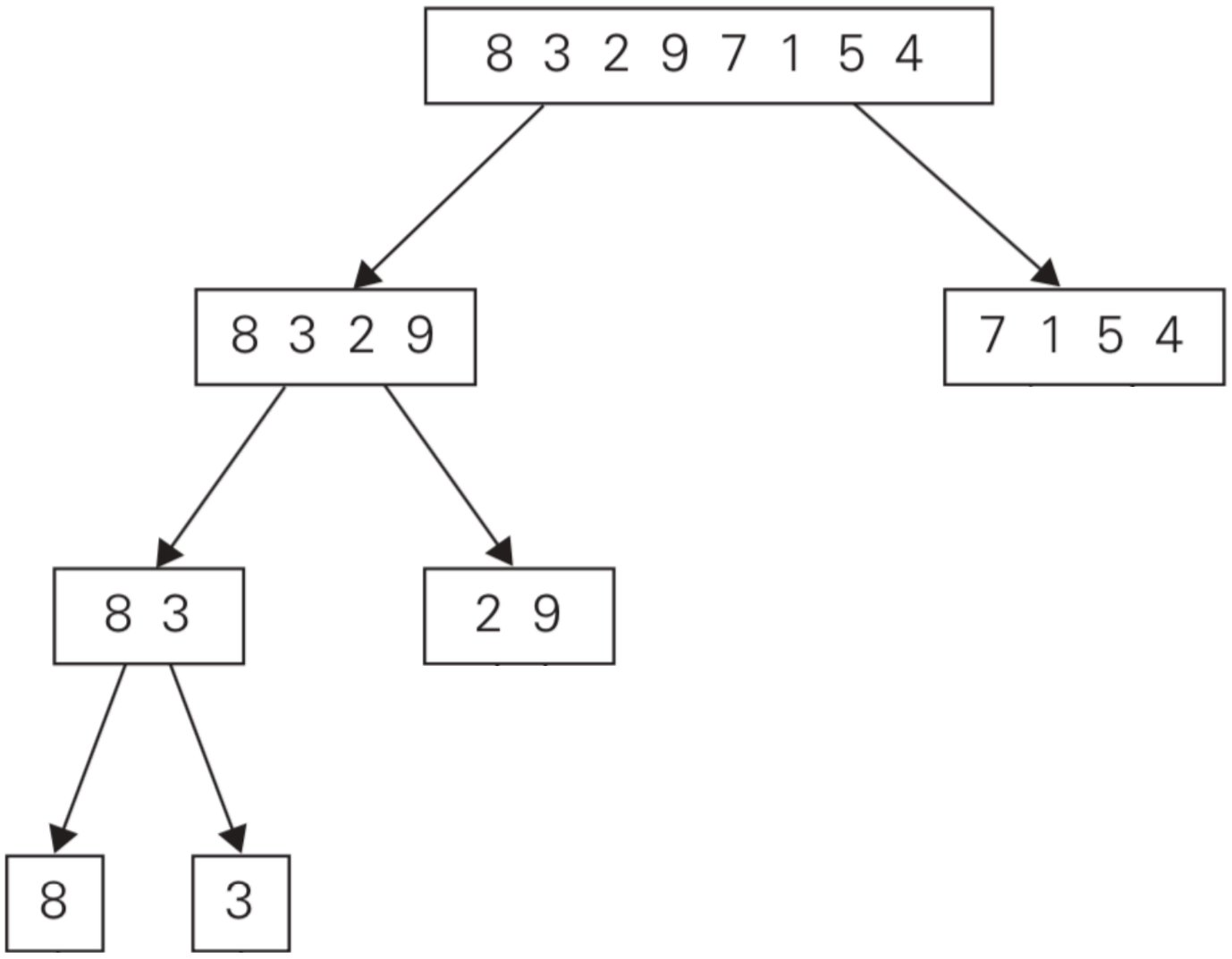
Mergesort



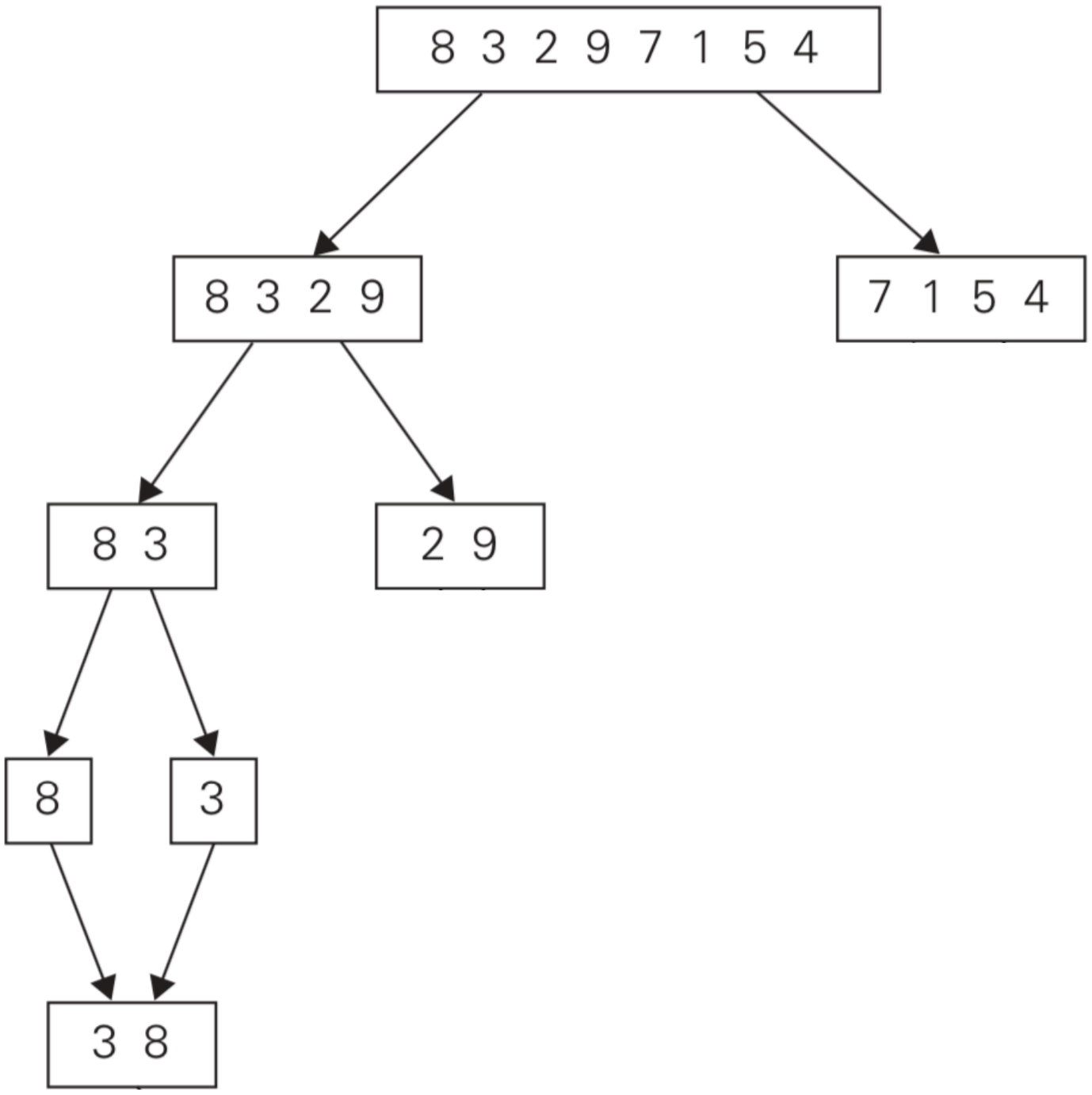
Mergesort



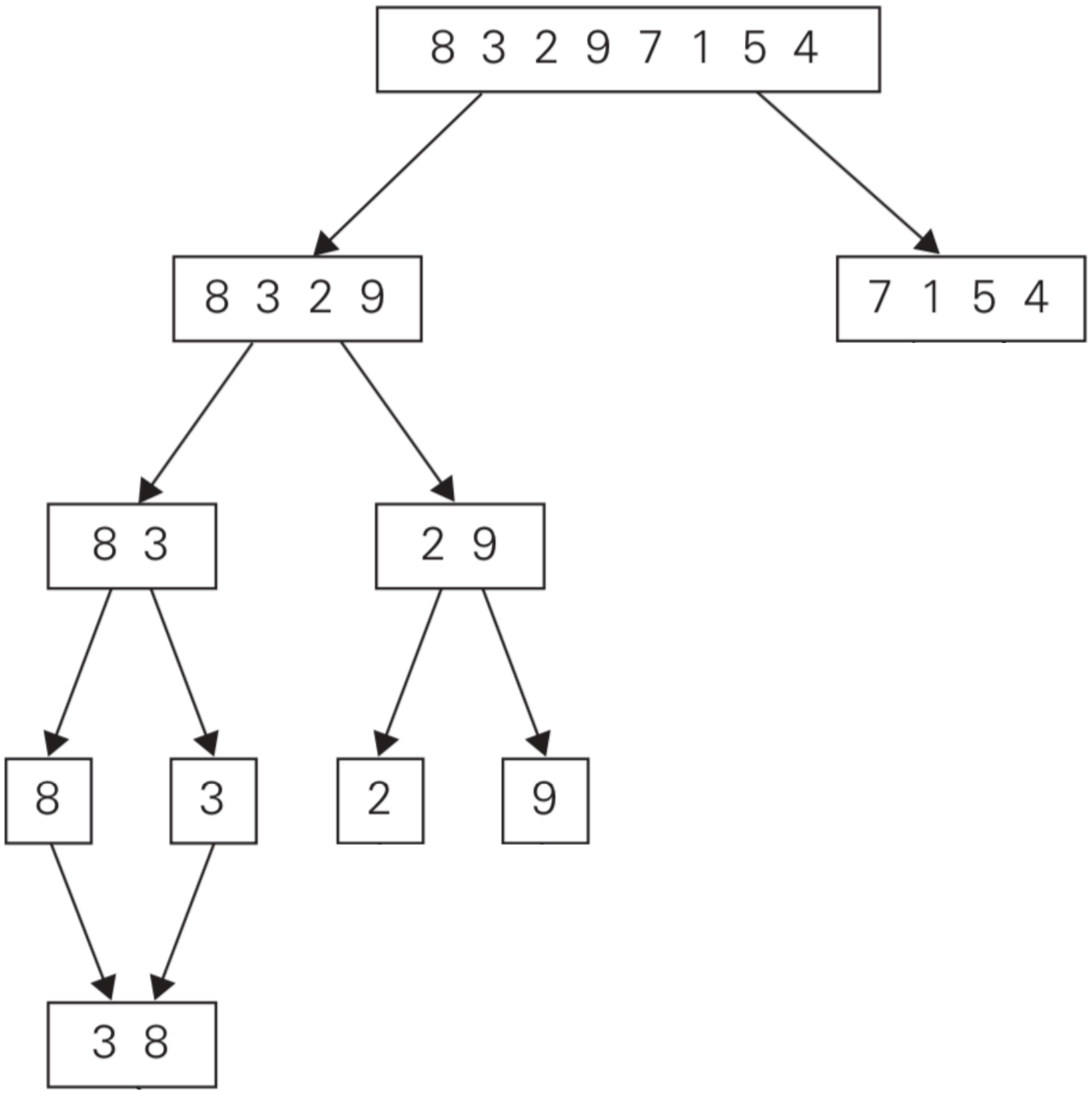
Mergesort



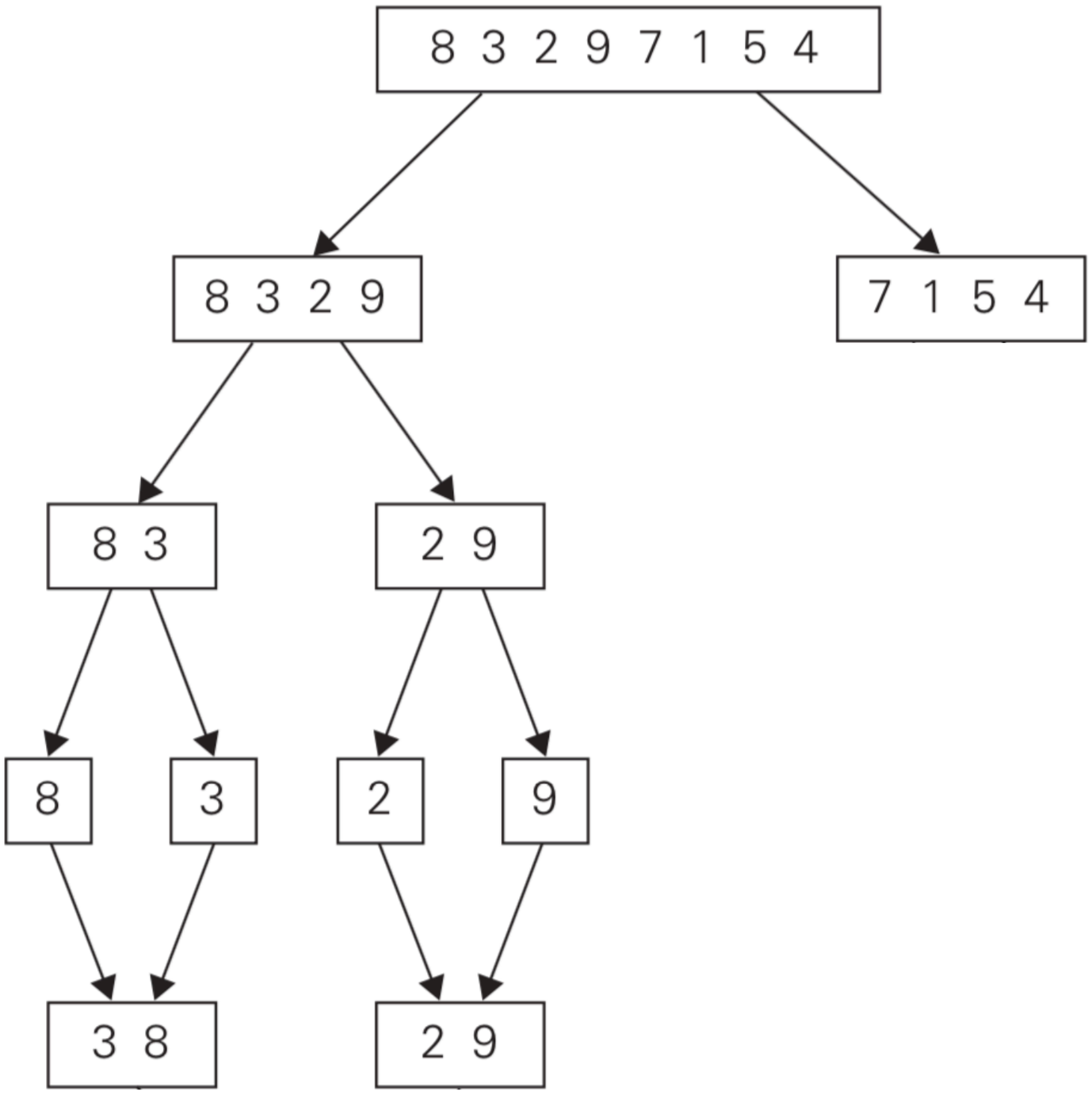
Mergesort



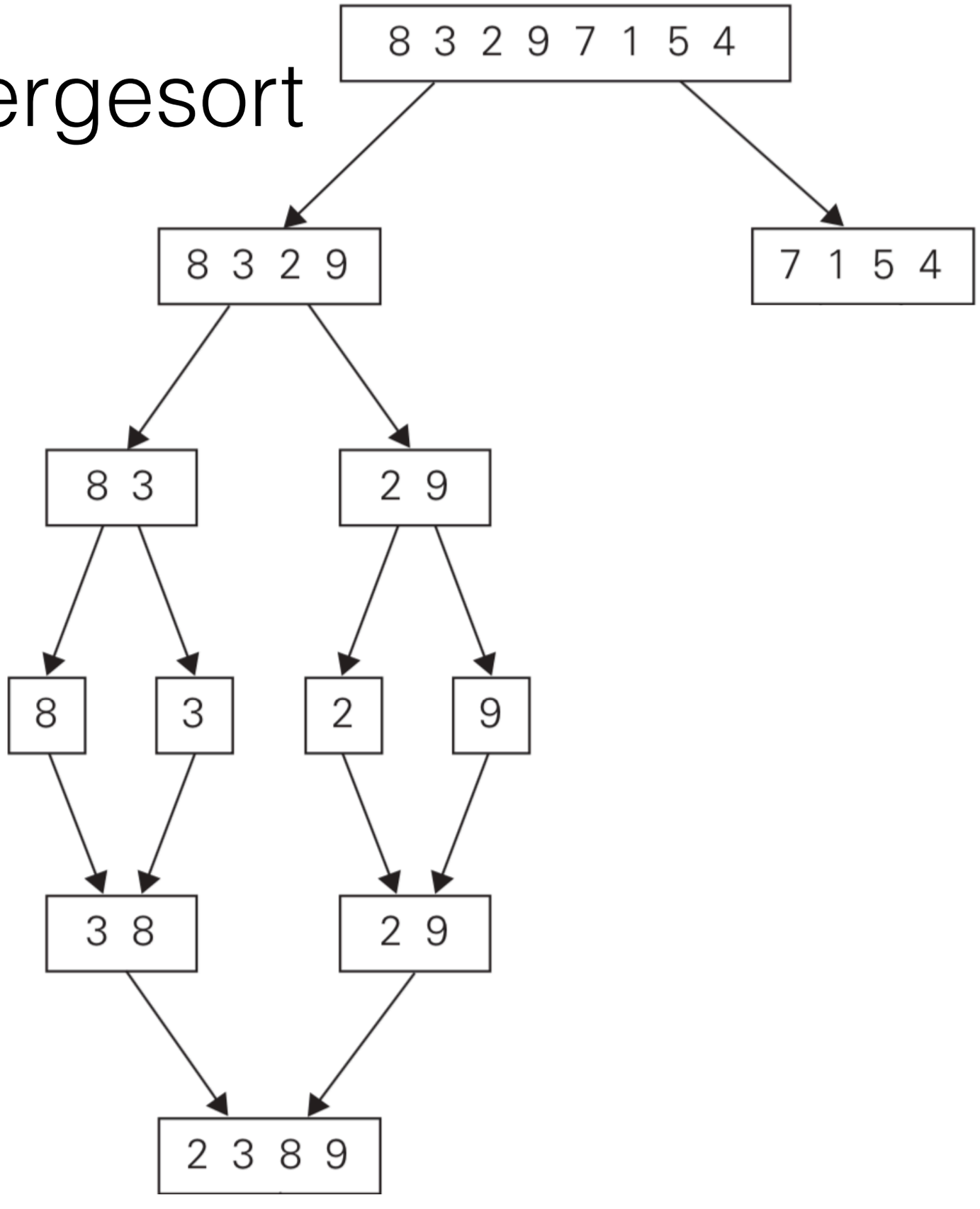
Mergesort



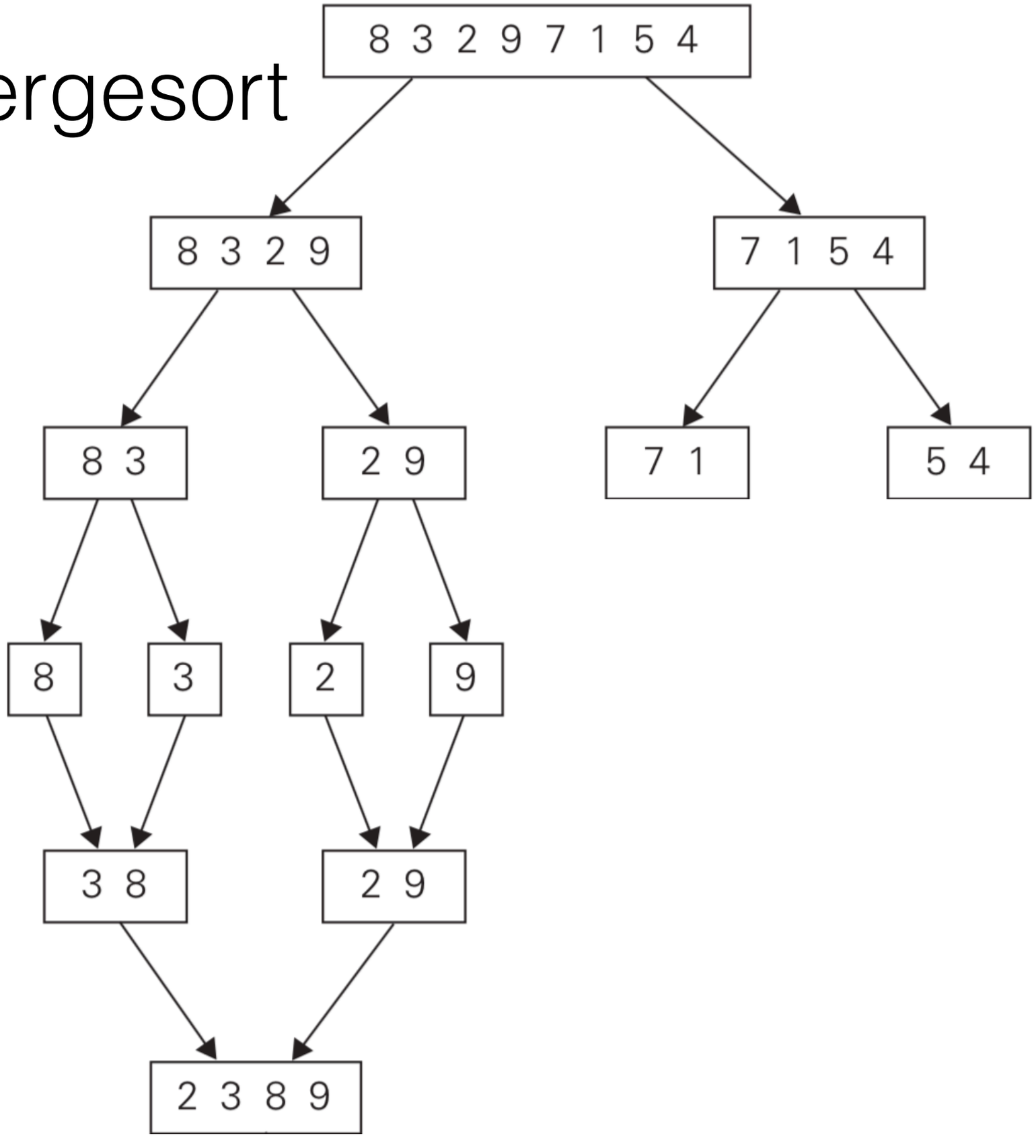
Mergesort



Mergesort

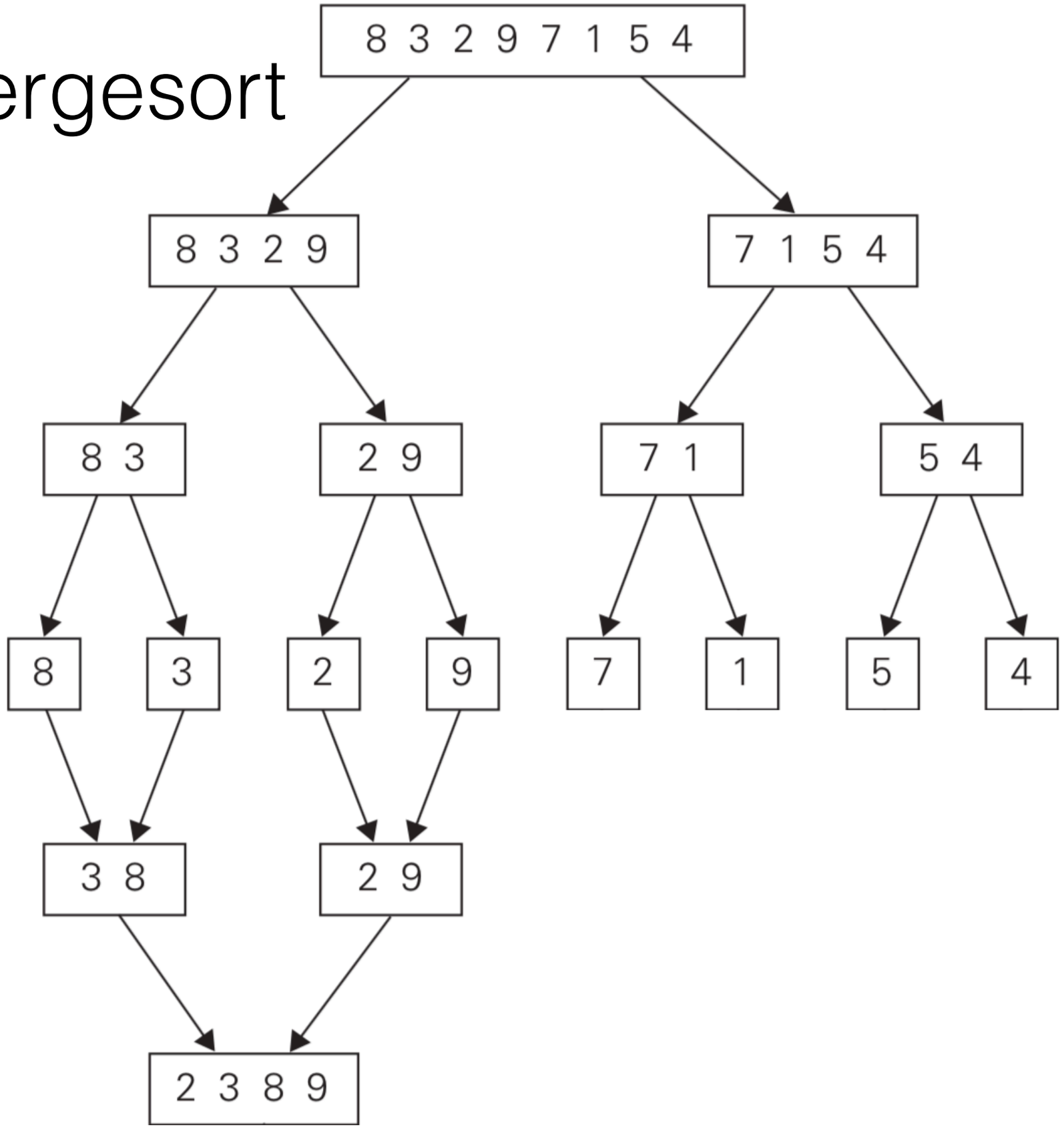


Mergesort

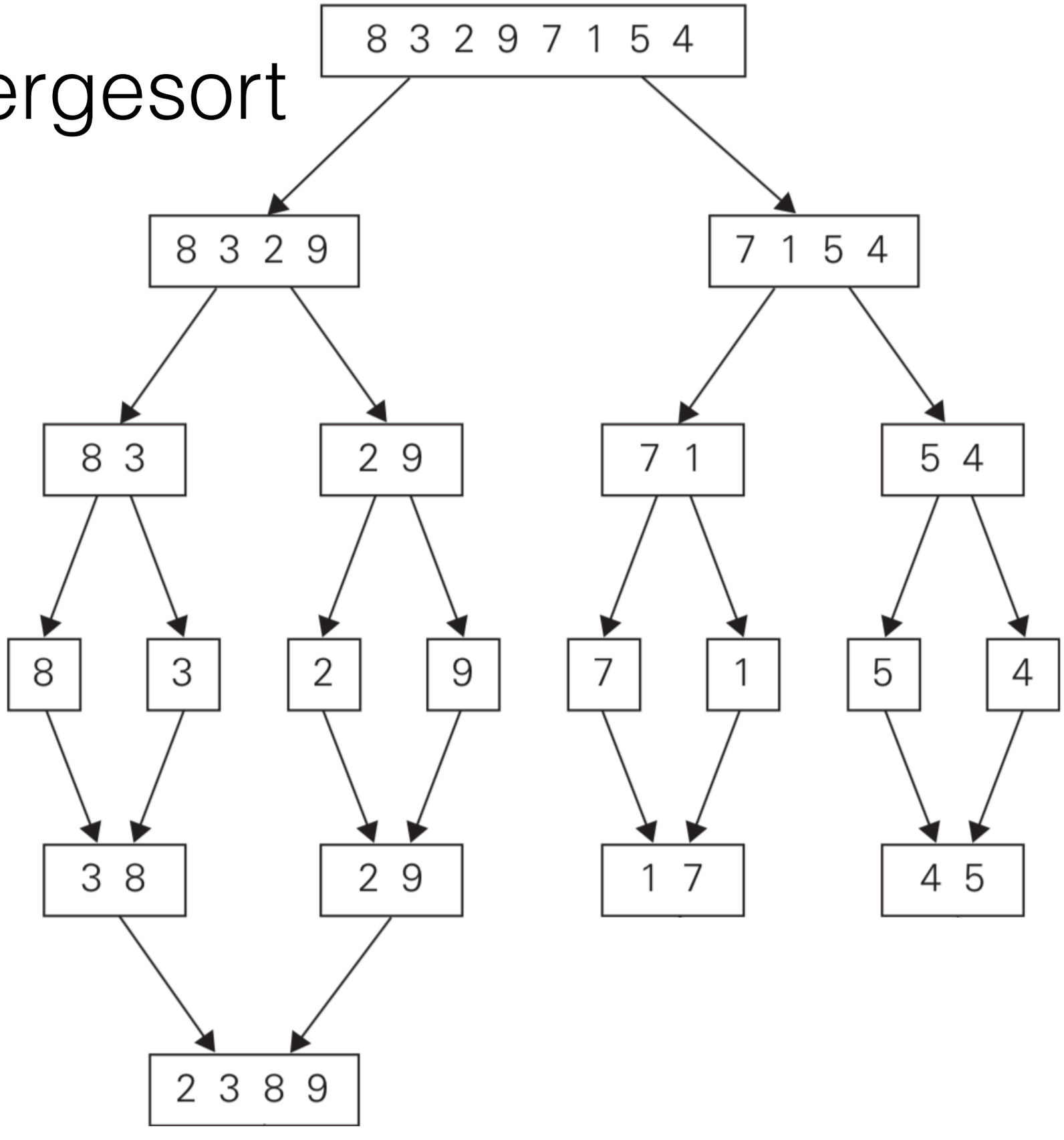


UNIVERSITY OF MELBOURNE

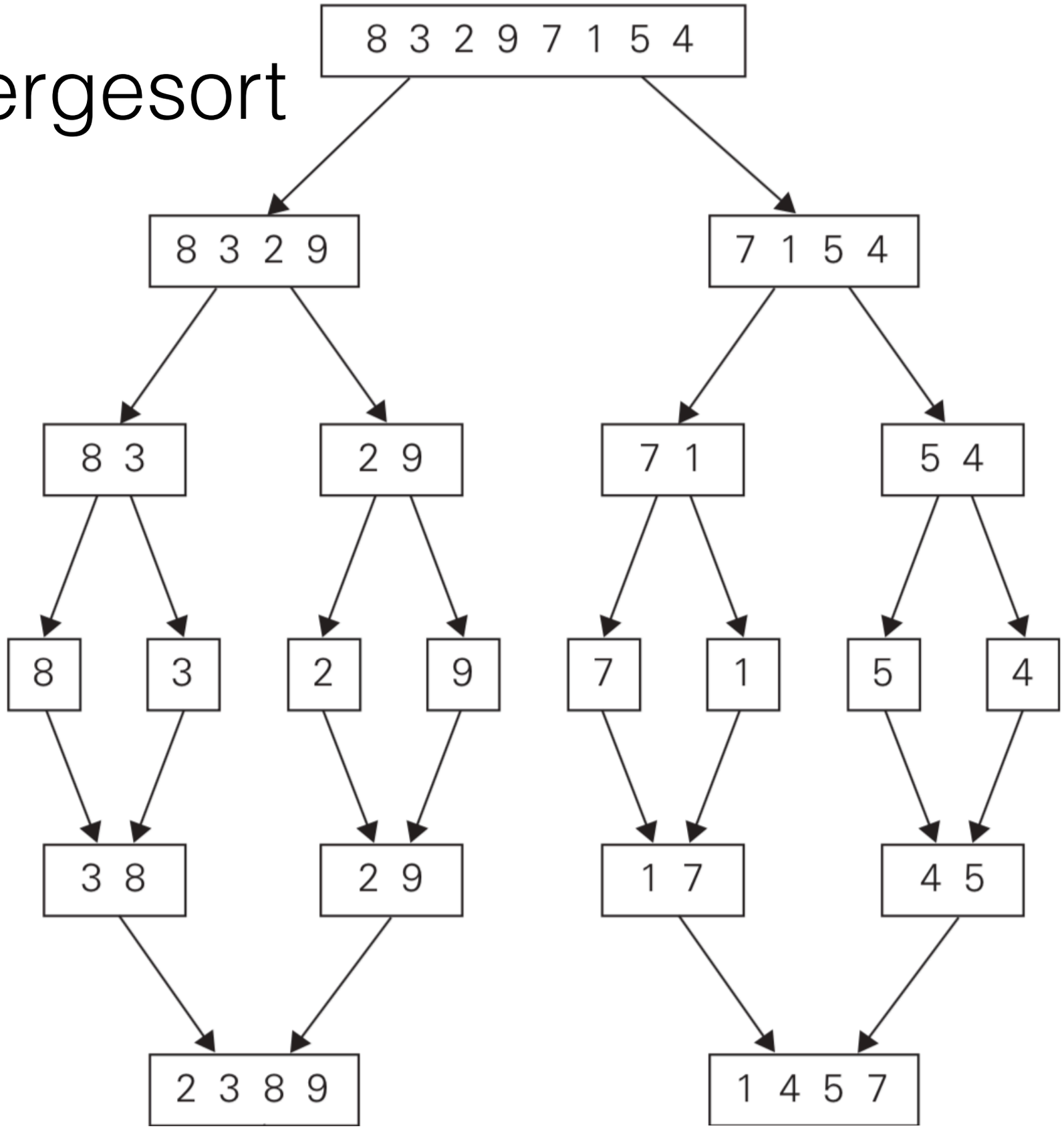
Mergesort



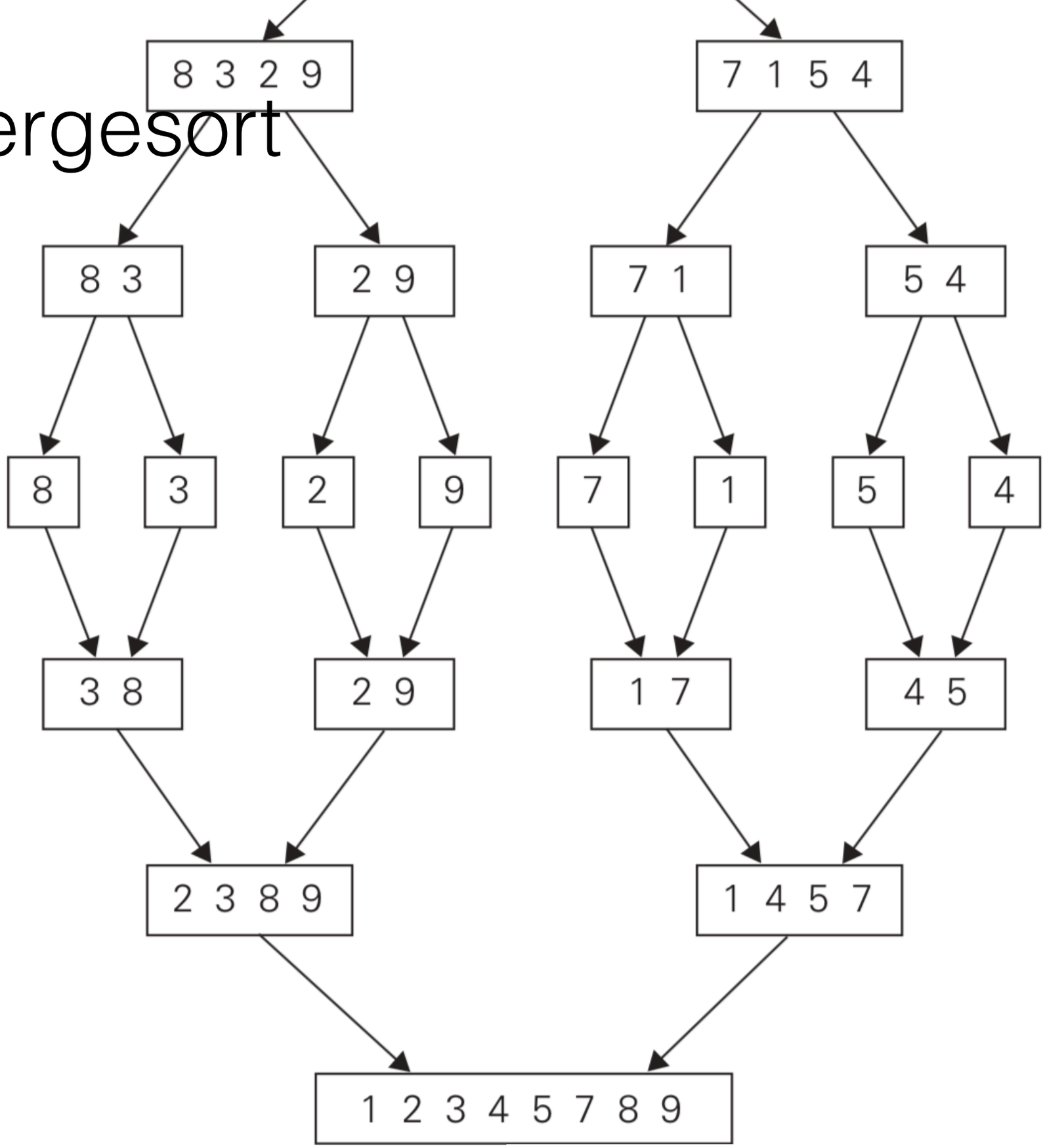
Mergesort



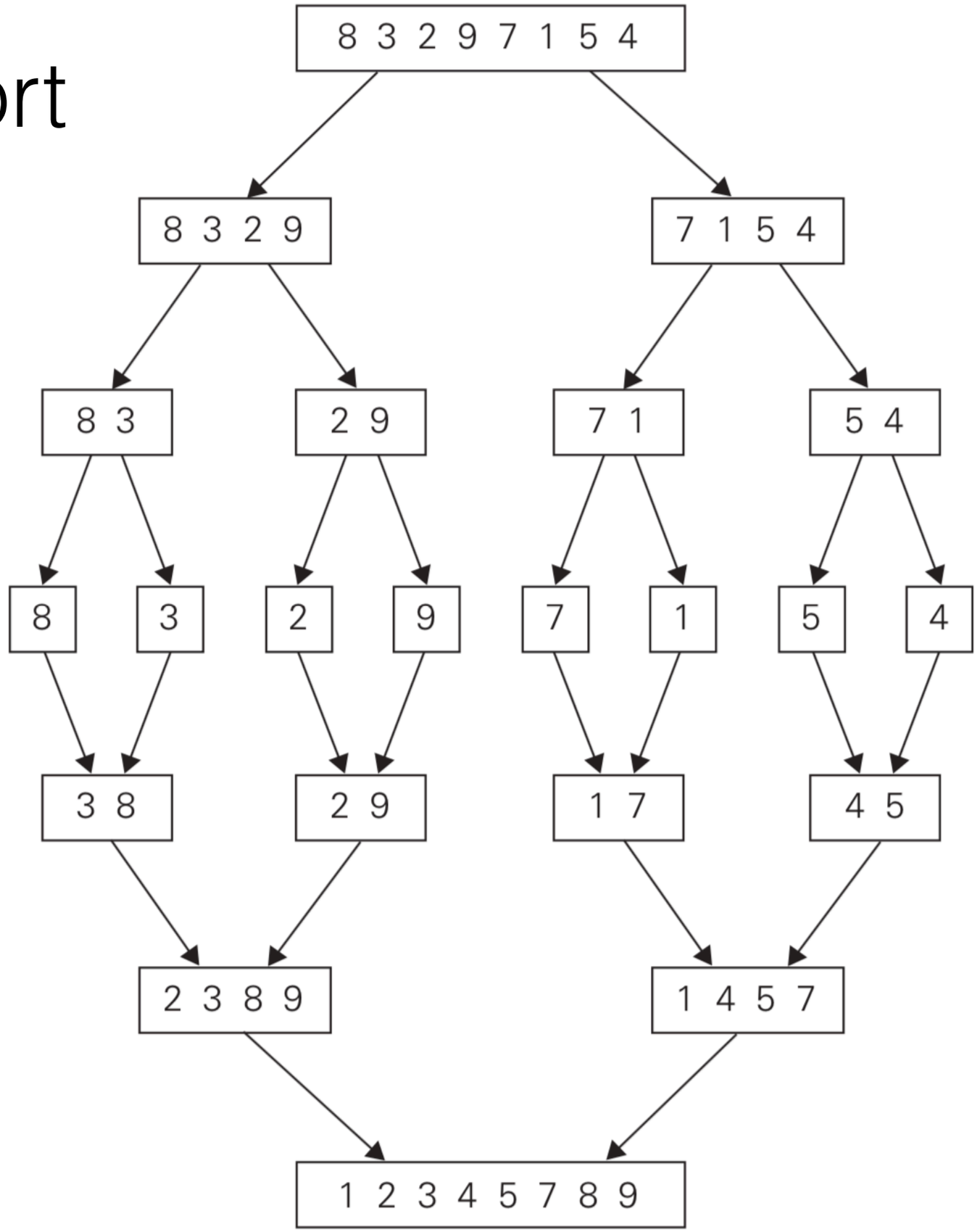
Mergesort



Mergesort



Mergesort



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort



THE UNIVERSITY OF MELBOURNE

procedure MERGESORT($A[\cdot], n$) ▷ Sort $A[0]..A[n-1]$
  if $n > 1$ **then**
    for $i \leftarrow 0$ **to** $\lfloor n/2 \rfloor - 1$ **do** ▷ Copy left half of A to B
      $B[i] \leftarrow A[i]$
    for $i \leftarrow 0$ **to** $\lceil n/2 \rceil - 1$ **do** ▷ Copy right half of A to C
      $C[i] \leftarrow A[\lfloor n/2 \rfloor + i]$
    MERGESORT($B, \lfloor n/2 \rfloor$) ▷ Sort B
    MERGESORT($C, \lceil n/2 \rceil$) ▷ Sort C
    MERGE($B, \lfloor n/2 \rfloor, C, \lceil n/2 \rceil, A$) ▷ Merge B and C into A

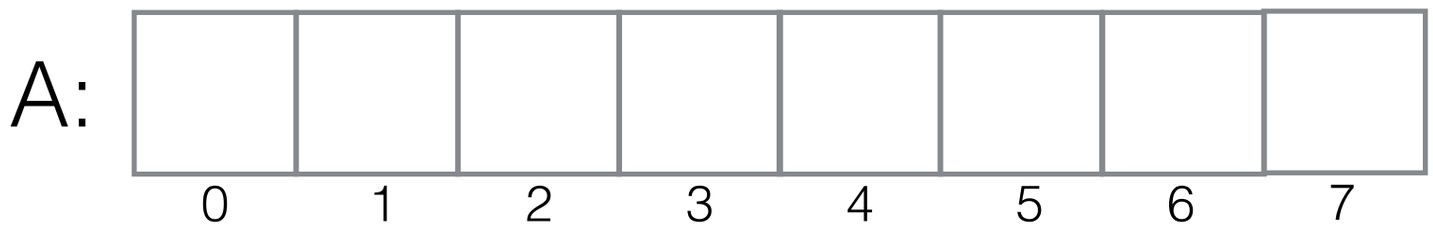
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Merging Arrays



THE UNIVERSITY OF MELBOURNE





Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Merging Arrays



THE UNIVERSITY OF MELBOURNE

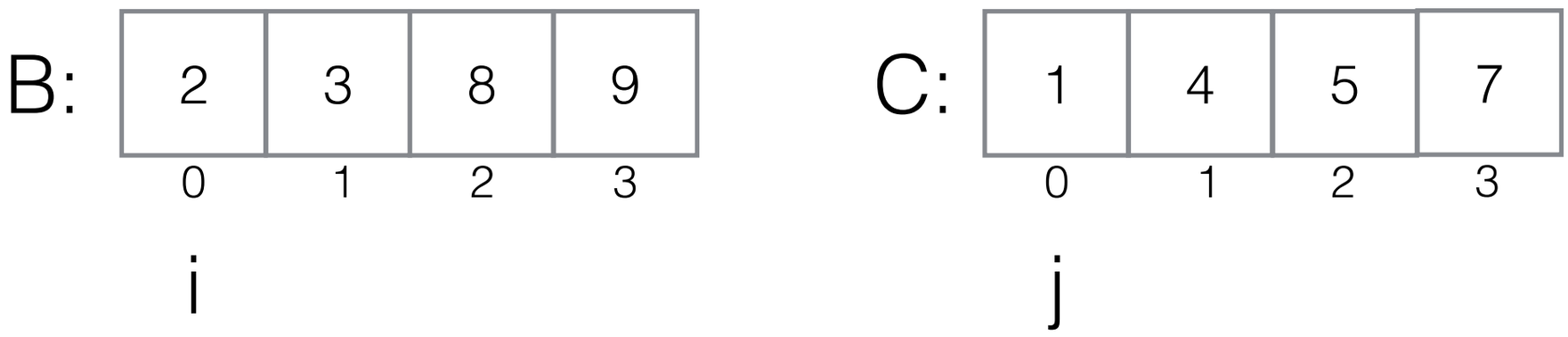
procedure MERGE($B[\cdot], p, C[\cdot], q, A[\cdot]$)
    $i \leftarrow 0; j \leftarrow 0; k \leftarrow 0$
    while $i < p$ and $j < q$ **do**
        if $B[i] \leq C[j]$ **then**
            $A[k] \leftarrow B[i]$
            $i \leftarrow i + 1$
        else
            $A[k] \leftarrow C[j]$
            $j \leftarrow j + 1$
        $k \leftarrow k + 1$
if $i = p$ **then**
    copy $C[j]..C[q-1]$ to $A[k]..A[p+q-1]$ ▷ (a for loop)
else
    copy $B[i]..B[p-1]$ to $A[k]..A[p+q-1]$ ▷ (a for loop)

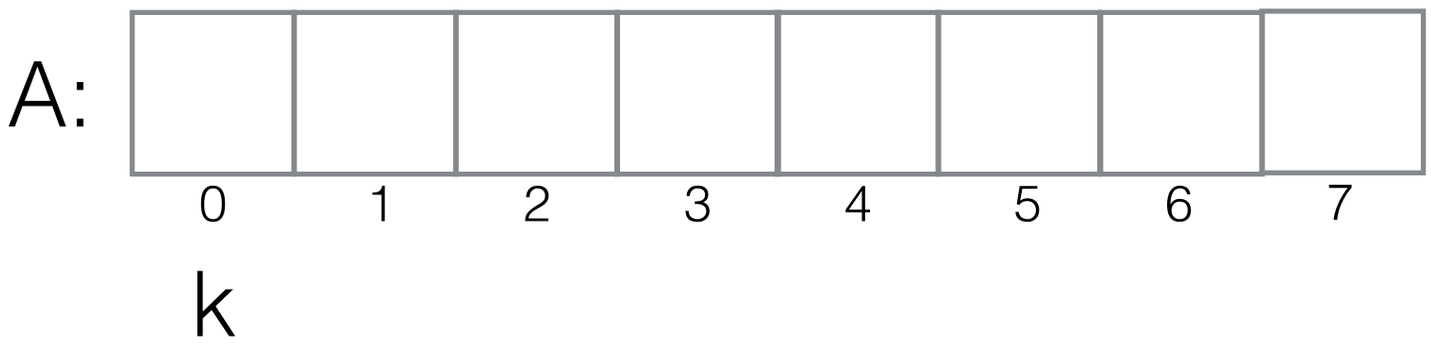
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Merging Arrays



THE UNIVERSITY OF MELBOURNE



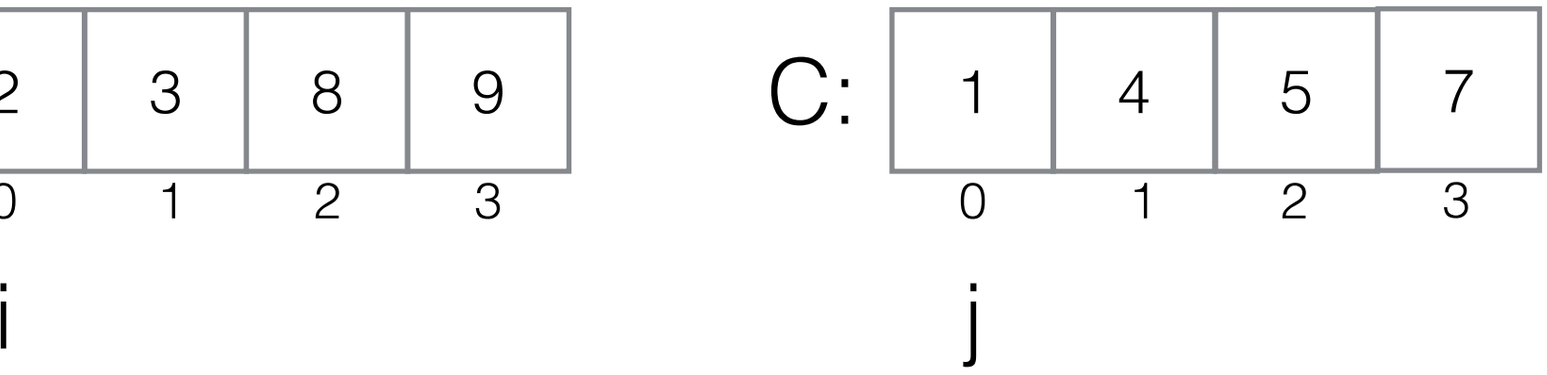


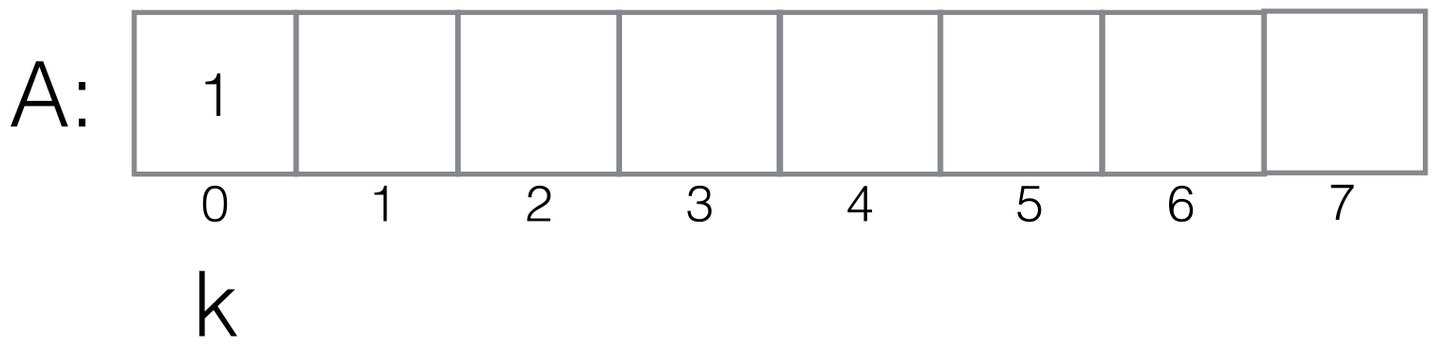
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Merging Arrays



THE UNIVERSITY OF MELBOURNE



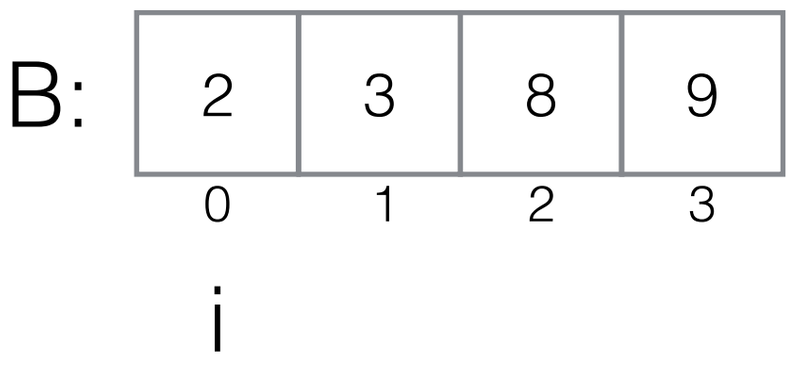


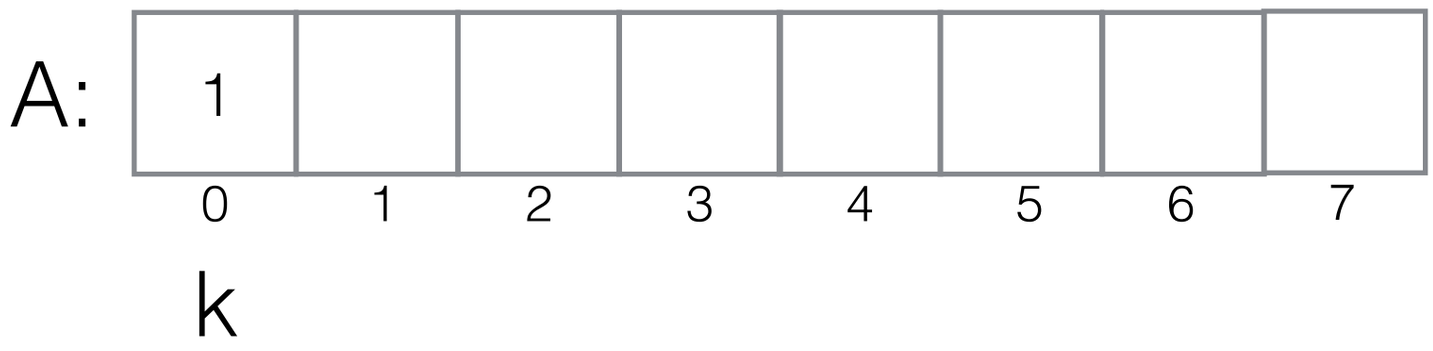
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Merging Arrays



THE UNIVERSITY OF MELBOURNE



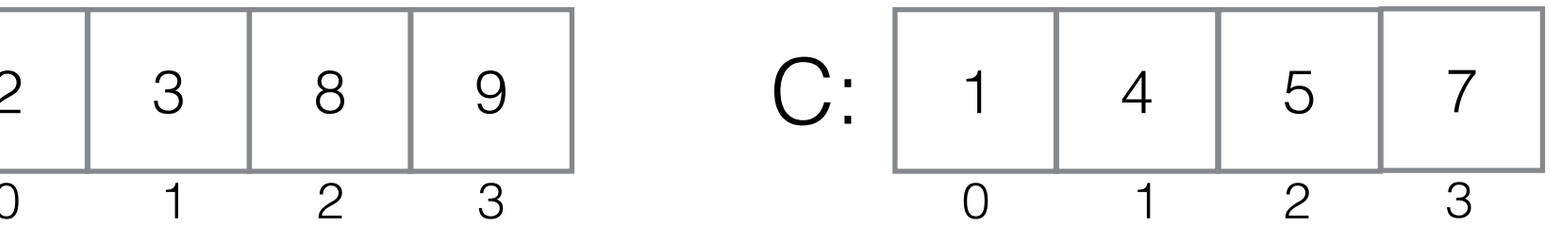


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Merging Arrays

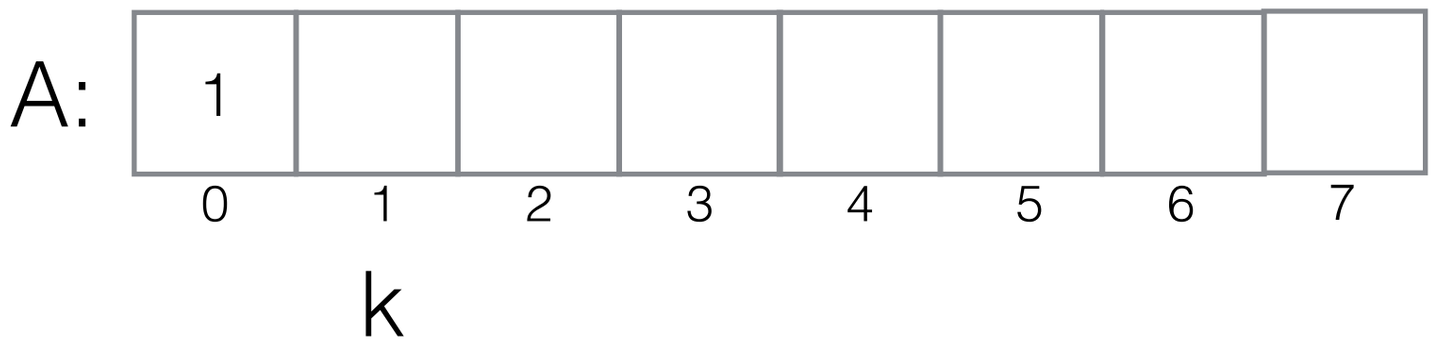


THE UNIVERSITY OF MELBOURNE



p: 4

q: 4

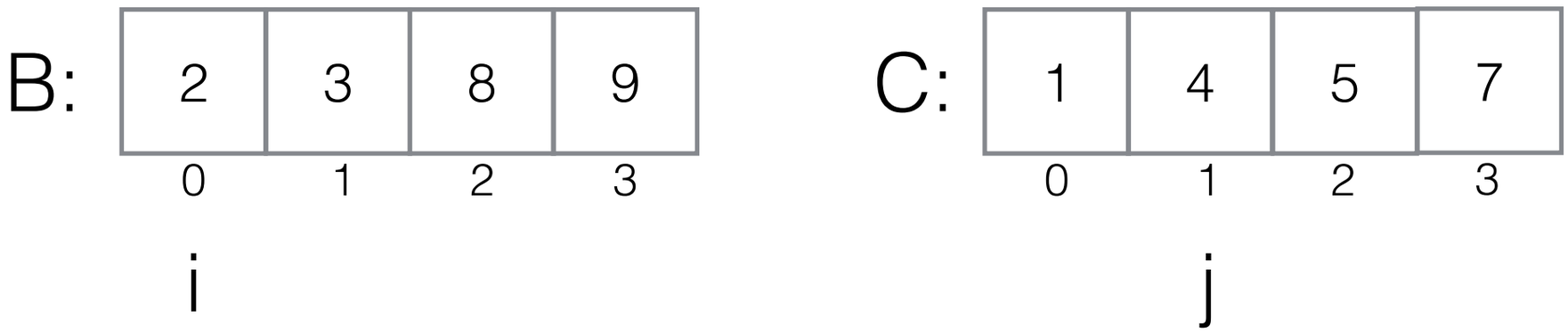


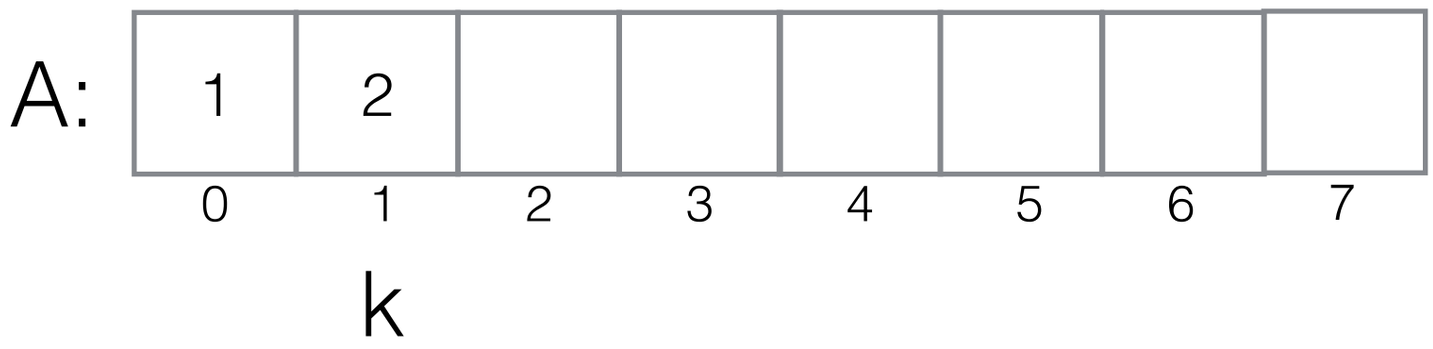
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Merging Arrays



THE UNIVERSITY OF MELBOURNE

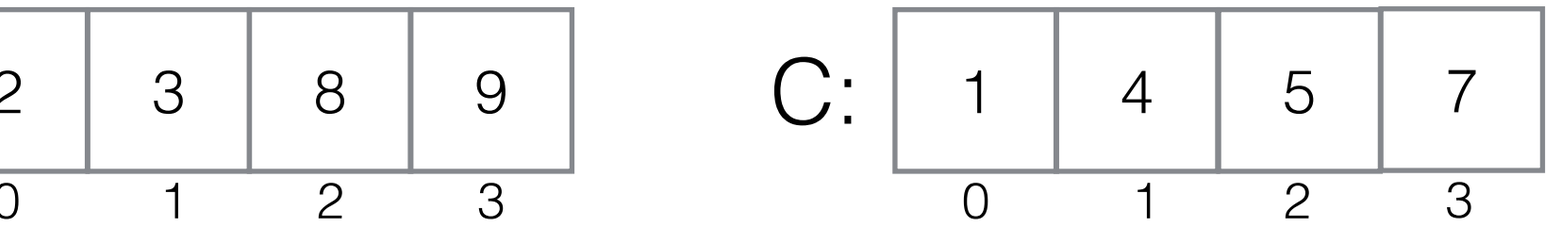




Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

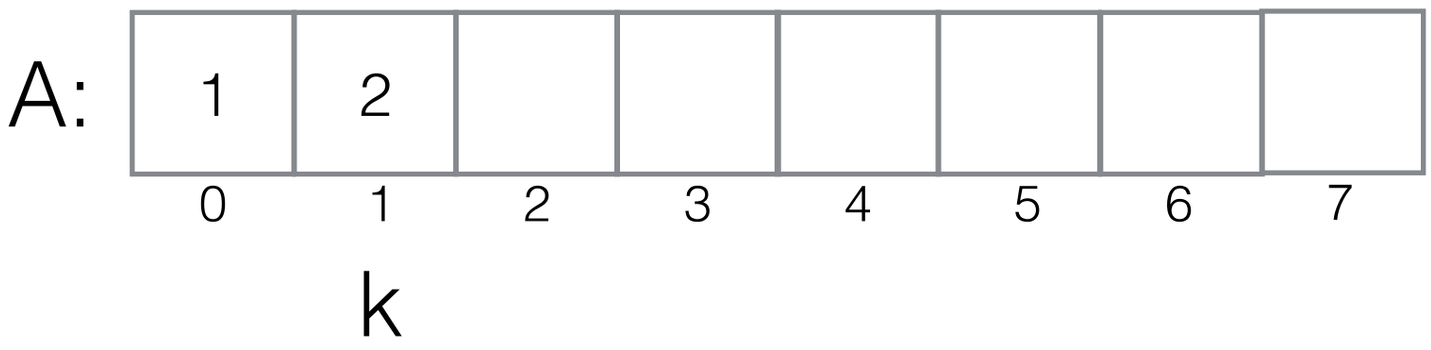
Mergesort: Merging Arrays





p: 4

q: 4

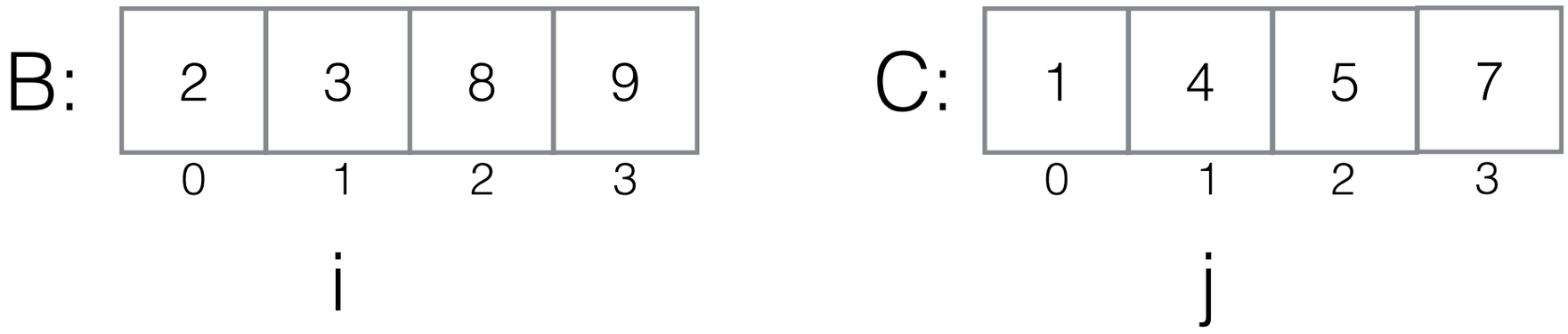


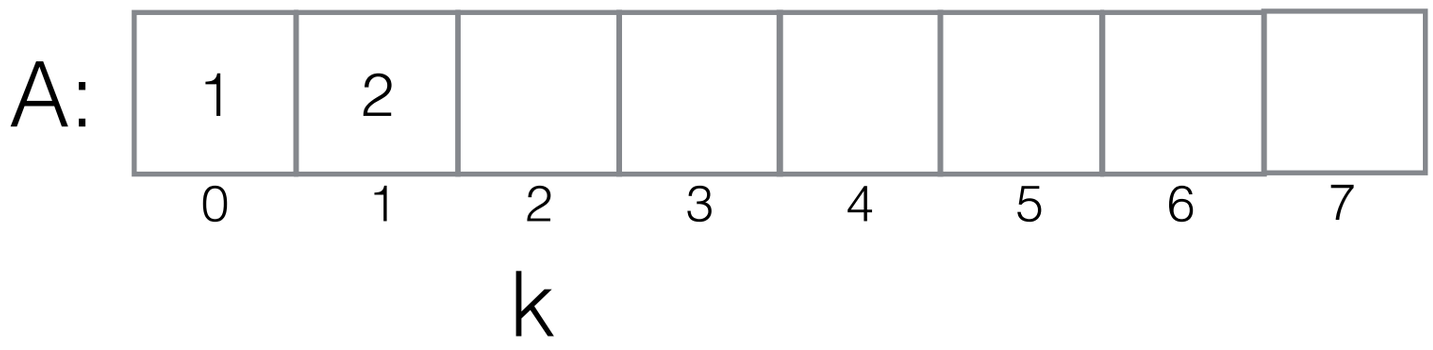
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Merging Arrays



THE UNIVERSITY OF MELBOURNE

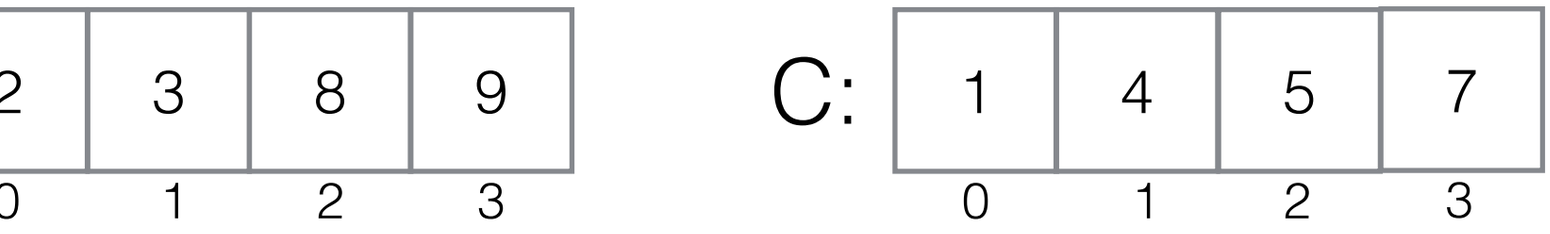




Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

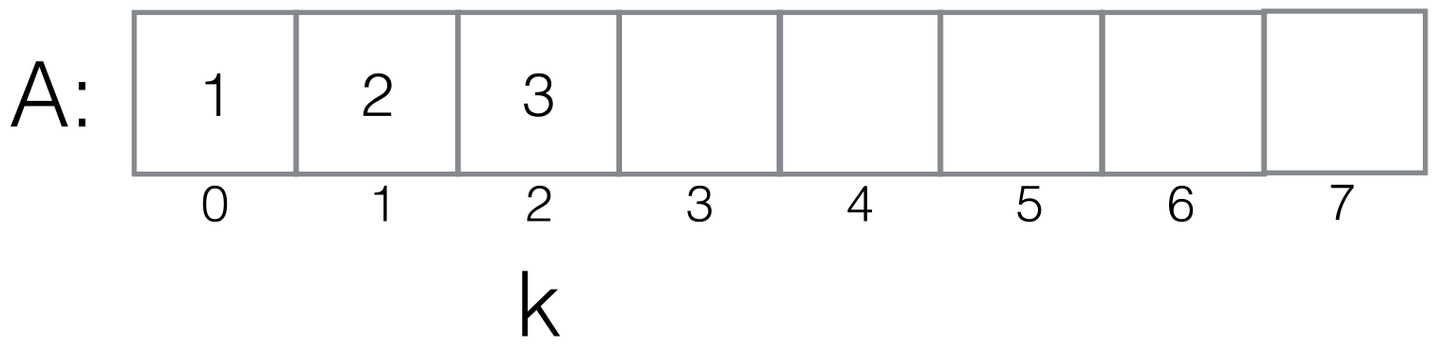
Mergesort: Merging Arrays





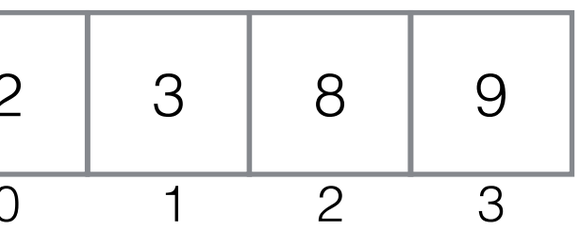
p: 4

q: 4



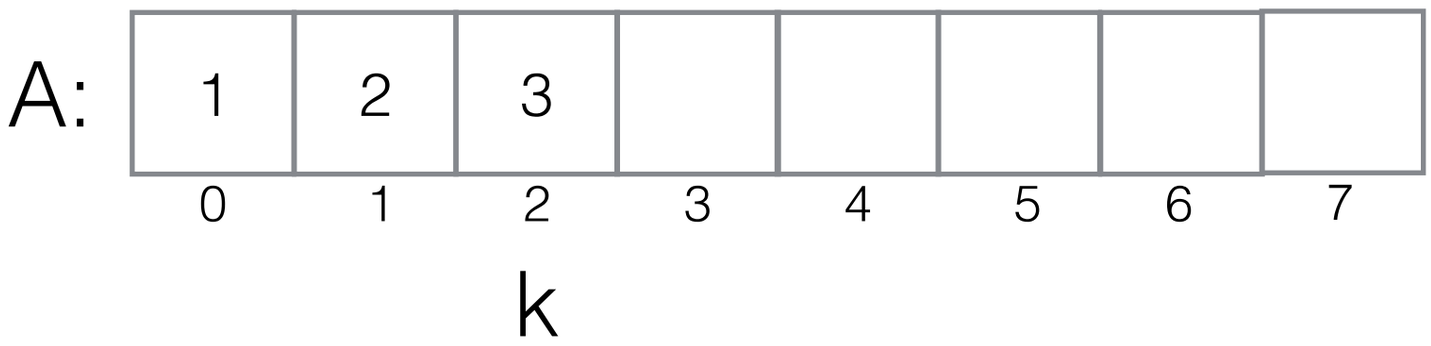
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Merging Arrays



p: 4

q: 4

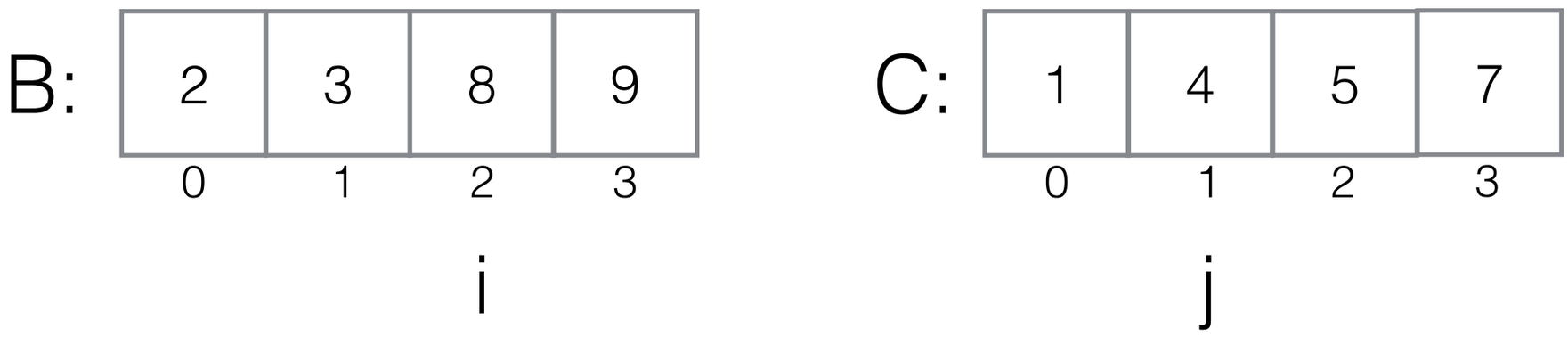


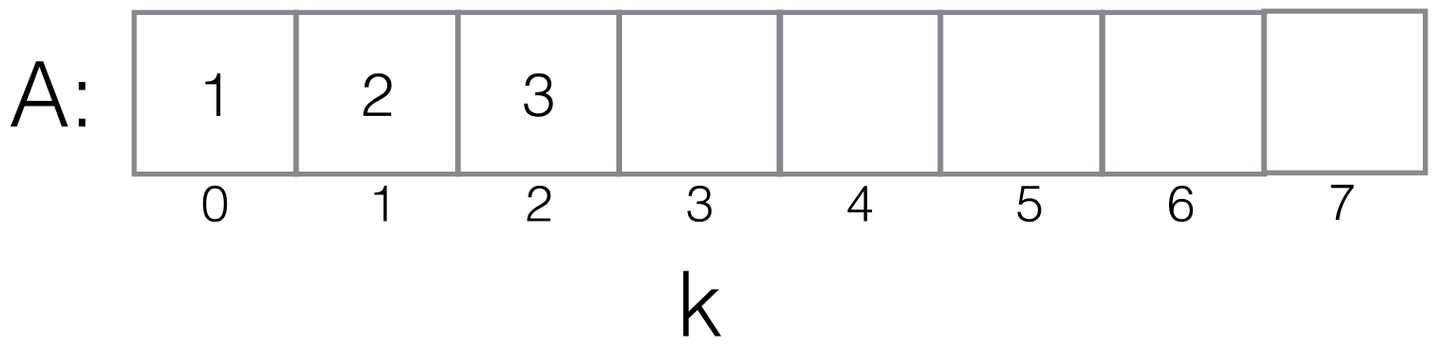
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Merging Arrays



THE UNIVERSITY OF MELBOURNE



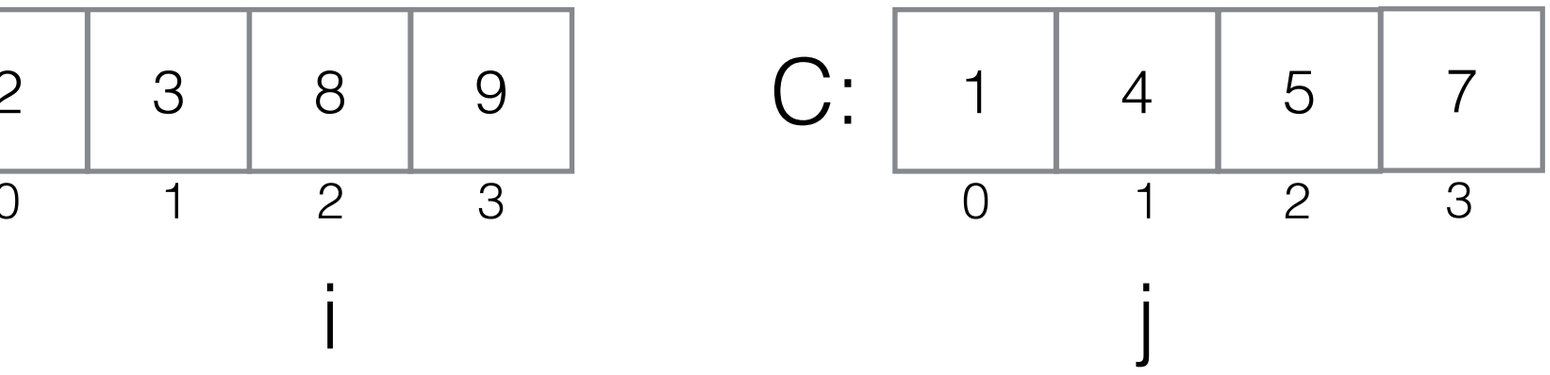


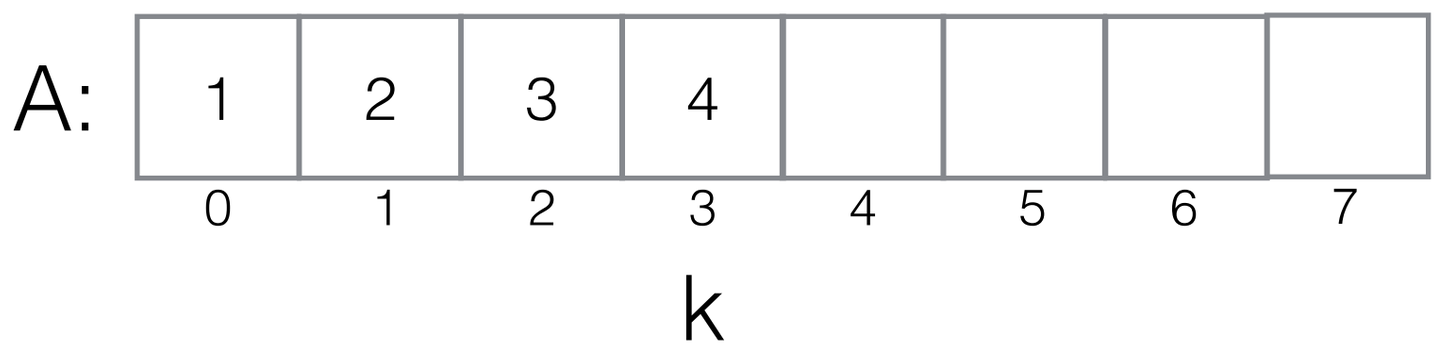
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Merging Arrays



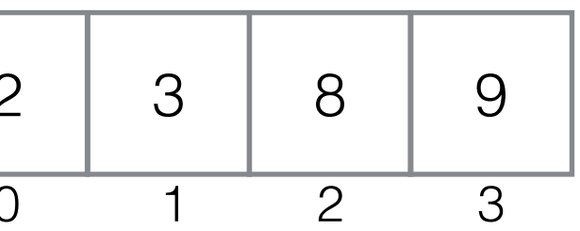
THE UNIVERSITY OF MELBOURNE

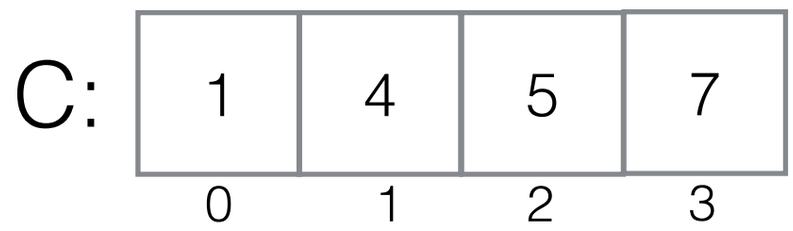




Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

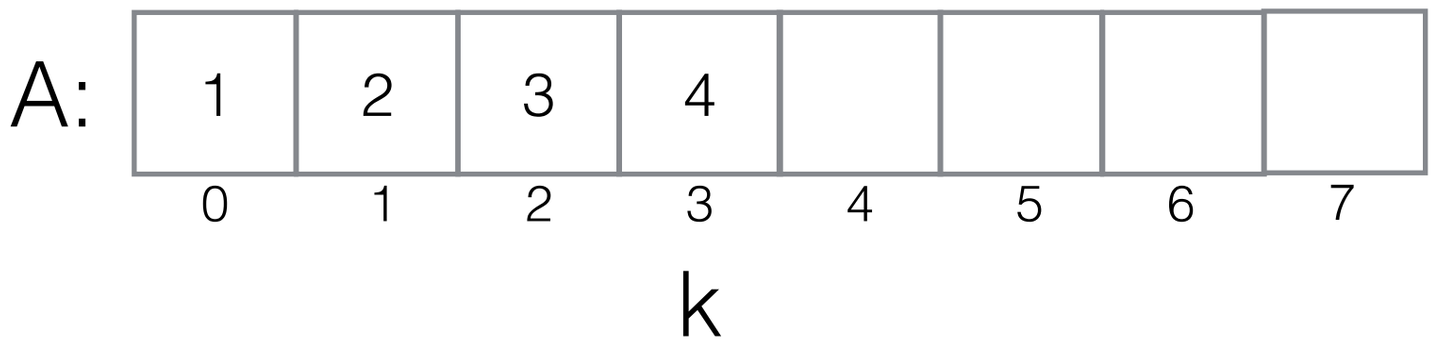
Mergesort: Merging Arrays





p: 4

q: 4

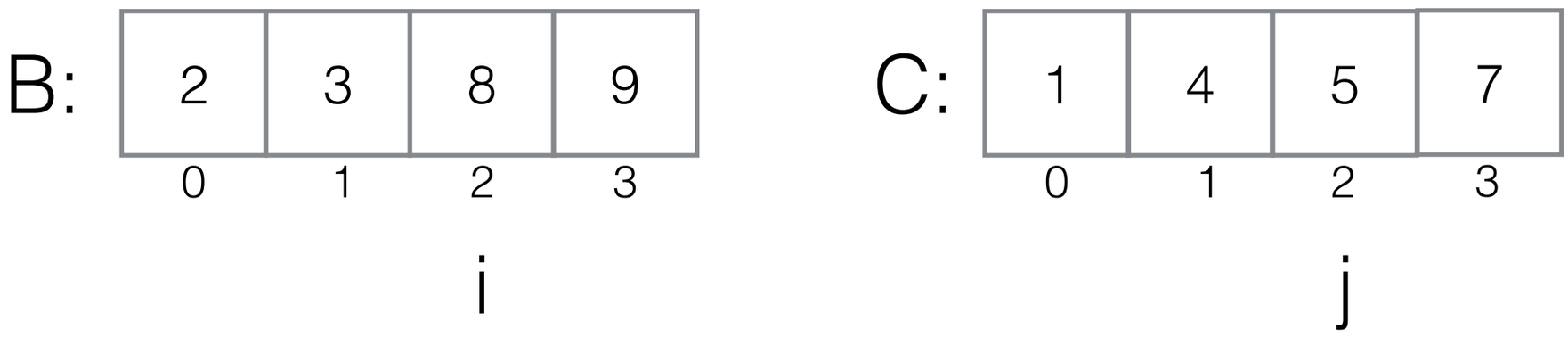


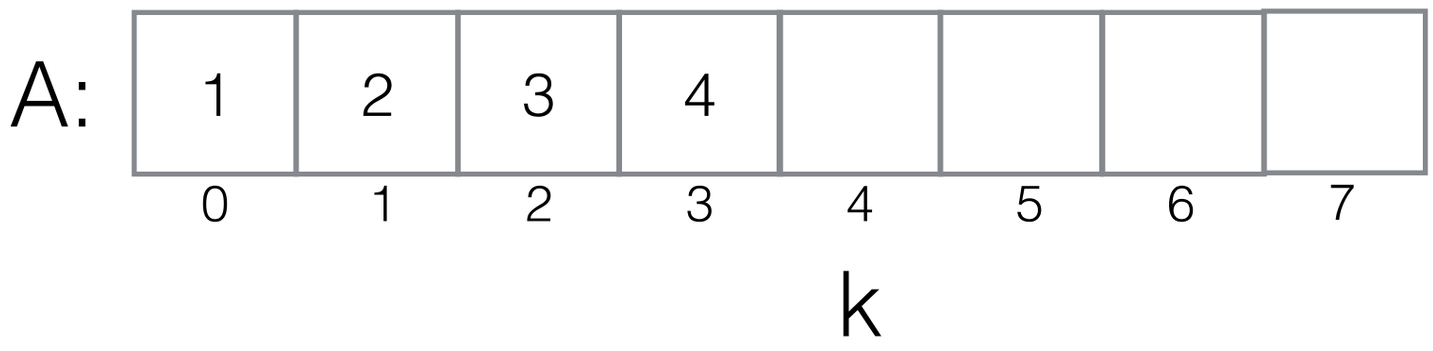
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Merging Arrays



THE UNIVERSITY OF MELBOURNE



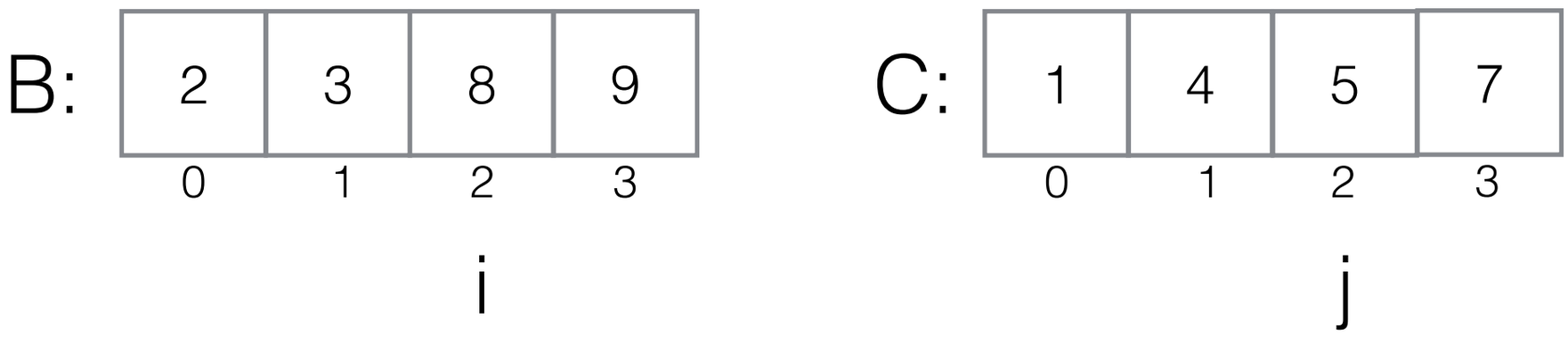


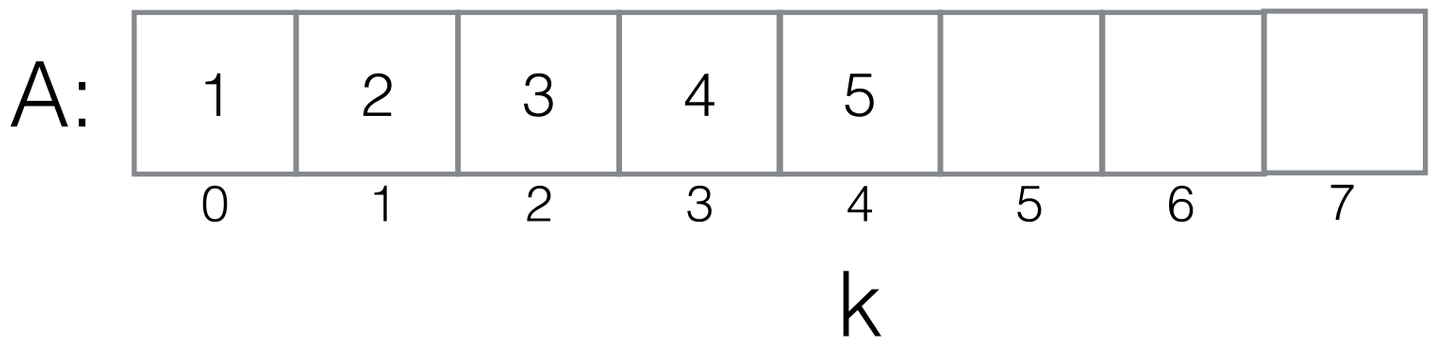
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Merging Arrays



THE UNIVERSITY OF MELBOURNE



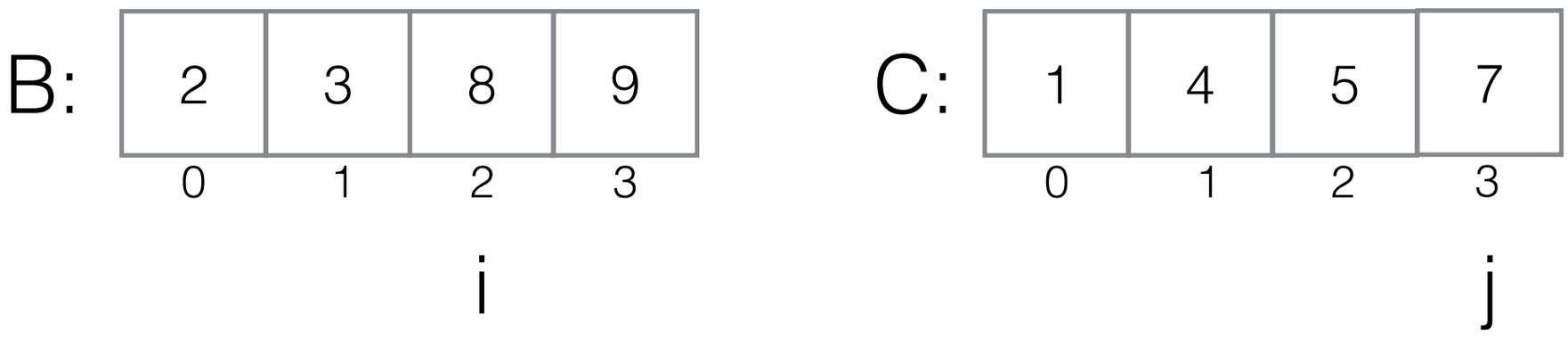


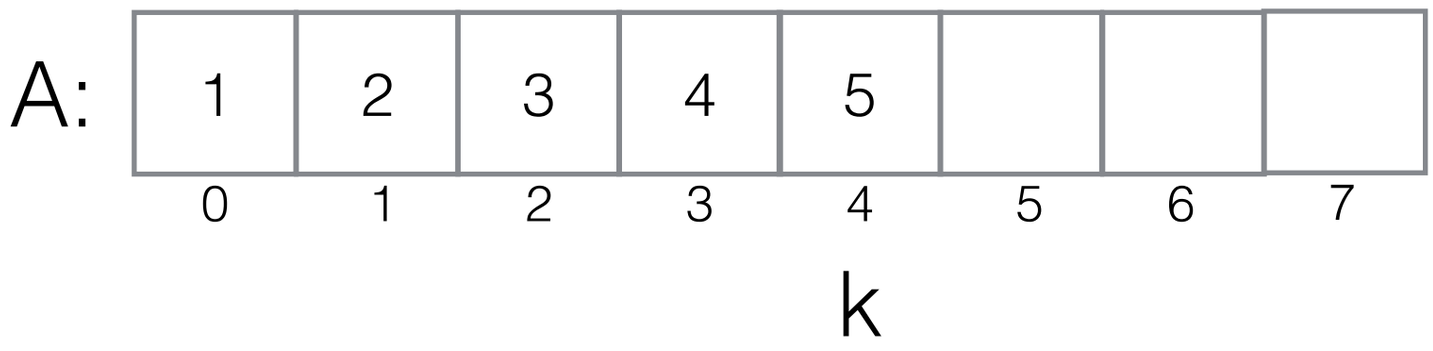
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Merging Arrays



THE UNIVERSITY OF MELBOURNE



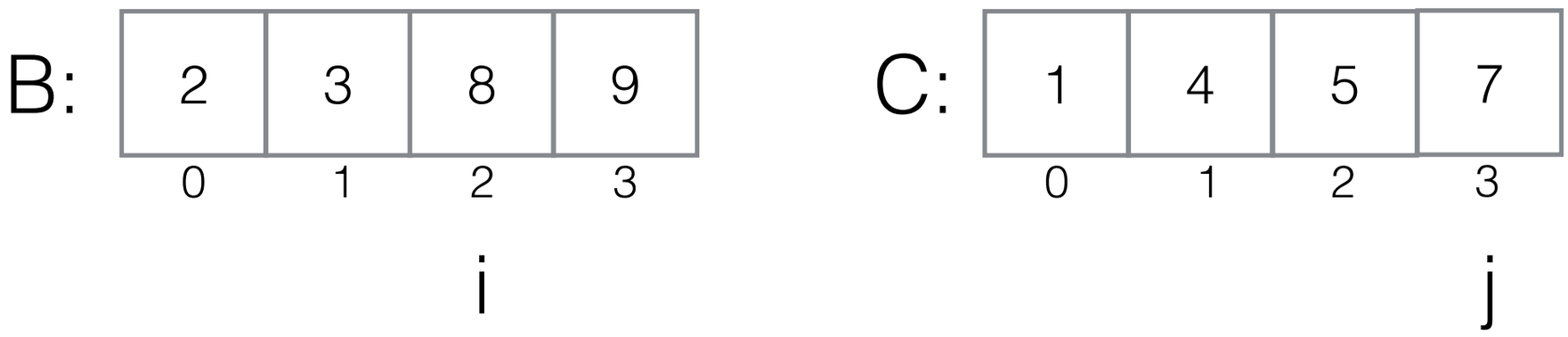


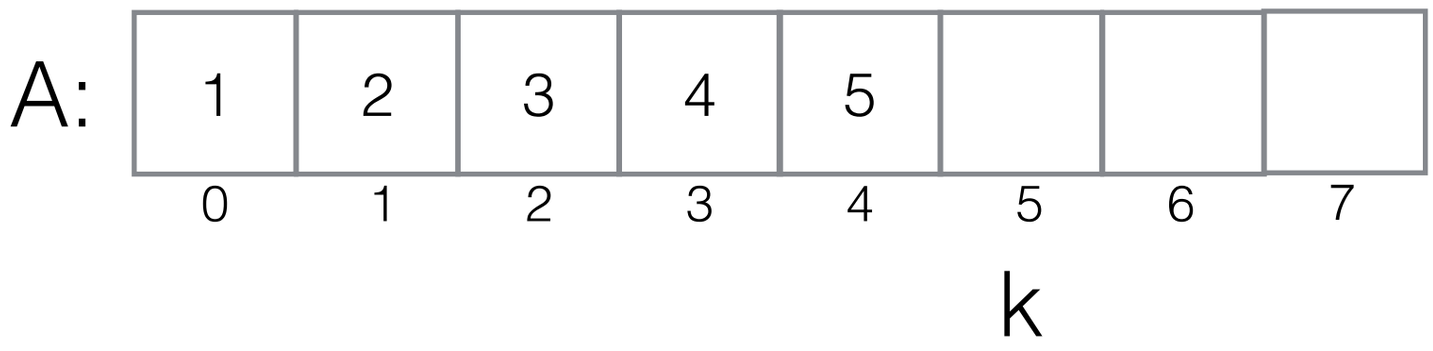
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Merging Arrays



THE UNIVERSITY OF MELBOURNE



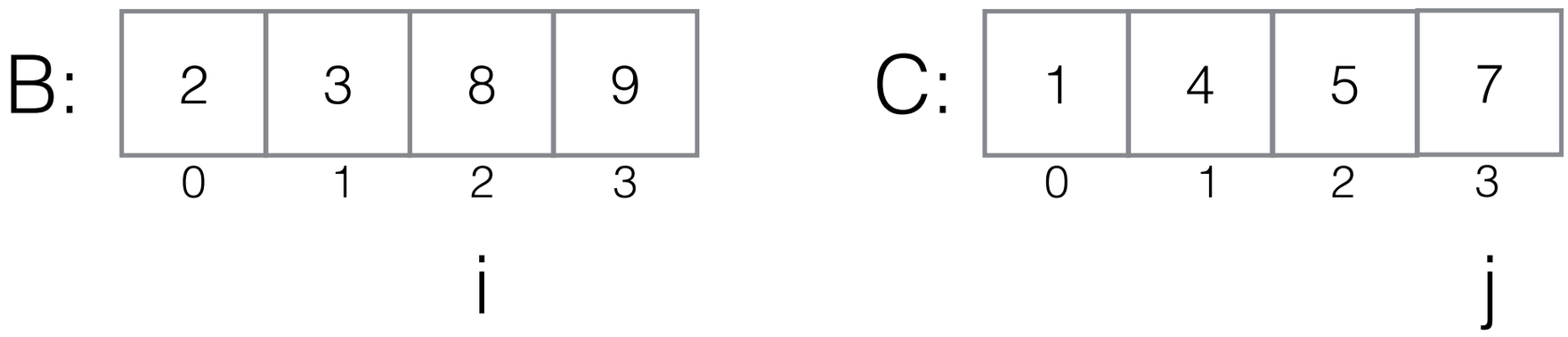


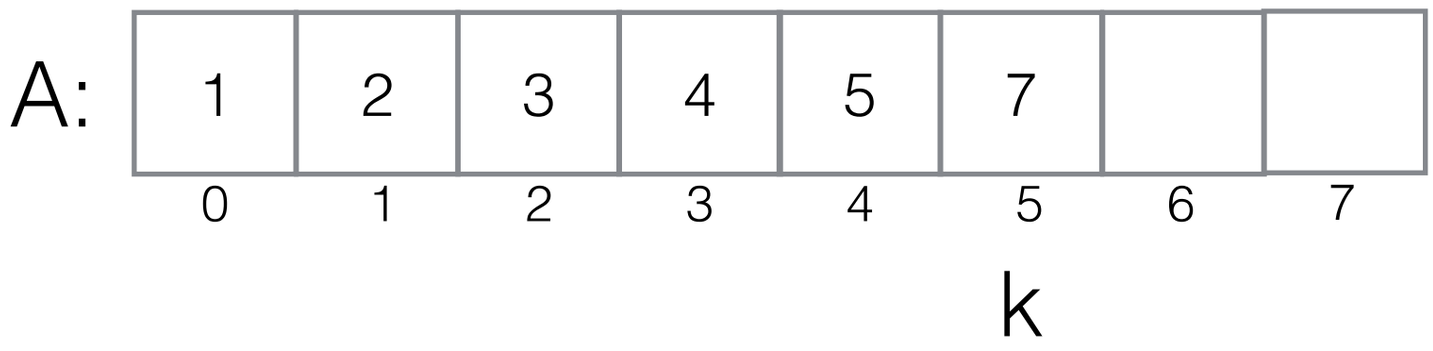
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Merging Arrays



THE UNIVERSITY OF MELBOURNE





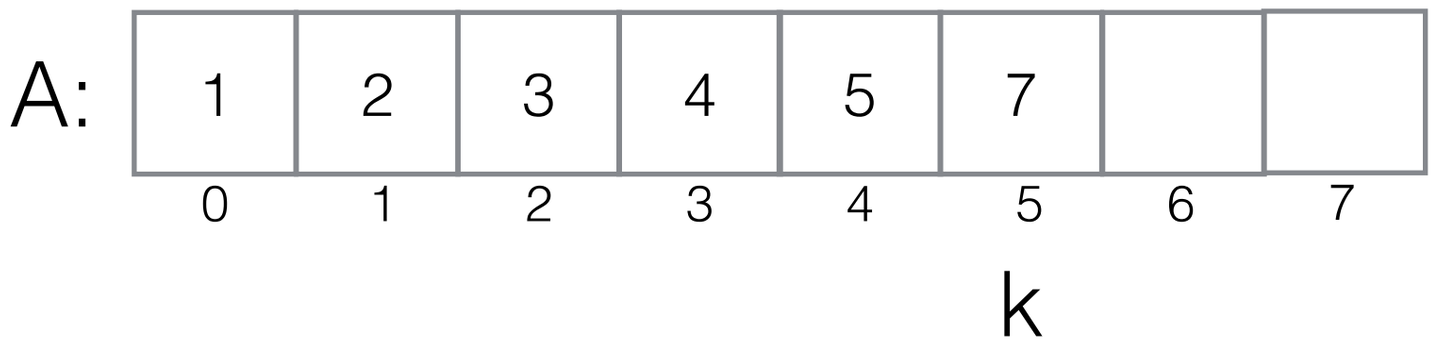
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Merging Arrays



THE UNIVERSITY OF MELBOURNE





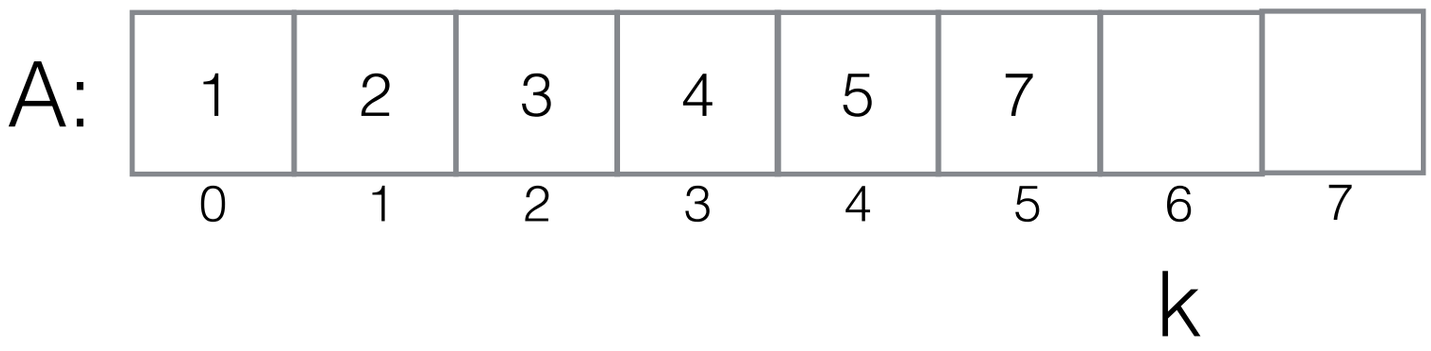
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Merging Arrays



THE UNIVERSITY OF MELBOURNE





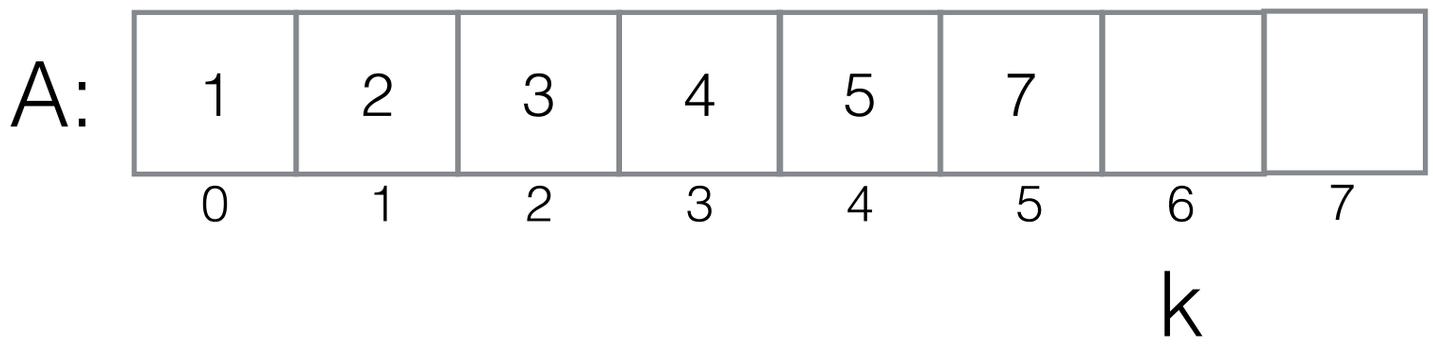
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Merging Arrays



THE UNIVERSITY OF MELBOURNE





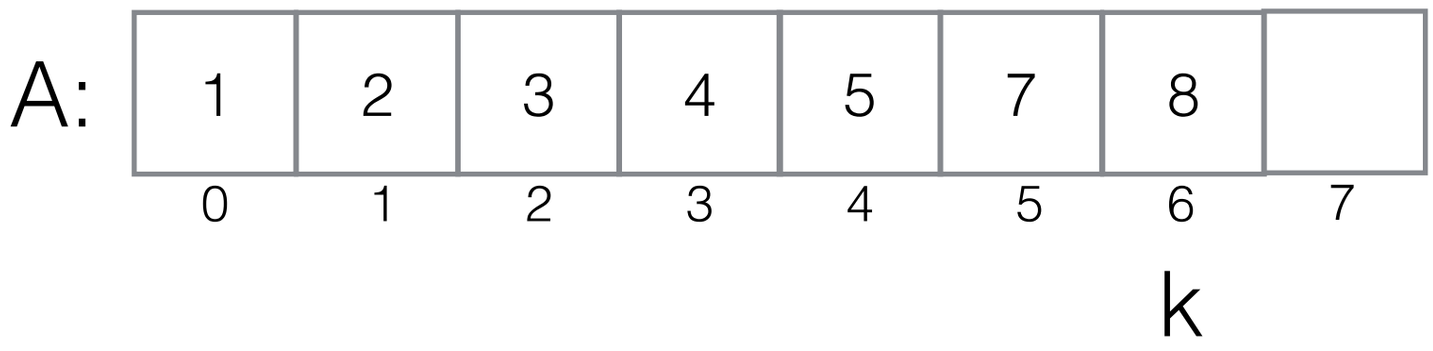
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Merging Arrays



THE UNIVERSITY OF MELBOURNE



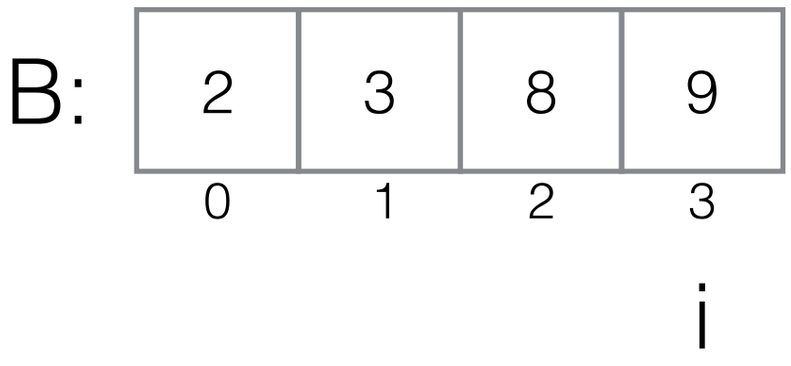


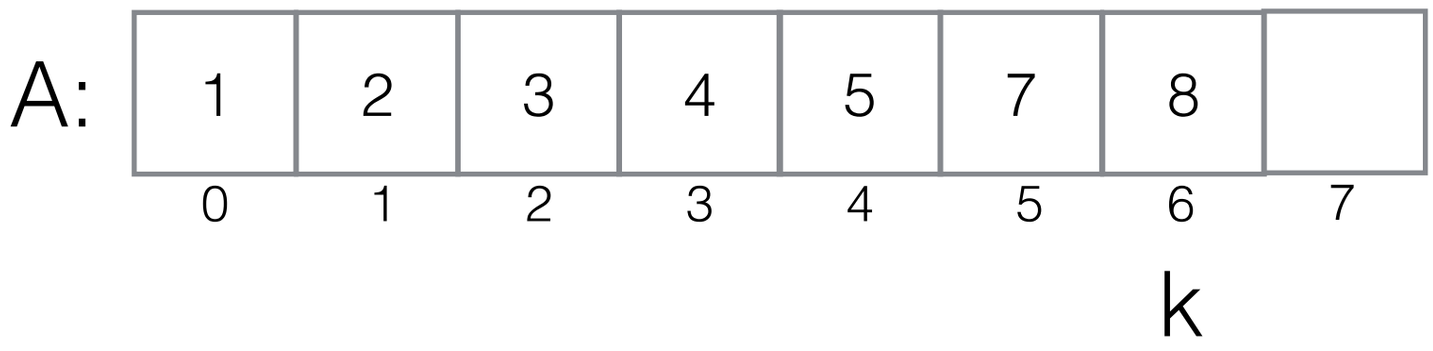
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Merging Arrays



THE UNIVERSITY OF MELBOURNE



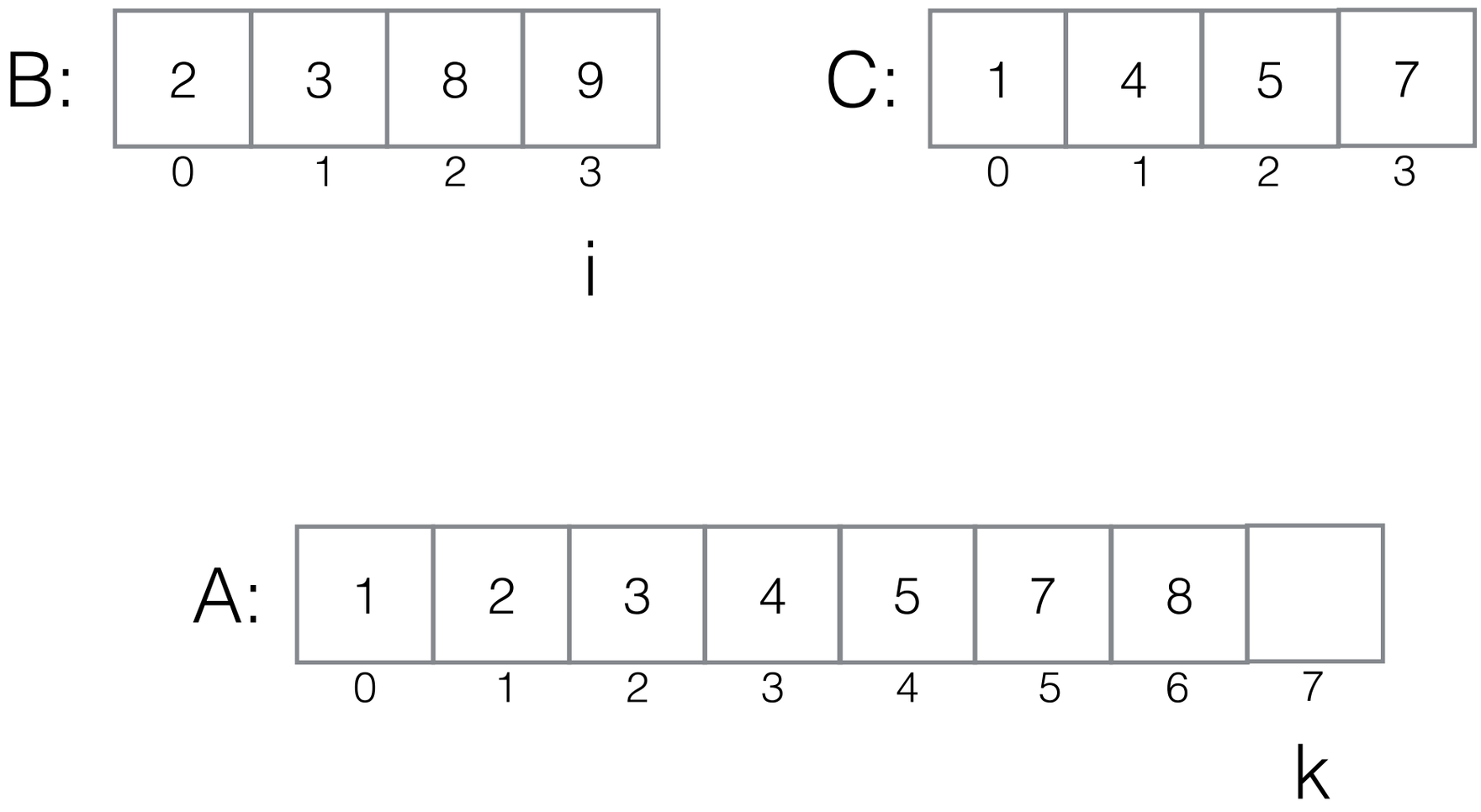


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Merging Arrays



THE UNIVERSITY OF MELBOURNE

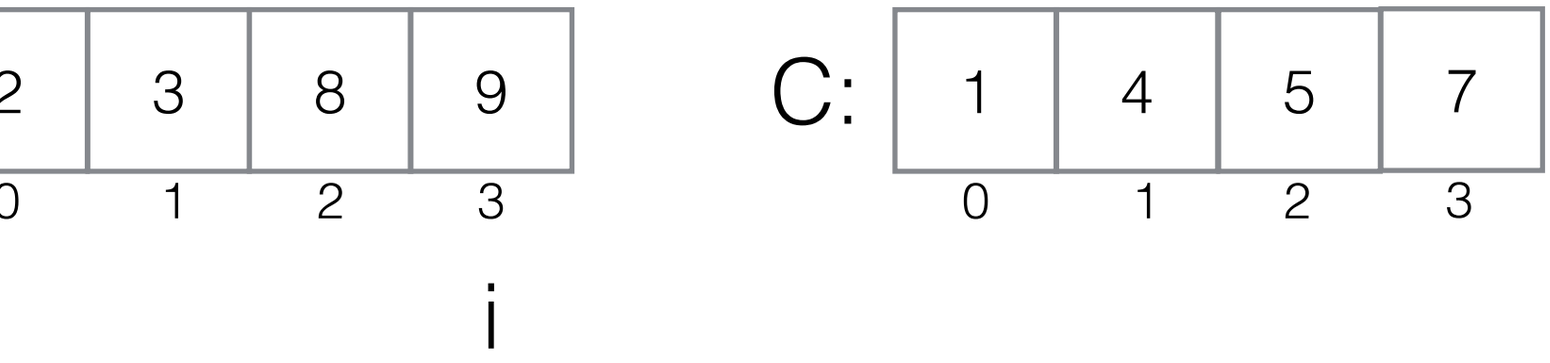


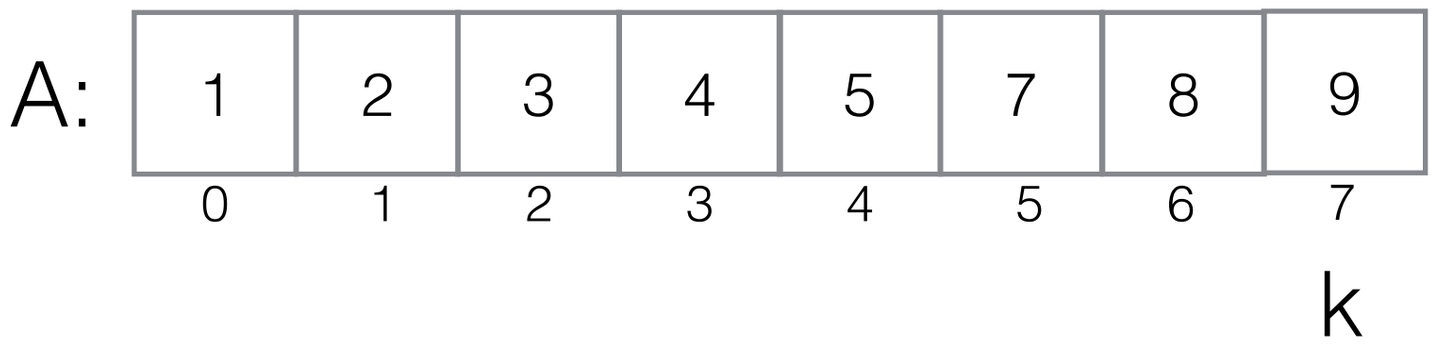
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Merging Arrays



THE UNIVERSITY OF MELBOURNE





Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort Analysis



THE UNIVERSITY OF MELBOURNE

procedure MERGESORT($A[\cdot], n$) ▷ Sort $A[0]..A[n-1]$
  if $n > 1$ **then**
    for $i \leftarrow 0$ **to** $\lfloor n/2 \rfloor - 1$ **do** ▷ Copy left half of A to B
      $B[i] \leftarrow A[i]$
    for $i \leftarrow 0$ **to** $\lceil n/2 \rceil - 1$ **do** ▷ Copy right half of A to C
      $C[i] \leftarrow A[\lfloor n/2 \rfloor + i]$
    MERGESORT($B, \lfloor n/2 \rfloor$) ▷ Sort B
    MERGESORT($C, \lceil n/2 \rceil$) ▷ Sort C
    MERGE($B, \lfloor n/2 \rfloor, C, \lceil n/2 \rceil, A$) ▷ Merge B and C into A

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Merge: Analysis



THE UNIVERSITY OF MELBOURNE

procedure MERGE($B[\cdot], p, C[\cdot], q, A[\cdot]$)
    $i \leftarrow 0; j \leftarrow 0; k \leftarrow 0$
    while $i < p$ and $j < q$ **do**
        if $B[i] \leq C[j]$ **then**
            $A[k] \leftarrow B[i]$
            $i \leftarrow i + 1$
        else
            $A[k] \leftarrow C[j]$
            $j \leftarrow j + 1$
        $k \leftarrow k + 1$
if $i = p$ **then**
    copy $C[j]..C[q-1]$ to $A[k]..A[p+q-1]$ ▷ (a for loop)
else
    copy $B[i]..B[p-1]$ to $A[k]..A[p+q-1]$ ▷ (a for loop)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Analysis



THE UNIVERSITY OF MELBOURNE

- How many comparisons will MERGE need to make in the worst case, when given arrays of size ⌊n/2⌋ and ⌈n/2⌉ ?

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Analysis

- How many comparisons will MERGE need to make in the worst case, when given arrays of size ⌊n/2⌋ and ⌈n/2⌉ ?

- If the largest and second-largest elements are in different arrays, then n – 1 comparisons. Hence the cost equation for Mergesort is

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Analysis

- How many comparisons will MERGE need to make in the worst case, when given arrays of size ⌊n/2⌋ and ⌈n/2⌉ ?

- If the largest and second-largest elements are in different arrays, then n – 1 comparisons. Hence the cost equation for Mergesort is

$$C(n) = \begin{cases} 0 & \text{if } n < 2 \\ 2C(n/2) + n - 1 & \text{otherwise} \end{cases}$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Analysis

- How many comparisons will MERGE need to make in the worst case, when given arrays of size ⌊n/2⌋ and ⌈n/2⌉ ?

- If the largest and second-largest elements are in different arrays, then n – 1 comparisons. Hence the cost equation for Mergesort is

$$C(n) = \begin{cases} 0 & \text{if } n < 2 \\ 2C(n/2) + n - 1 & \text{otherwise} \end{cases}$$

- By the Master Theorem, C(n) ∈ Θ(n log n).

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Properties



THE UNIVERSITY OF MELBOURNE

- For large n, the number of comparisons made tends to be around 75% of the worst-case scenario.
- Is mergesort in-place?
- Is mergesort stable?

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Properties

- For large n, the number of comparisons made tends to be around 75% of the worst-case scenario.

- Is mergesort in-place? no

- Is mergesort stable?

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Properties

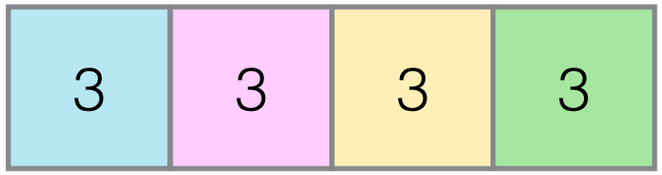
- For large n, the number of comparisons made tends to be around 75% of the worst-case scenario.

- Is mergesort in-place? no

- Is mergesort stable? ?

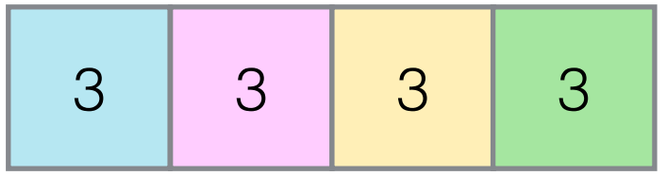
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

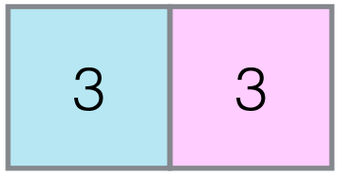
Mergesort: Stability



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

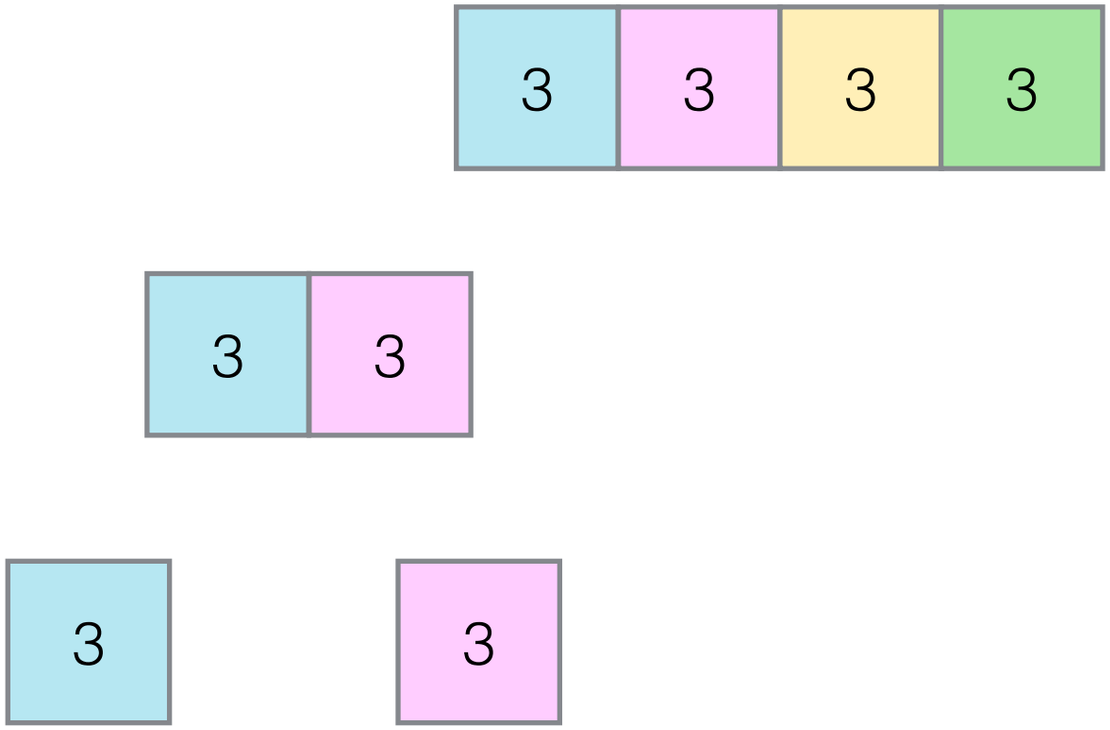
Mergesort: Stability





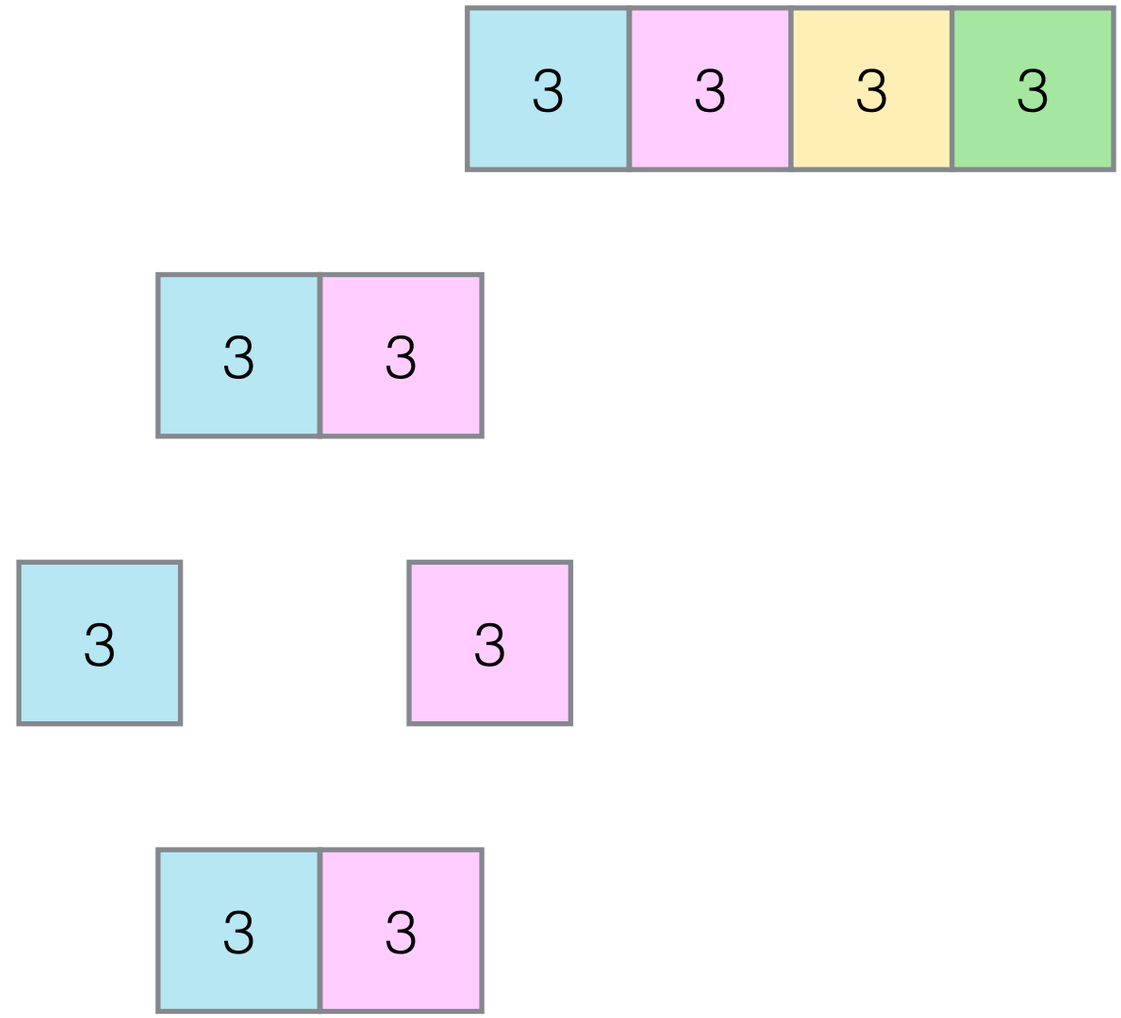
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Stability



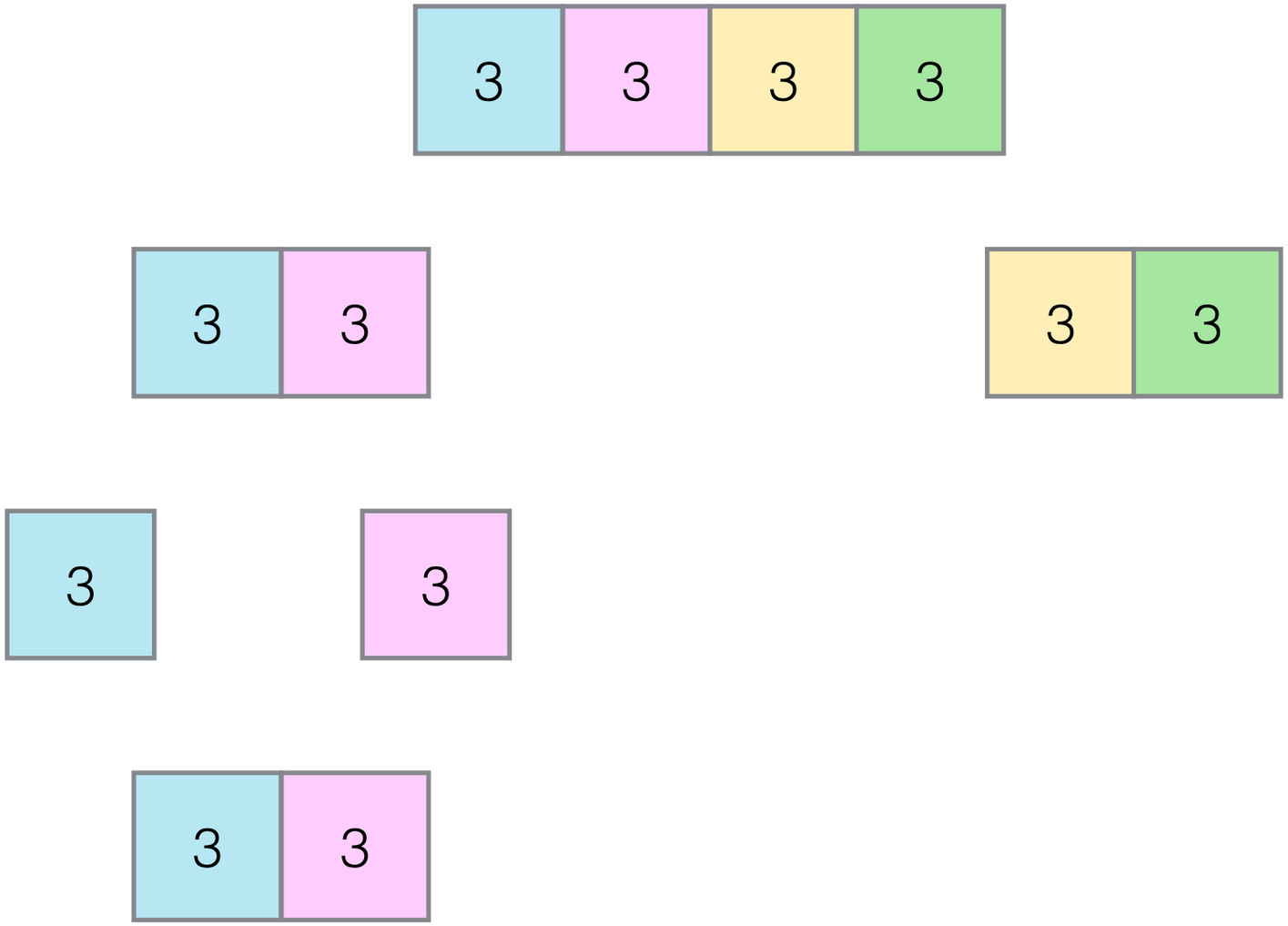
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Stability



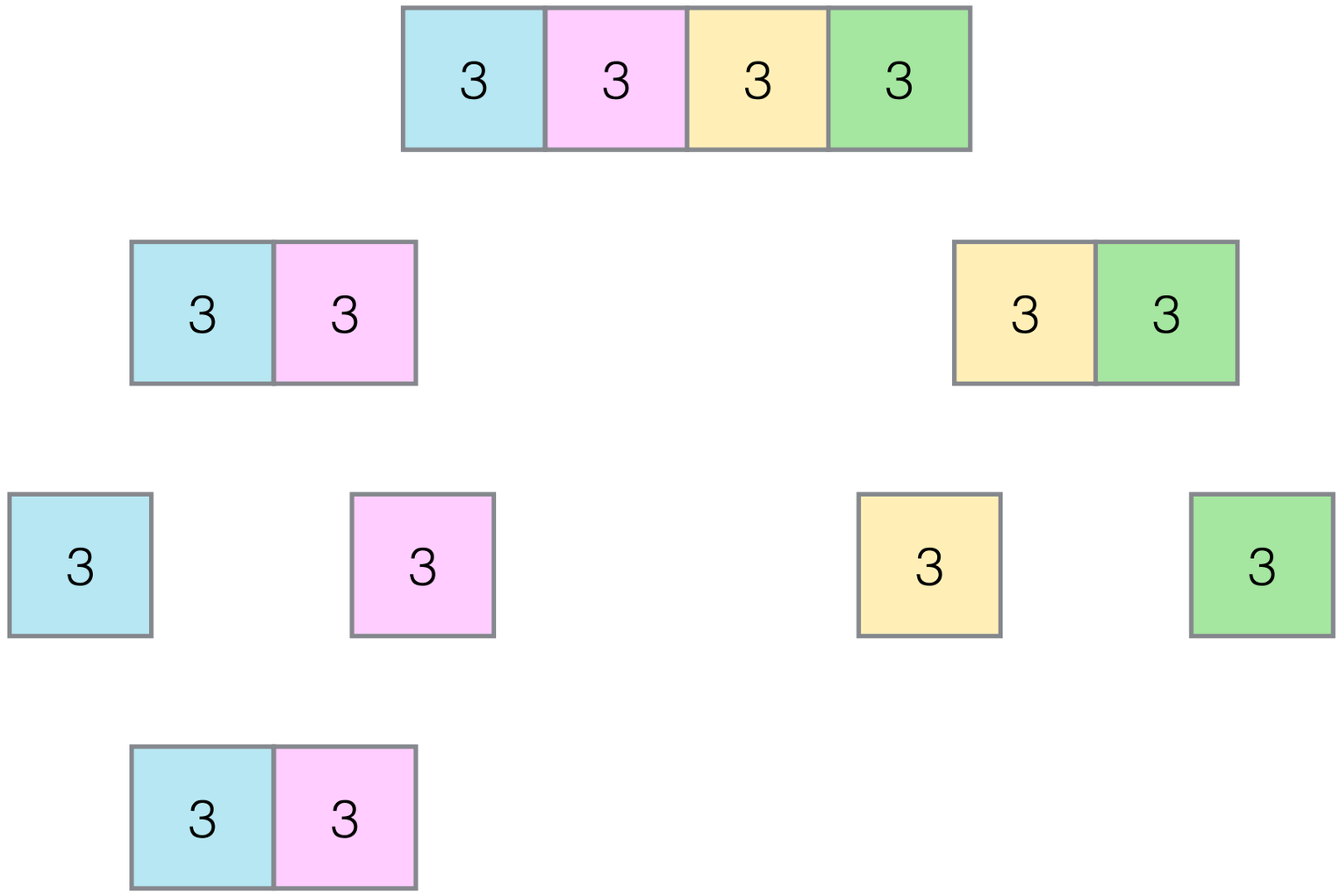
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Stability



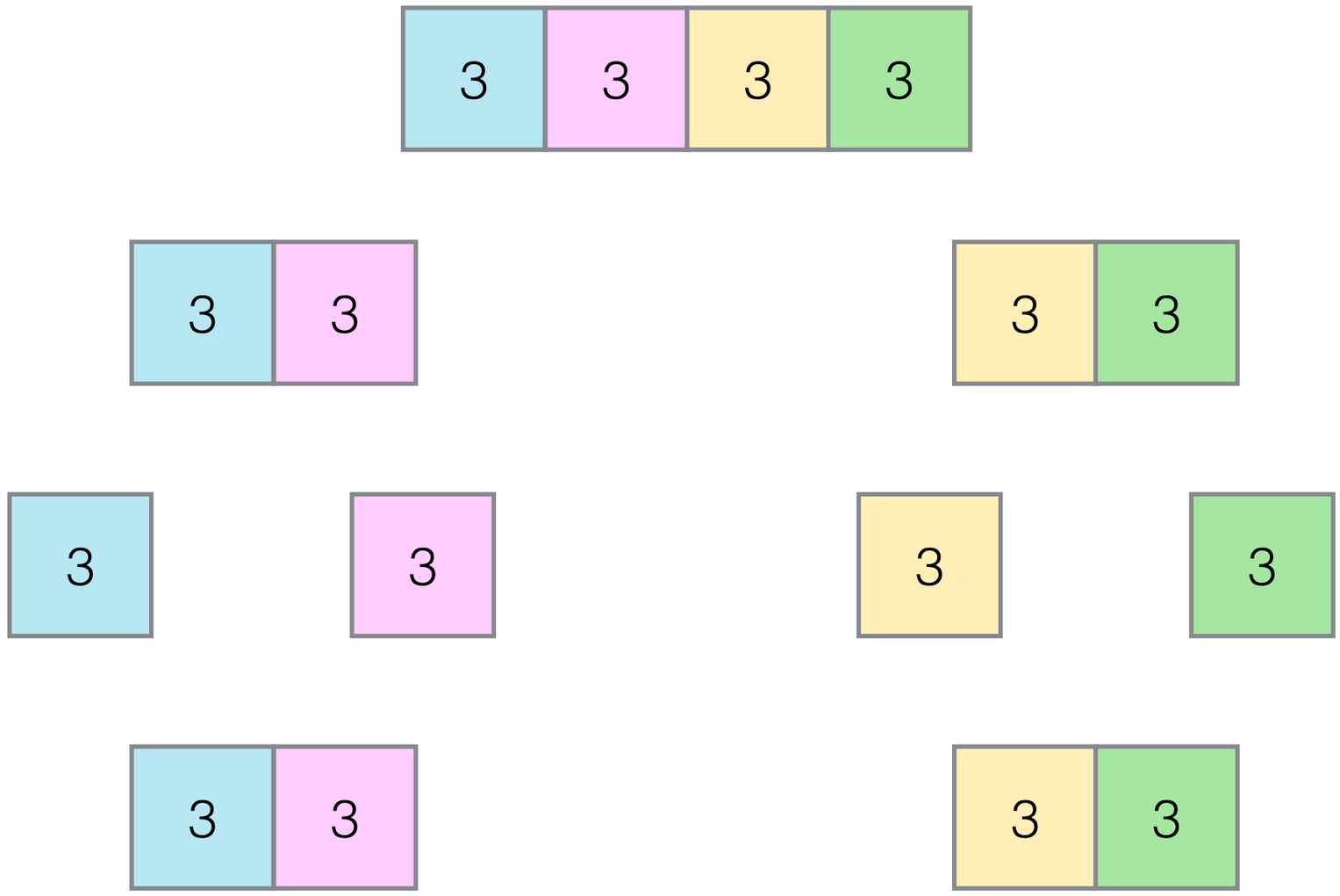
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Stability



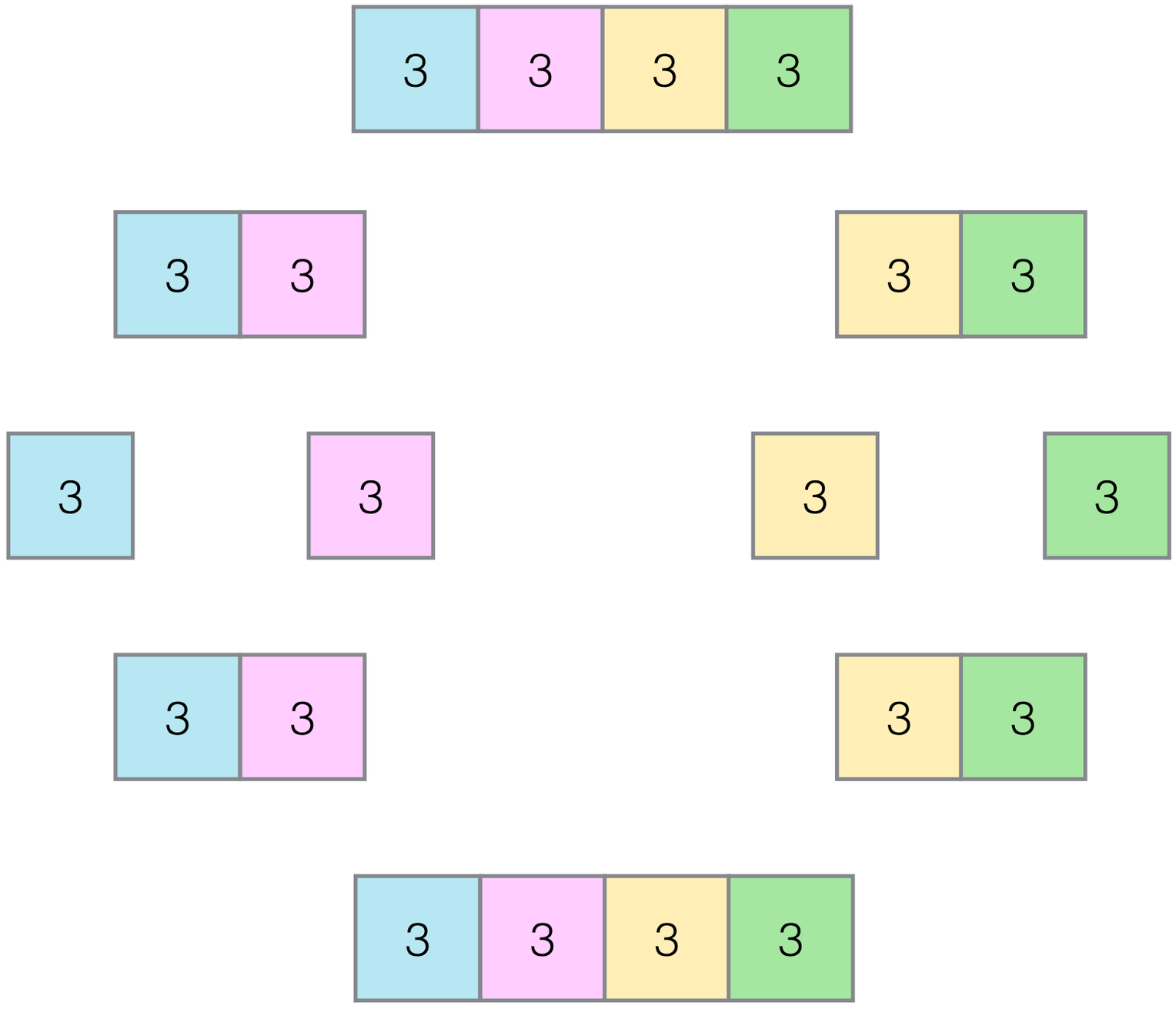
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Stability



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Stability



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Properties



THE UNIVERSITY OF MELBOURNE

no

yes

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Properties

- For large n, the number of comparisons made tends to be around 75% of the worst-case scenario.

- Is mergesort in-place? no

- Is mergesort stable? yes

- If comparisons are fast, mergesort ranks between quicksort and heapsort (covered next week) for time, assuming random data.

- Mergesort is the method of choice for linked lists and for very large collections of data.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Mergesort: Properties



THE UNIVERSITY OF MELBOURNE

- For large n, the number of comparisons made tends to be around 75% of the worst-case scenario.
- Is mergesort in-place?
- Is mergesort stable?

procedure MERGE($B[\cdot], p, C[\cdot], q, A[\cdot]$)

if $i = p$ **then**
    copy $C[j]..C[q-1]$ to $A[k]..A[p+q-1]$ ▷ (a for loop)
else
    copy $B[i]..B[p-1]$ to $A[k]..A[p+q-1]$ ▷ (a for loop)

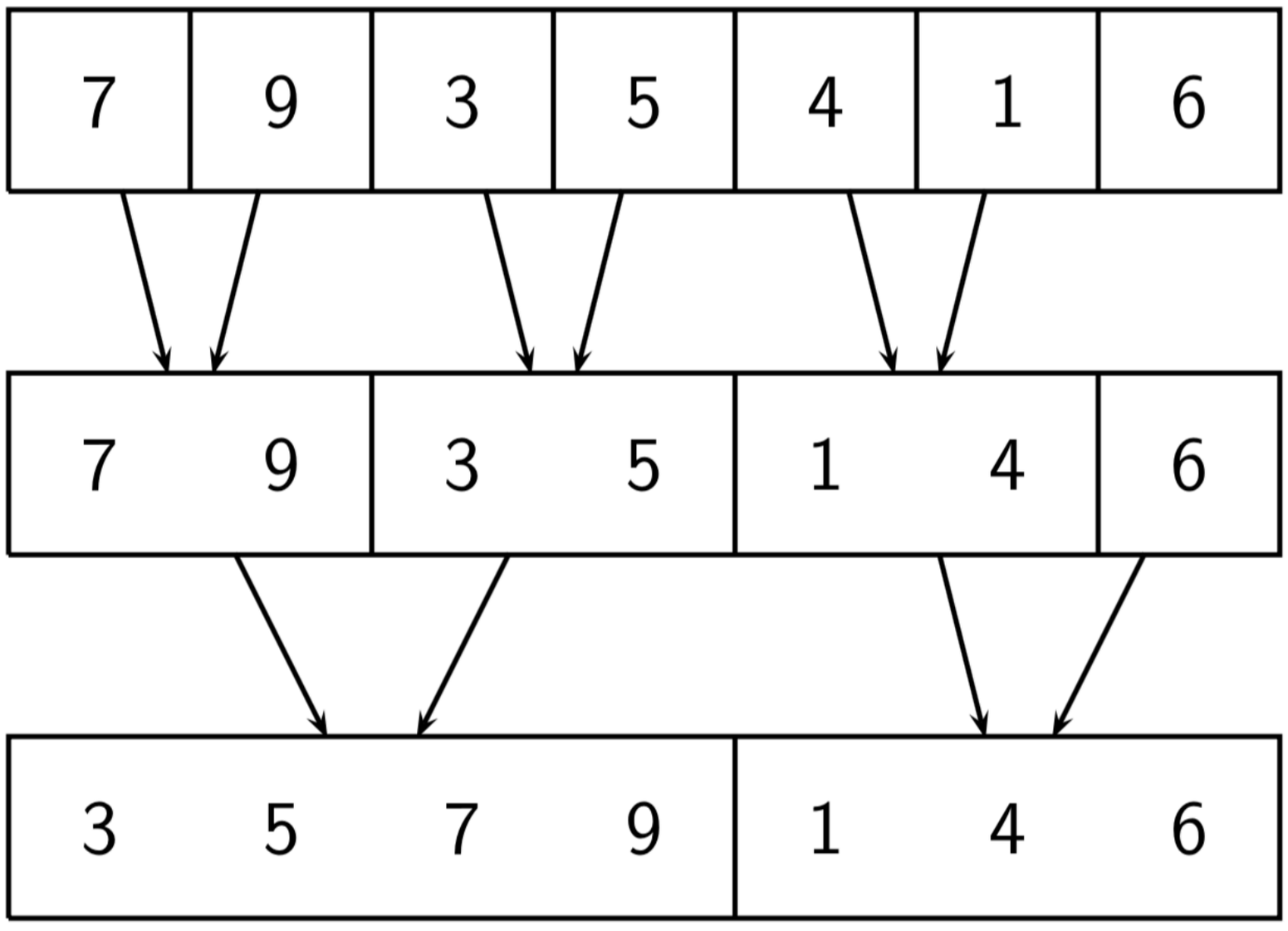
- Mergesort is the method of choice for linked lists and for very large collections of data.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Bottom-Up Mergesort

- An alternative way of doing mergesort:

- Generate **runs** of length 2, then of length 4, and so on:



Copyright University of Melbourne 2020, provided under Creative Commons Attribution License

Quicksort



https://en.wikipedia.org/wiki/Tony_Hoare

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort

- Quicksort takes a divide-and-conquer approach that is different to mergesort's.

- It uses the **partitioning** idea from QuickSelect, picking a pivot element, and partitioning the array around that, so as to obtain this situation:

$$\underbrace{A[0] \ldots A[s-1]}_{\text{all are } \leq A[s]} \quad A[s] \quad \underbrace{A[s+1] \ldots A[n-1]}_{\text{all are } \geq A[s]}$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort

- Quicksort takes a divide-and-conquer approach that is different to mergesort's.

- It uses the **partitioning** idea from QuickSelect, picking a pivot element, and partitioning the array around that, so as to obtain this situation:

$$\underbrace{A[0] \ldots A[s-1]}_{\text{all are } \leq A[s]} \quad A[s] \quad \underbrace{A[s+1] \ldots A[n-1]}_{\text{all are } \geq A[s]}$$

- The element A[s] will be in its final position (it is the (s + 1)th smallest element).

- All that then needs to be done is to sort the segment to the left, recursively, as well as the segment to the right.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort

- Very short and elegant:

procedure QUICKSORT($A[\cdot], lo, hi$)
  if $lo < hi$ **then**
    $s \leftarrow$ PARTITION(A, lo, hi)
    QUICKSORT($A, lo, s-1$)
    QUICKSORT($A, s+1, hi$)

- Initial call: Quicksort(A, 0, n – 1).

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort: Example



THE UNIVERSITY OF MELBOURNE

procedure QUICKSORT($A[\cdot], lo, hi$)
  if $lo < hi$ **then**
    $s \leftarrow$ PARTITION(A, lo, hi)
    QUICKSORT($A, lo, s-1$)
    QUICKSORT($A, s+1, hi$)

A:

9	23	8	41	22	3	37
0	1	2	3	4	5	6

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort: Example



THE UNIVERSITY OF MELBOURNE

procedure QUICKSORT($A[\cdot], lo, hi$)
  if $lo < hi$ **then**
    $s \leftarrow$ PARTITION(A, lo, hi)
    QUICKSORT($A, lo, s-1$)
    QUICKSORT($A, s+1, hi$)

A:

9	23	8	41	22	3	37
0	1	2	3	4	5	6

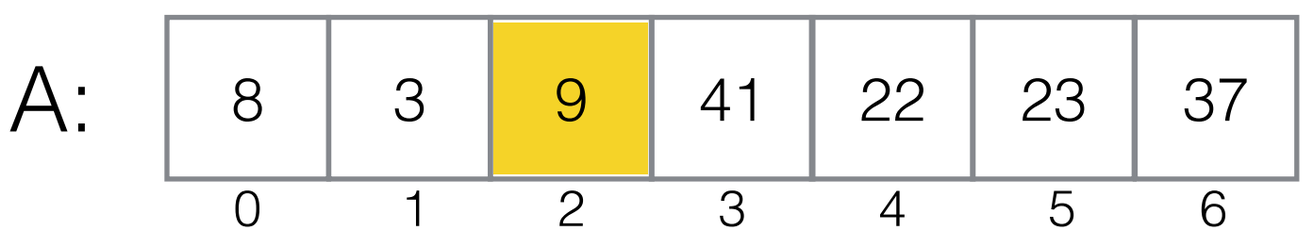
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort: Example



THE UNIVERSITY OF MELBOURNE

procedure QUICKSORT($A[\cdot], lo, hi$)
  if $lo < hi$ **then**
    $s \leftarrow$ PARTITION(A, lo, hi)
    QUICKSORT($A, lo, s-1$)
    QUICKSORT($A, s+1, hi$)



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort: Example



THE UNIVERSITY OF MELBOURNE

procedure QUICKSORT($A[\cdot], lo, hi$)
  if $lo < hi$ **then**
    $s \leftarrow$ PARTITION(A, lo, hi)
    QUICKSORT($A, lo, s-1$)
    QUICKSORT($A, s+1, hi$)

A:

8	3	9	41	22	23	37
0	1	2	3	4	5	6

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort: Example



THE UNIVERSITY OF MELBOURNE

procedure QUICKSORT($A[\cdot], lo, hi$)
  if $lo < hi$ **then**
    $s \leftarrow$ PARTITION(A, lo, hi)
    QUICKSORT($A, lo, s-1$)
    QUICKSORT($A, s+1, hi$)



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort: Example



THE UNIVERSITY OF MELBOURNE

procedure QUICKSORT($A[\cdot], lo, hi$)
  if $lo < hi$ **then**
    $s \leftarrow$ PARTITION(A, lo, hi)
    QUICKSORT($A, lo, s-1$)
    QUICKSORT($A, s+1, hi$)



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort: Example



THE UNIVERSITY OF MELBOURNE

procedure QUICKSORT($A[\cdot], lo, hi$)
  if $lo < hi$ **then**
    $s \leftarrow$ PARTITION(A, lo, hi)
    QUICKSORT($A, lo, s-1$)
    QUICKSORT($A, s+1, hi$)



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort: Example



THE UNIVERSITY OF MELBOURNE

procedure QUICKSORT($A[\cdot], lo, hi$)
  if $lo < hi$ **then**
    $s \leftarrow$ PARTITION(A, lo, hi)
    QUICKSORT($A, lo, s-1$)
    QUICKSORT($A, s+1, hi$)

A:

3	8	9	41	22	23	37
0	1	2	3	4	5	6

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort: Example



THE UNIVERSITY OF MELBOURNE

procedure QUICKSORT($A[\cdot], lo, hi$)
  if $lo < hi$ **then**
    $s \leftarrow$ PARTITION(A, lo, hi)
    QUICKSORT($A, lo, s-1$)
    QUICKSORT($A, s+1, hi$)

A:

0	1	2	3	4	5	6
3	8	9	41	22	23	37

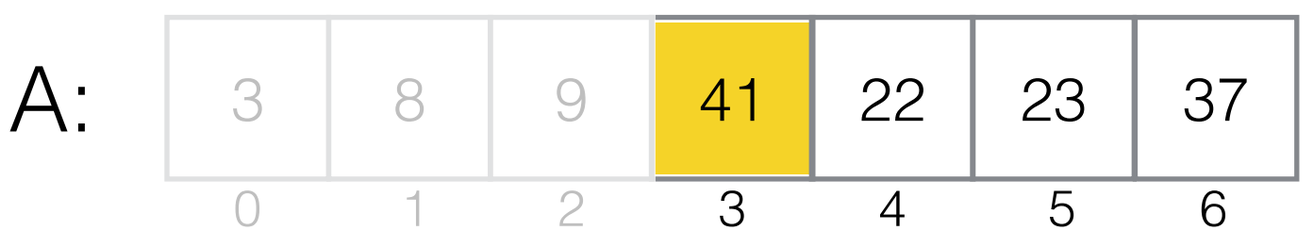
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort: Example



THE UNIVERSITY OF MELBOURNE

procedure QUICKSORT($A[\cdot], lo, hi$)
  if $lo < hi$ **then**
    $s \leftarrow$ PARTITION(A, lo, hi)
    QUICKSORT($A, lo, s-1$)
    QUICKSORT($A, s+1, hi$)



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort: Example



THE UNIVERSITY OF MELBOURNE

procedure QUICKSORT($A[\cdot], lo, hi$)
  if $lo < hi$ **then**
    $s \leftarrow$ PARTITION(A, lo, hi)
    QUICKSORT($A, lo, s-1$)
    QUICKSORT($A, s+1, hi$)



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort: Example



THE UNIVERSITY OF MELBOURNE

procedure QUICKSORT($A[\cdot], lo, hi$)
  if $lo < hi$ **then**
    $s \leftarrow$ PARTITION(A, lo, hi)
    QUICKSORT($A, lo, s-1$)
    QUICKSORT($A, s+1, hi$)

A:

0	1	2	3	4	5	6
3	8	9	37	22	23	41

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort: Example



THE UNIVERSITY OF MELBOURNE

procedure QUICKSORT($A[\cdot], lo, hi$)
  if $lo < hi$ **then**
    $s \leftarrow$ PARTITION(A, lo, hi)
    QUICKSORT($A, lo, s-1$)
    QUICKSORT($A, s+1, hi$)

A:

0	1	2	3	4	5	6
3	8	9	37	22	23	41

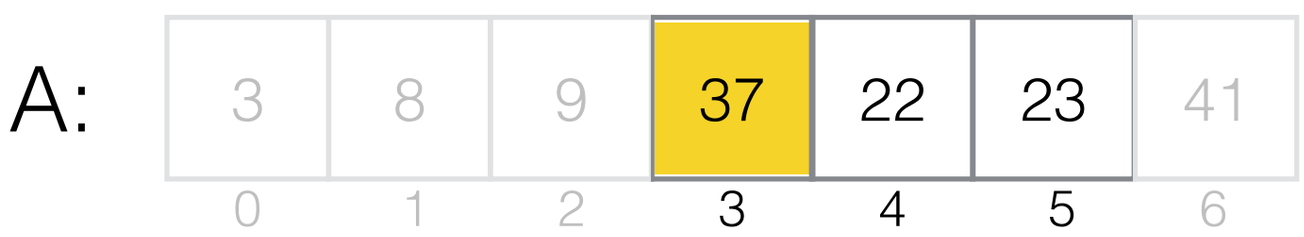
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort: Example



THE UNIVERSITY OF MELBOURNE

procedure QUICKSORT($A[\cdot], lo, hi$)
  if $lo < hi$ **then**
    $s \leftarrow$ PARTITION(A, lo, hi)
    QUICKSORT($A, lo, s-1$)
    QUICKSORT($A, s+1, hi$)



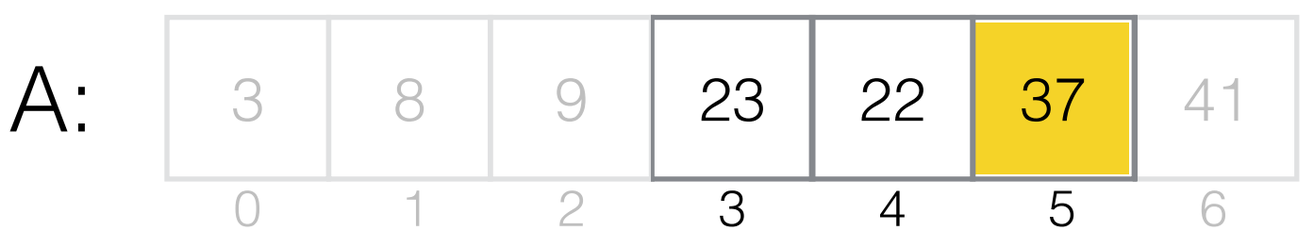
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort: Example



THE UNIVERSITY OF MELBOURNE

procedure QUICKSORT($A[\cdot], lo, hi$)
  if $lo < hi$ **then**
    $s \leftarrow$ PARTITION(A, lo, hi)
    QUICKSORT($A, lo, s-1$)
    QUICKSORT($A, s+1, hi$)



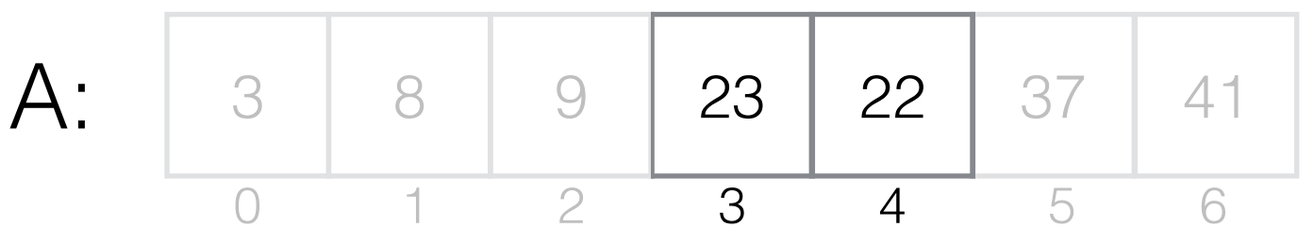
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort: Example



THE UNIVERSITY OF MELBOURNE

procedure QUICKSORT($A[\cdot], lo, hi$)
  if $lo < hi$ **then**
    $s \leftarrow$ PARTITION(A, lo, hi)
    QUICKSORT($A, lo, s-1$)
    QUICKSORT($A, s+1, hi$)



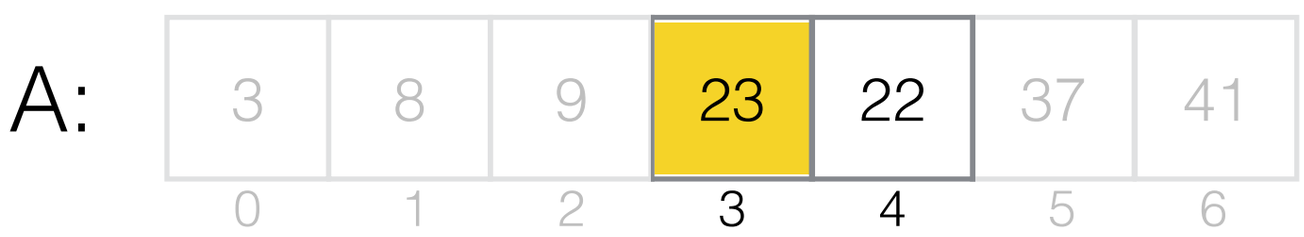
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort: Example



THE UNIVERSITY OF MELBOURNE

procedure QUICKSORT($A[\cdot], lo, hi$)
  if $lo < hi$ **then**
    $s \leftarrow$ PARTITION(A, lo, hi)
    QUICKSORT($A, lo, s-1$)
    QUICKSORT($A, s+1, hi$)



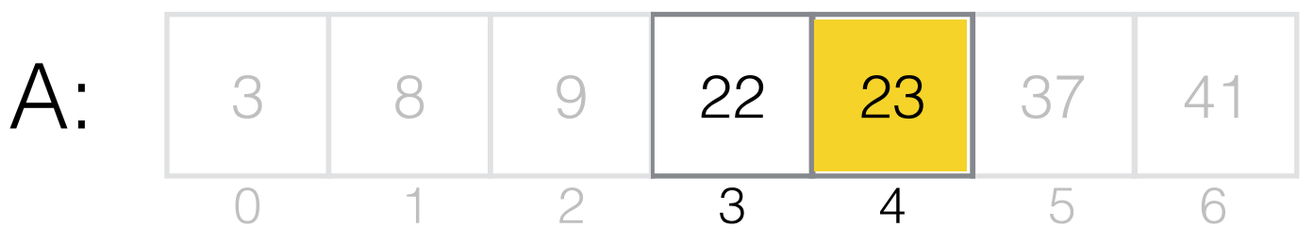
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort: Example



THE UNIVERSITY OF MELBOURNE

procedure QUICKSORT($A[\cdot], lo, hi$)
  if $lo < hi$ **then**
    $s \leftarrow$ PARTITION(A, lo, hi)
    QUICKSORT($A, lo, s-1$)
    QUICKSORT($A, s+1, hi$)



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort: Example



THE UNIVERSITY OF MELBOURNE

procedure QUICKSORT($A[\cdot], lo, hi$)
  if $lo < hi$ **then**
    $s \leftarrow$ PARTITION(A, lo, hi)
    QUICKSORT($A, lo, s-1$)
    QUICKSORT($A, s+1, hi$)

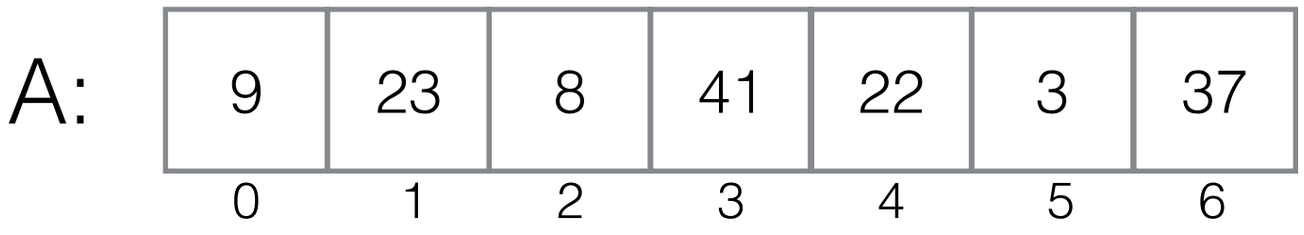
A:

3	8	9	22	23	37	41
0	1	2	3	4	5	6

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Hoare Partitioning

- The standard way of doing partitioning in Quicksort



p: 9

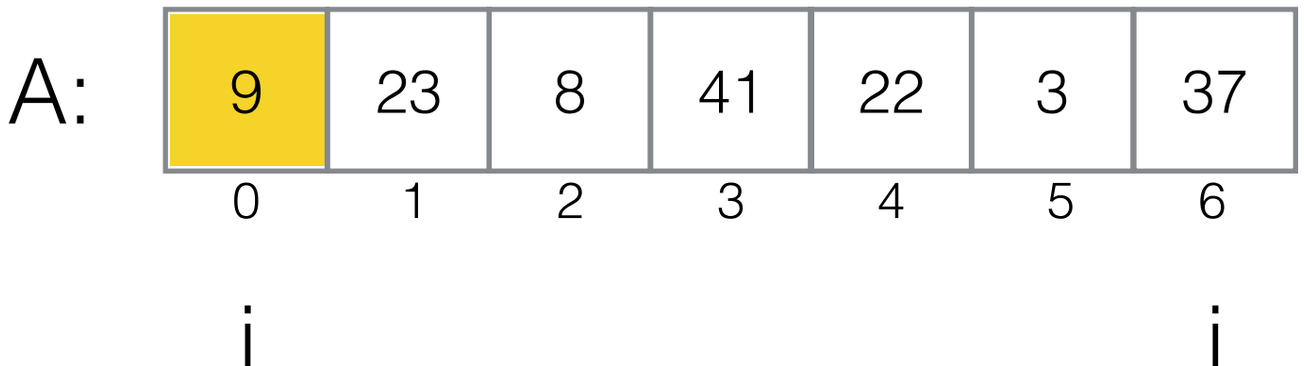
i

j

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Hoare Partitioning

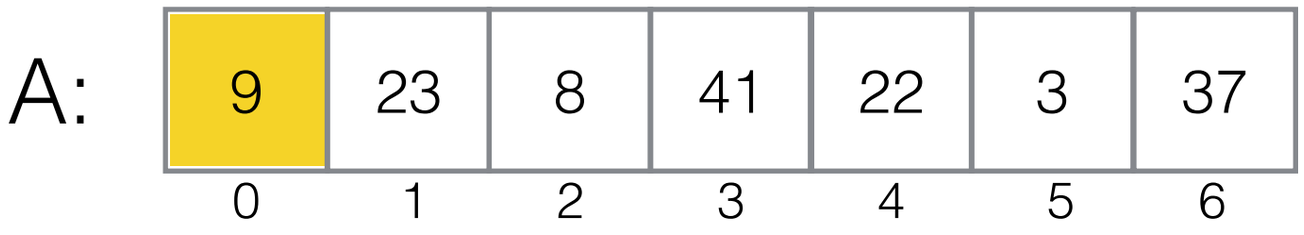
- The standard way of doing partitioning in Quicksort



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Hoare Partitioning

- The standard way of doing partitioning in Quicksort



p: 9

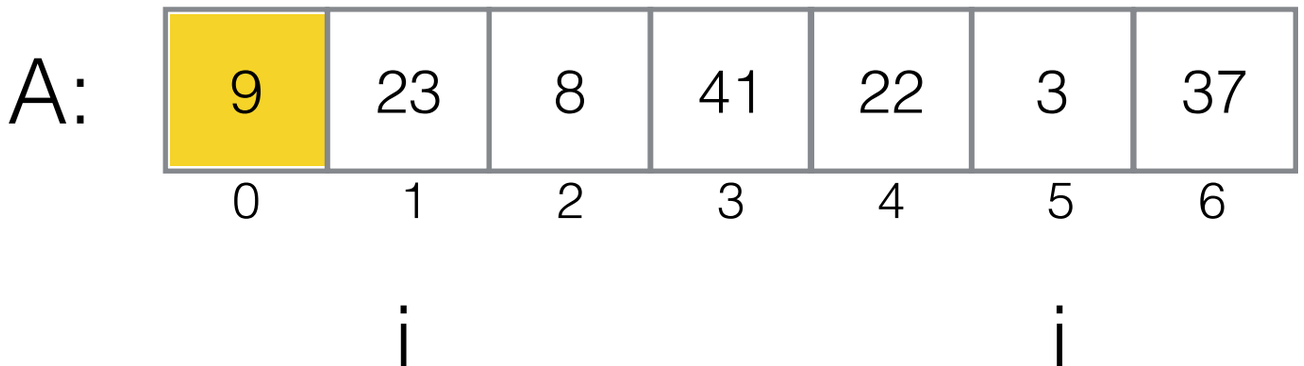
i

j

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Hoare Partitioning

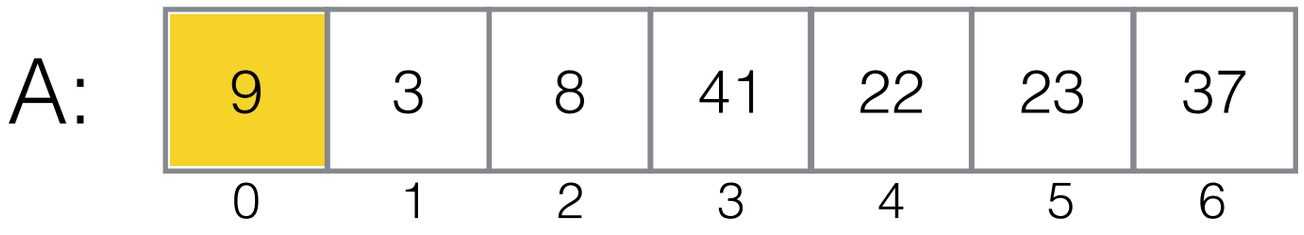
- The standard way of doing partitioning in Quicksort



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Hoare Partitioning

- The standard way of doing partitioning in Quicksort



p: 9

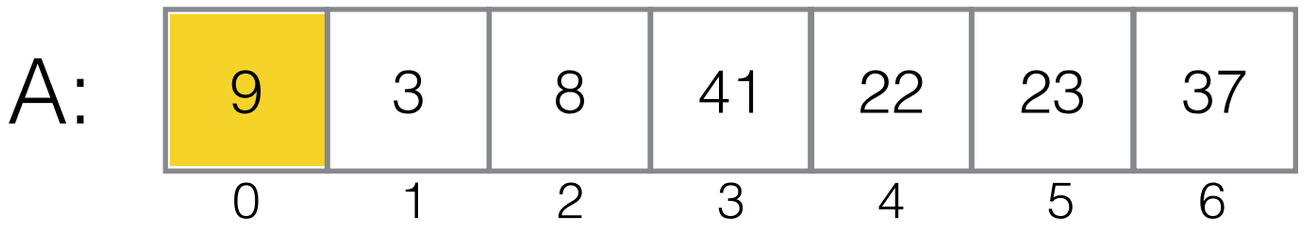
i

j

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Hoare Partitioning

- The standard way of doing partitioning in Quicksort



p: 9

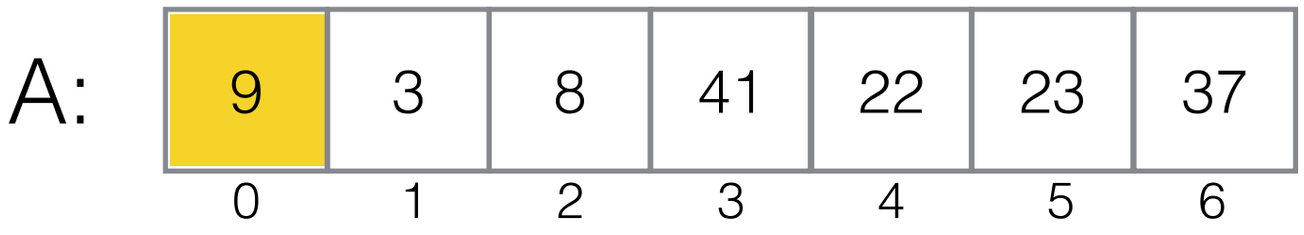
i

j

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Hoare Partitioning

- The standard way of doing partitioning in Quicksort



p: 9

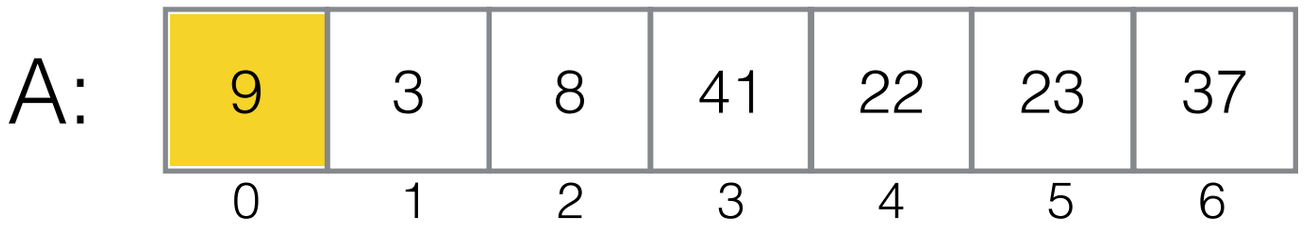
i

j

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Hoare Partitioning

- The standard way of doing partitioning in Quicksort



p: 9

i j

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Hoare Partitioning



THE UNIVERSITY OF MELBOURNE

function PARTITION($A[\cdot], lo, hi$)
    $p \leftarrow A[lo]$; $i \leftarrow lo$; $j \leftarrow hi$
    repeat
        while $i < hi$ and $A[i] \leq p$ **do** $i \leftarrow i + 1$
        while $j \geq lo$ and $A[j] > p$ **do** $j \leftarrow j - 1$
        $swap(A[i], A[j])$
    until $i \geq j$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Hoare Partitioning



THE UNIVERSITY OF MELBOURNE

function PARTITION($A[\cdot], lo, hi$)
    $p \leftarrow A[lo]$; $i \leftarrow lo$; $j \leftarrow hi$
    repeat
        while $i < hi$ and $A[i] \leq p$ **do** $i \leftarrow i + 1$
        while $j \geq lo$ and $A[j] > p$ **do** $j \leftarrow j - 1$
        swap$(A[i], A[j])$
    until $i \geq j$

A:

9	3	8	41	22	23	37
0	1	2	3	4	5	6

i j

p: 9

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Hoare Partitioning



THE UNIVERSITY OF MELBOURNE

function PARTITION($A[\cdot], lo, hi$)
    $p \leftarrow A[lo]$; $i \leftarrow lo$; $j \leftarrow hi$
    repeat
        while $i < hi$ and $A[i] \leq p$ **do** $i \leftarrow i + 1$
        while $j \geq lo$ and $A[j] > p$ **do** $j \leftarrow j - 1$
        $swap(A[i], A[j])$
    until $i \geq j$

A:

9	3	8	41	22	23	37
0	1	2	3	4	5	6

p: 9

i
j

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Hoare Partitioning



THE UNIVERSITY OF MELBOURNE

function PARTITION($A[\cdot], lo, hi$)
    $p \leftarrow A[lo]$; $i \leftarrow lo$; $j \leftarrow hi$
    repeat
        while $i < hi$ and $A[i] \leq p$ **do** $i \leftarrow i + 1$
        while $j \geq lo$ and $A[j] > p$ **do** $j \leftarrow j - 1$
        $swap(A[i], A[j])$
    until $i \geq j$

A:

9	3	8	41	22	23	37
0	1	2	3	4	5	6

j i

p: 9

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Hoare Partitioning



THE UNIVERSITY OF MELBOURNE

function PARTITION($A[\cdot], lo, hi$)
    $p \leftarrow A[lo]$; $i \leftarrow lo$; $j \leftarrow hi$
    repeat
        while $i < hi$ and $A[i] \leq p$ **do** $i \leftarrow i + 1$
        while $j \geq lo$ and $A[j] > p$ **do** $j \leftarrow j - 1$
        $swap(A[i], A[j])$
until $i \geq j$

A:

9	3	41	8	22	23	37
0	1	2	3	4	5	6

j i

p: 9

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Hoare Partitioning



THE UNIVERSITY OF MELBOURNE

function PARTITION($A[\cdot], lo, hi$)
    $p \leftarrow A[lo]$; $i \leftarrow lo$; $j \leftarrow hi$
    repeat
        while $i < hi$ and $A[i] \leq p$ **do** $i \leftarrow i + 1$
        while $j \geq lo$ and $A[j] > p$ **do** $j \leftarrow j - 1$
        $swap(A[i], A[j])$
until $i \geq j$
$swap(A[i], A[j])$
$swap(A[lo], A[j])$
return j

A:

9	3	8	41	22	23	37
0	1	2	3	4	5	6

j i

p: 9

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Hoare Partitioning



THE UNIVERSITY OF MELBOURNE

function PARTITION($A[\cdot], lo, hi$)
    $p \leftarrow A[lo]$; $i \leftarrow lo$; $j \leftarrow hi$
    repeat
        while $i < hi$ and $A[i] \leq p$ **do** $i \leftarrow i + 1$
        while $j \geq lo$ and $A[j] > p$ **do** $j \leftarrow j - 1$
        $swap(A[i], A[j])$
until $i \geq j$
$swap(A[i], A[j])$
$swap(A[lo], A[j])$
return j

A:

8	3	9	41	22	23	37
0	1	2	3	4	5	6

j i

p: 9

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Analysis: Best Case Analysis



THE UNIVERSITY OF MELBOURNE

$$C_{best}(n) = \begin{cases} 0 & \text{if } n < 2 \\ 2C_{best}(n/2) + n & \text{otherwise} \end{cases}$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Analysis: Best Case Analysis



THE UNIVERSITY OF MELBOURNE

- The best case happens when the pivot is the median; that results in two sub-tasks of equal size.

$$C_{best}(n) = \begin{cases} 0 & \text{if } n < 2 \\ 2C_{best}(n/2) + n & \text{otherwise} \end{cases}$$

The 'n' is for the n key comparisons performed by Partition.

- By the Master Theorem, $C_{best}(n) \in \Theta(n \log n)$, just as for mergesort, so quicksort's best case is (asymptotically) no better than mergesort's worst case.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Worst Case



THE UNIVERSITY OF MELBOURNE

A:

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Worst Case



THE UNIVERSITY OF MELBOURNE

A:

4	9	13	22	41	83	96
0	1	2	3	4	5	6

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Analysis: Worst Case Analysis



$$C_{worst}(n) = \begin{cases} 0 & \text{if } n < 2 \\ C_{worst}(n-1) + n & \text{otherwise} \end{cases}$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Analysis: Worst Case Analysis



THE UNIVERSITY OF MELBOURNE

- The worst case happens if the array is already sorted.

- In that case, we don't really have divide-and-conquer, because each recursive call deals with a problem size that has only been decremented by 1:

$$C_{worst}(n) = \begin{cases} 0 & \text{if } n < 2 \\ C_{worst}(n-1) + n & \text{otherwise} \end{cases}$$

- That is, $C_{worst}(n) = n + (n - 1) + \cdots + 3 + 2 \in \Theta(n^2)$.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Improvements: Median-of-Three

- It would be better if the pivot was chosen randomly.

- A cheap and useful approximation to this is to take the median of three candidates:

A[lo] A[⌊(lo + hi)/2⌋] A[hi]



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Improvements: Median-of-Three

- It would be better if the pivot was chosen randomly.

- A cheap and useful approximation to this is to take the median of three candidates:

A[lo] A[⌊(lo + hi)/2⌋] A[hi]



- Reorganise the three elements so that p_2 is the median, and p_3 is the largest of the three.

- Now run quicksort as before.

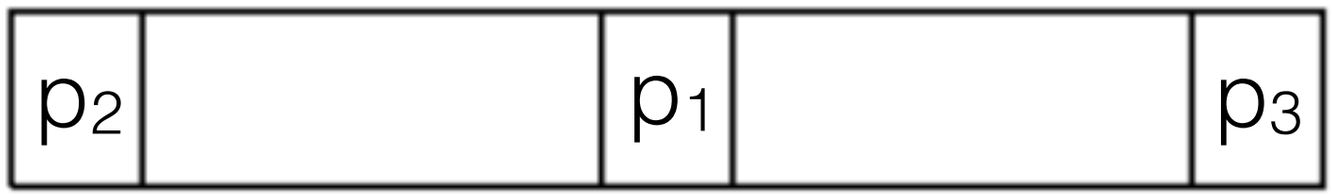
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Improvements: Median-of-Three

- It would be better if the pivot was chosen randomly.

- A cheap and useful approximation to this is to take the median of three candidates:

A[lo] A[⌊(lo + hi)/2⌋] A[hi]



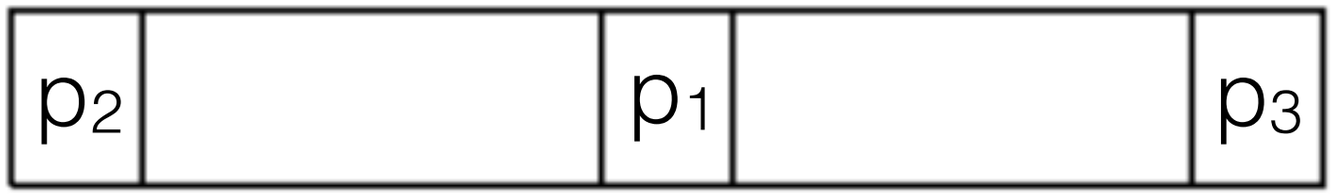
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Improvements: Median-of-Three

- It would be better if the pivot was chosen randomly.

- A cheap and useful approximation to this is to take the median of three candidates:

A[lo] A[⌊(lo + hi)/2⌋] A[hi]



- Reorganise the three elements so that p_2 is the median, and p_3 is the largest of the three.

- Now run quicksort as before.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Improvements: Early Cut-Off

- A second useful improvement is to stop quicksort early and switch to insertion sort. This is easily implemented:

procedure QUICKALMOSTSORT($A[\cdot], lo, hi$)
  if $lo + 10 < hi$ **then**
    $s \leftarrow$ PARTITION(A, lo, hi)
    QUICKALMOSTSORT($A, lo, s - 1$)
    QUICKALMOSTSORT($A, s + 1, hi$)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Improvements: Early Cut-Off



THE UNIVERSITY OF MELBOURNE

- A second useful improvement is to stop quicksort early and switch to insertion sort. This is easily implemented:

procedure SORT($A[\cdot], n$)
    QUICKALMOSTSORT($A, 0, n-1$)
    INSERTIONSORT(A, n)

procedure QUICKALMOSTSORT($A[\cdot], lo, hi$)
    if $lo + 10 < hi$ **then**
        $s \leftarrow$ PARTITION(A, lo, hi)
        QUICKALMOSTSORT($A, lo, s-1$)
        QUICKALMOSTSORT($A, s+1, hi$)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Properties

- With these (and other) improvements, quicksort is considered the best available sorting method for arrays of random data.

- A major reason for its speed is the very tight inner loop in PARTITION.

- Although mergesort has a better performance guarantee, quicksort is faster on average.

- In the best case, we get $\lceil \log_2 n \rceil$ recursive levels. It can be shown that on random data, the expected number is $2 \log_e n \approx 1.38 \log_2 n$. So quicksort's average behaviour is very close to the best-case behaviour.

- Is it in-place?

- Is quicksort stable?

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Properties

- With these (and other) improvements, quicksort is considered the best available sorting method for arrays of random data.

- A major reason for its speed is the very tight inner loop in PARTITION.

- Although mergesort has a better performance guarantee, quicksort is faster on average.

- In the best case, we get $\lceil \log_2 n \rceil$ recursive levels. It can be shown that on random data, the expected number is $2 \log_e n \approx 1.38 \log_2 n$. So quicksort's average behaviour is very close to the best-case behaviour.

- Is it in-place? yes

- Is quicksort stable?

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Properties

- With these (and other) improvements, quicksort is considered the best available sorting method for arrays of random data.

- A major reason for its speed is the very tight inner loop in PARTITION.

- Although mergesort has a better performance guarantee, quicksort is faster on average.

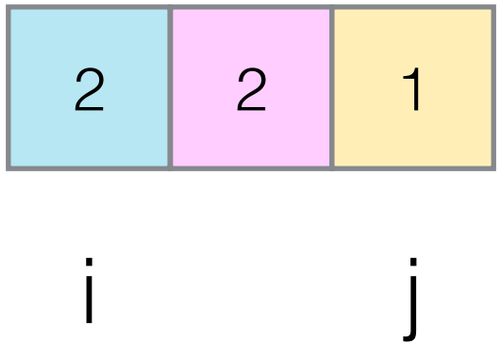
- In the best case, we get $\lceil \log_2 n \rceil$ recursive levels. It can be shown that on random data, the expected number is $2 \log_e n \approx 1.38 \log_2 n$. So quicksort's average behaviour is very close to the best-case behaviour.

- Is it in-place? yes

- Is quicksort stable? ?

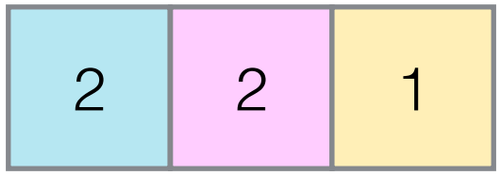
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Stability



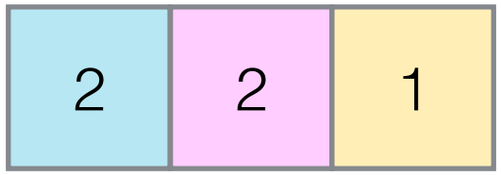
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Stability



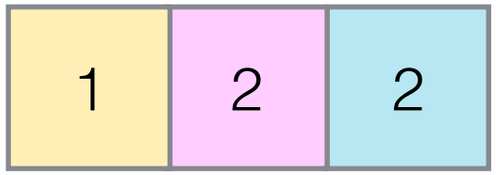
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Stability



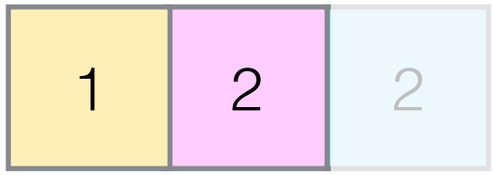
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Stability



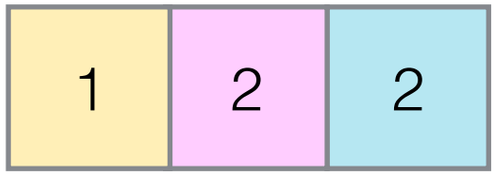
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Stability



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

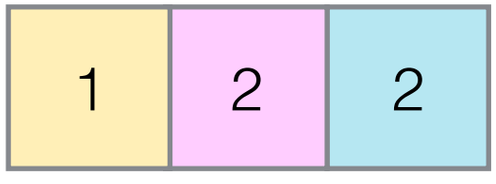
Quicksort Stability



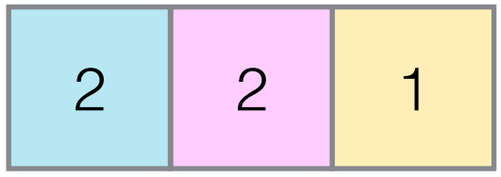
This is where we finished

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Stability



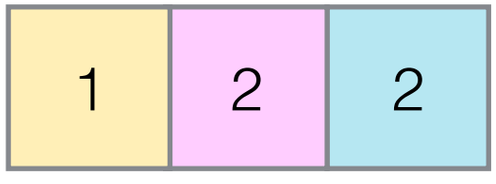
This is where we finished



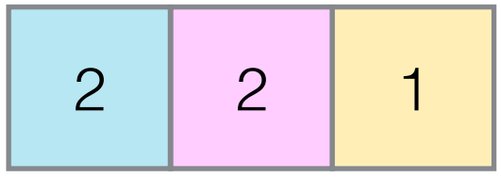
This is where we started

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Stability



This is where we finished



This is where we started

Not stable

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Properties

- With these (and other) improvements, quicksort is considered the best available sorting method for arrays of random data.

- A major reason for its speed is the very tight inner loop in PARTITION.

- Although mergesort has a better performance guarantee, quicksort is faster on average.

- In the best case, we get $\lceil \log_2 n \rceil$ recursive levels. It can be shown that on random data, the expected number is $2 \log_e n \approx 1.38 \log_2 n$. So quicksort's average behaviour is very close to the best-case behaviour.

- Is it in-place? yes

- Is quicksort stable? no

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Next up

- Tree traversal methods, plus we apply the divide-and-conquer technique to the closest-pair problem.
- See next slides for another example of Master Theorem.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 4



THE UNIVERSITY OF MELBOURNE

$$T(n) = 2T(n/2) + n^2$$

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 4



THE UNIVERSITY OF MELBOURNE

$T(n) = 2T(n/2) + n^2$ a = 2, b = 2, d = 2

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 4



THE UNIVERSITY OF MELBOURNE

$T(n) = 2T(n/2) + n^2$

a = 2, b = 2, d = 2

$a < b^d$

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

So, by the Master Theorem, $T(n) \in \Theta(n^2)$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

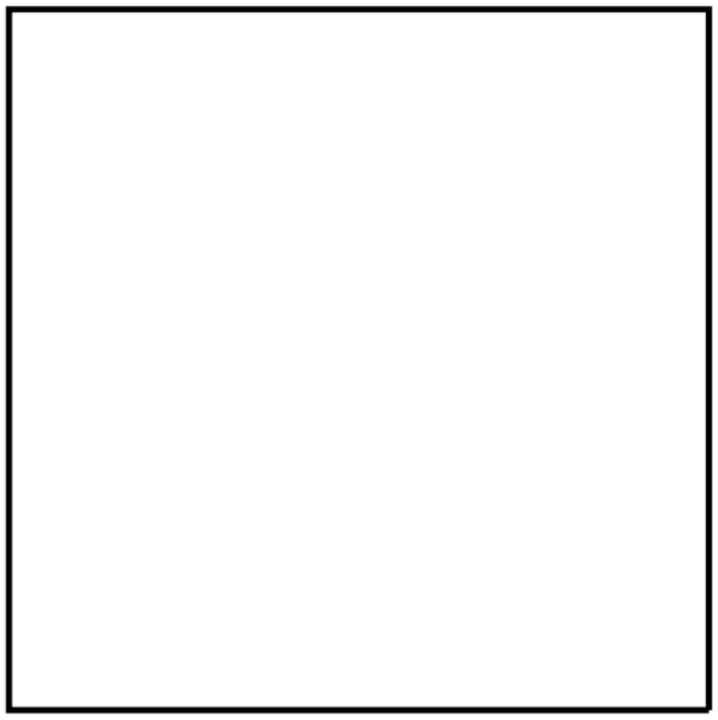
Master Theorem: Example 4



THE UNIVERSITY OF MELBOURNE

$T(n) = 2T(n/2) + n^2$

a = 2, b = 2, d = 2



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

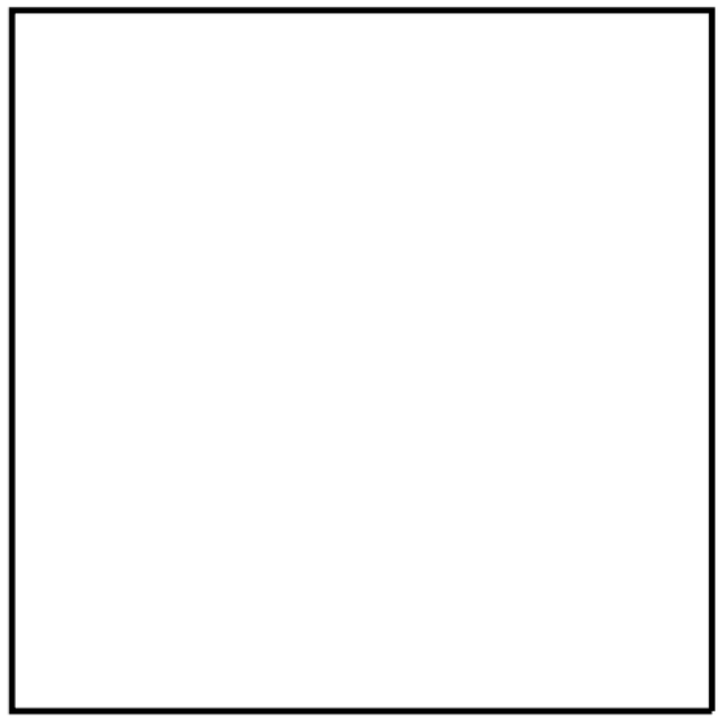
Master Theorem: Example 4



THE UNIVERSITY OF MELBOURNE

$T(n) = 2T(n/2) + n^2$ a = 2, b = 2, d = 2

$T(n) = 2(2T(n/4) + (n/2)^2) + n^2$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

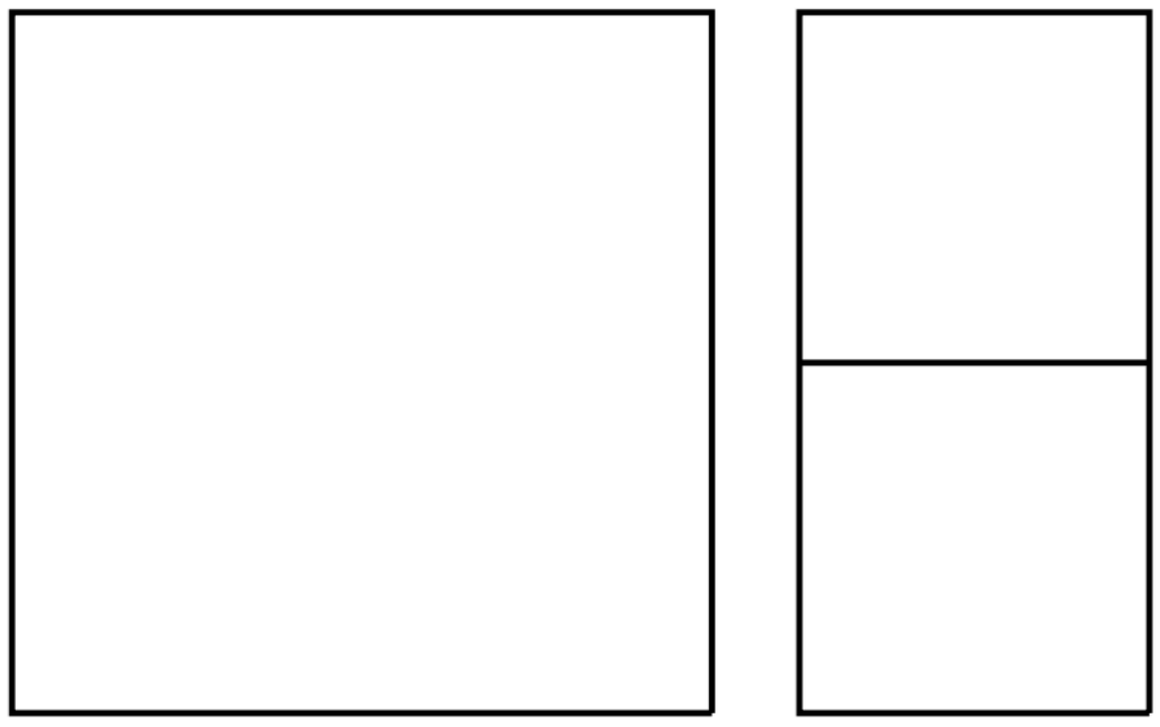
Master Theorem: Example 4



THE UNIVERSITY OF MELBOURNE

$T(n) = 2T(n/2) + n^2$ a = 2, b = 2, d = 2

$T(n) = 4T(n/4) + 2(n/2)^2 + n^2$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

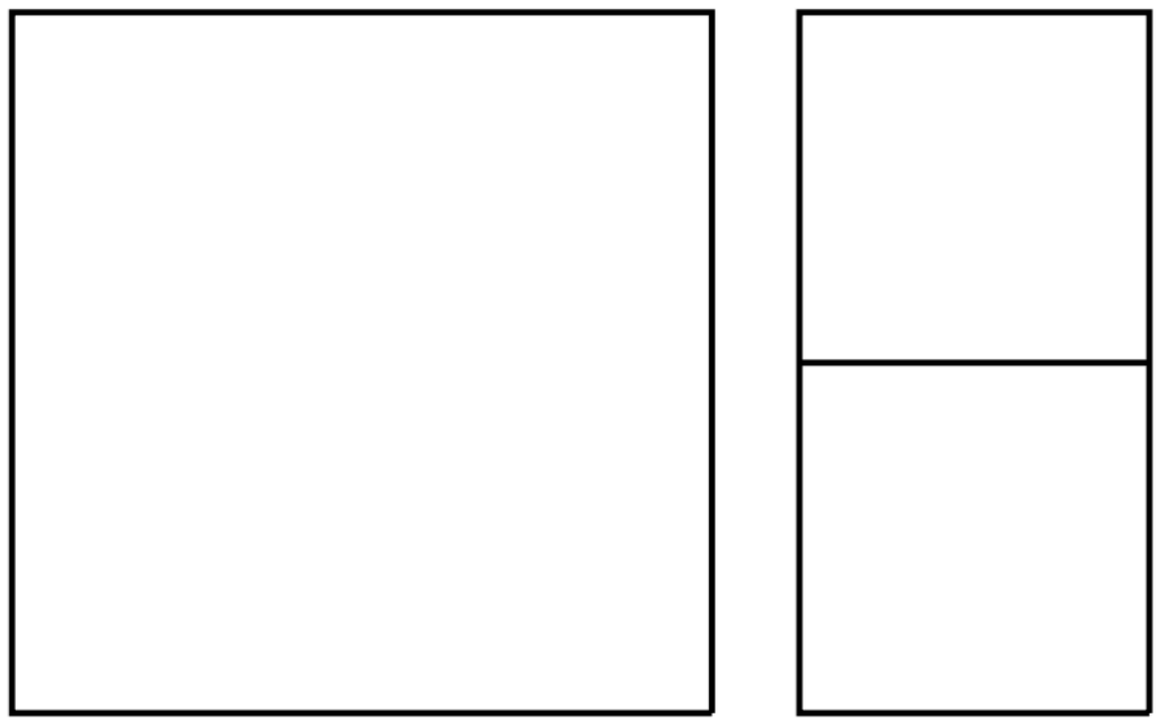
Master Theorem: Example 4



THE UNIVERSITY OF MELBOURNE

$T(n) = 2T(n/2) + n^2$ a = 2, b = 2, d = 2

$T(n) = 4(2T(n/8) + (n/4)^2) + 2(n/2)^2 + n^2$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

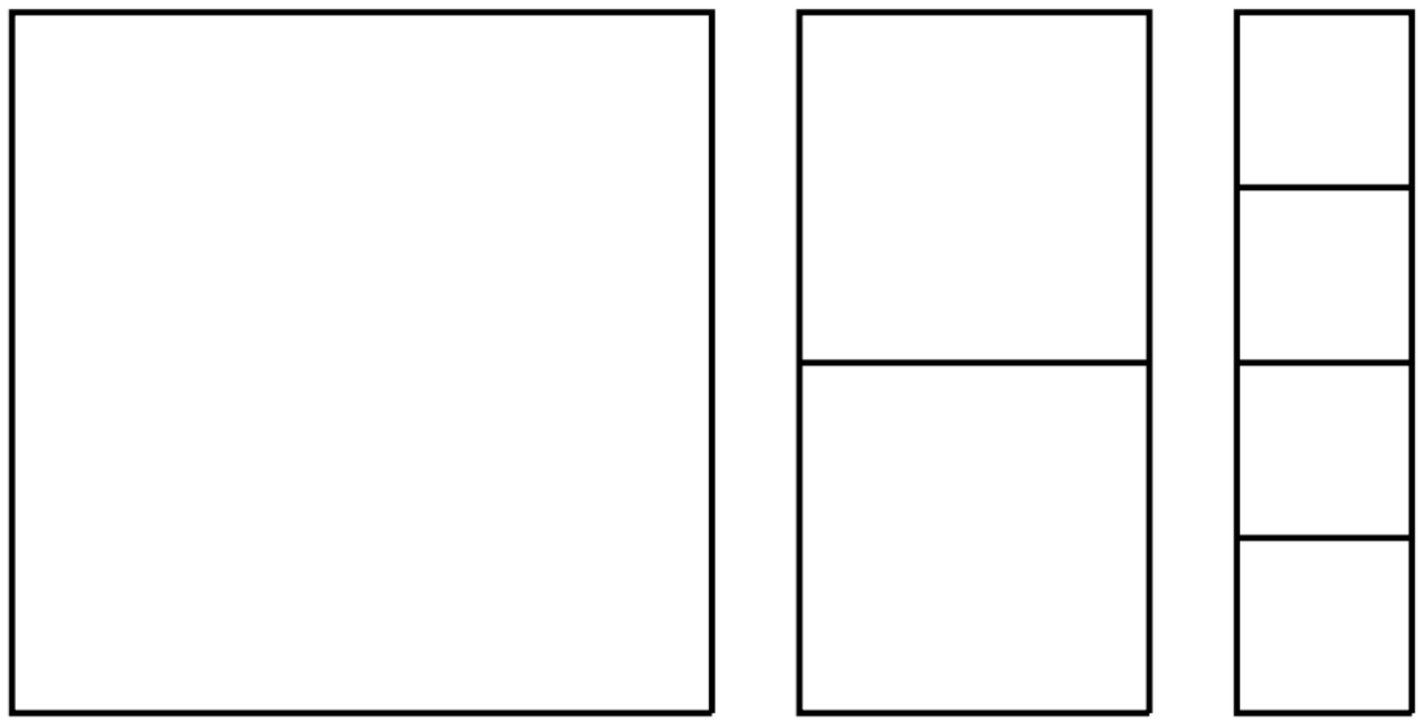
Master Theorem: Example 4



THE UNIVERSITY OF MELBOURNE

$T(n) = 2T(n/2) + n^2$ a = 2, b = 2, d = 2

$T(n) = 8T(n/8) + 4(n/4)^2 + 2(n/2)^2 + n^2$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

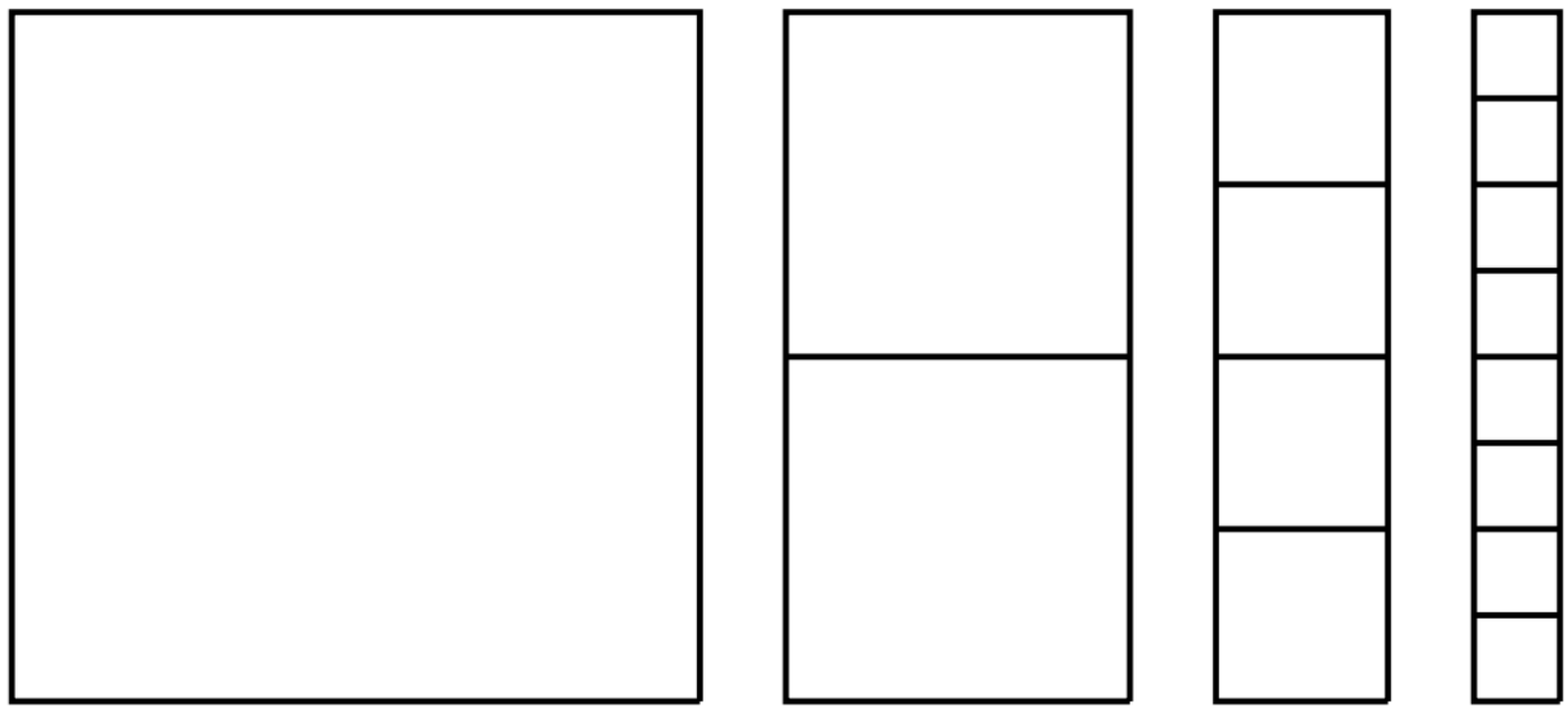
Master Theorem: Example 4



THE UNIVERSITY OF MELBOURNE

$T(n) = 2T(n/2) + n^2$ a = 2, b = 2, d = 2

$T(n) = 8T(n/8) + 4(n/4)^2 + 2(n/2)^2 + n^2$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Master Theorem: Example 4

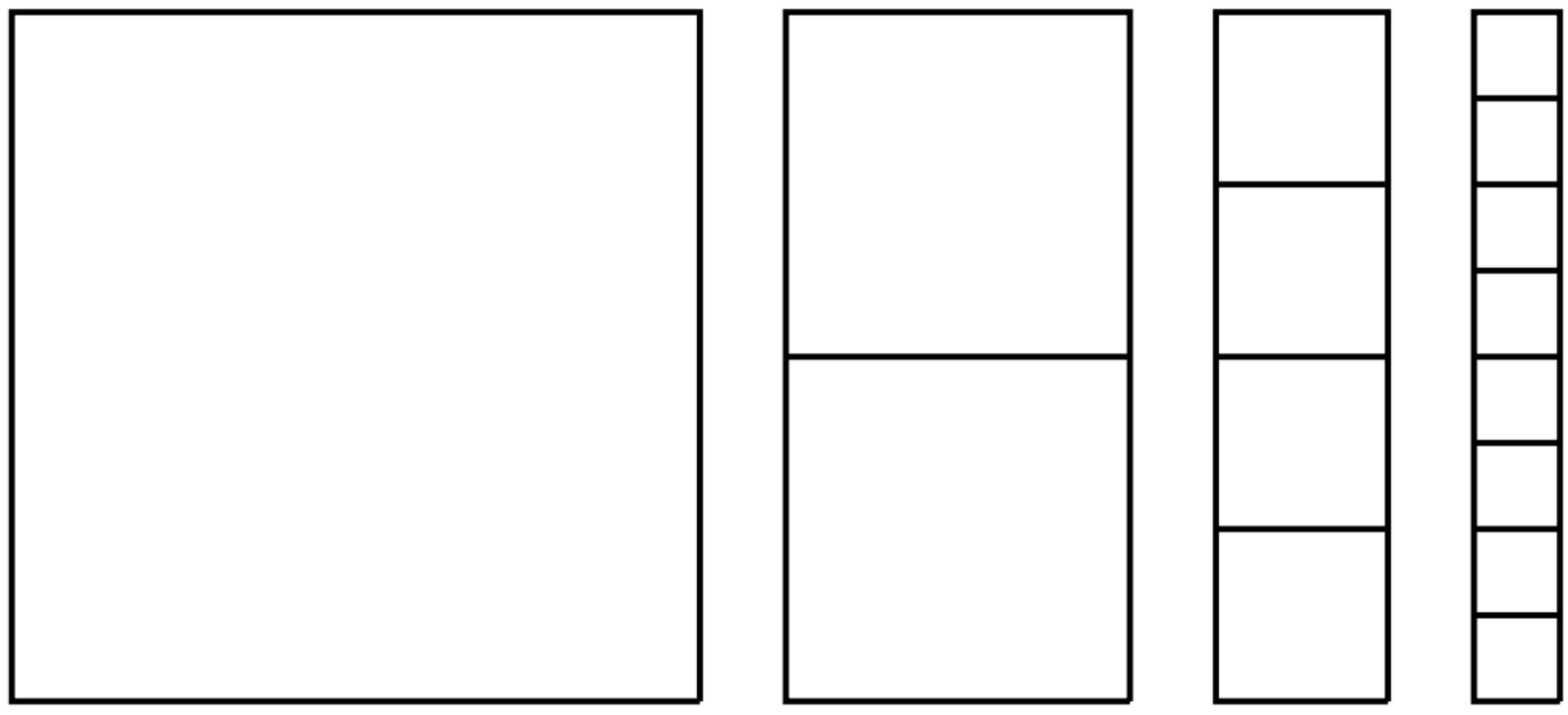


THE UNIVERSITY OF MELBOURNE

$T(n) = 2T(n/2) + n^2$ a = 2, b = 2, d = 2

$T(n) = 8T(n/8) + 4(n/4)^2 + 2(n/2)^2 + n^2$

$T(n) \in \Theta(n^2)$



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Quicksort Improvements: Median-of-Three

- In fact, with median-of-three, we can have a much faster version than before, simplifying tests in the innermost loops:

function PARTITION($A[\cdot], lo, hi$)
  $p \leftarrow A[lo]$; $i \leftarrow lo$; $j \leftarrow hi + 1$
  repeat
    ~~**while** $i < hi$ and $A[i] \leq p$ **do** $i \leftarrow i + 1$~~
    repeat $i \leftarrow i + 1$ **until** $A[i] \geq p$
    ~~**while** $j \geq lo$ and $A[j] > p$ **do** $j \leftarrow j - 1$~~
    repeat $j \leftarrow j - 1$ **until** $A[j] \leq p$
    $swap(A[i], A[j])$
  until $i \geq j$
  $swap(A[i], A[j])$
  $swap(A[lo], A[j])$
  return j

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License